

UNIVERSITÀ DI PISA

Corso di Laurea Magistrale in Informatica Umanistica

ItalianVRExperience: un'esperienza di realtà virtuale immersiva per l'acquisizione situata dell'italiano L2

Progettazione, sviluppo e valutazione sperimentale di un sillabo di sopravvivenza A1: dialogo simulato, tutoraggio grammaticale e riconoscimento vocale al servizio dell'apprendimento linguistico

RELATORE
Prof. Marcello Carrozzino

SECONDO RELATORE
Prof. Mirko Luigi Aurelio Tavosanis

CANDIDATA
Alessia Montalbano

MATRICOLA
565506

A.A. 2025/2026

Abstract

ItalianVRExperience is an immersive virtual reality prototype designed for adult English speakers with no prior knowledge of Italian. Developed in Unity 6 for the Meta Quest 2 headset, the system simulates four situated interaction scenarios: a train station, a residential neighborhood, a bar, and a home environment devoted to exploring descriptive vocabulary. In these settings, language is not the object of study but a necessary tool for orienting oneself in simulated communicative situations. The prototype integrates a controller-free interaction interface based on hand tracking, a Whisper-based speech recognition system, a grammatical and lexical tutor driven by a locally hosted language model (Ollama/Mistral), and an automatic behavioral logging module. The syllabus draws on the A1 survival communicative functions defined by the *Profilo della lingua italiana*, prioritising oral production and situated communicative competence.

The study investigates (1) whether a VR environment centered on spoken production fosters the acquisition of an A1 survival repertoire in Italian, (2) how robustly Whisper handles learners' pronunciation variability, (3) whether automatically logged behavioral data can complement a language post-test as a measure of learning, (4) how participants perceive the system's usability and sense of presence. Fifteen English-speaking absolute beginners took part in a single session of about 30 minutes, in a post-test only design. The data comprises a multiple-choice language post-test, behavioral logs, a perceptual evaluation of the speech transcriptions, and two standardized questionnaires: the System Usability Scale and the Slater-Usoh-Steed. Participants answered 86% of post-test items correctly, with the best results on formulaic expressions tied to situated interaction. Speech recognition succeeded on the first attempt in 62% of cases, but since learners could retry, between 95% and 98% of spoken interaction nodes were completed, with an average of 1.5 attempts per node. The automatic outcome agreed with the perceptual judgement in 81% of attempts; the non-rhotic /r/, one of the most widespread features of participants' pronunciation, was frequently normalised by Whisper as a result of the perceptual evaluation. Usability and presence were rated highly (System Usability Scale mean 86.7/100; presence mean 4.36/5). Overall, the prototype emerges as a promising basis for A1 oral practice, and behavioral logs complement the post-test rather than replace it, yet Whisper sustains the dialogue without being able to replace a human listener for pronunciation feedback. These findings are exploratory.

Indice

1 Introduzione **1**

2 Stato dell'arte **3**
2.1 Storia e contesto: CALL, VR-CALL, DGBLL . . . 3
2.2 Soluzioni esistenti: applicazioni VR e per smartphone . . . 5
2.2.1 Applicazioni in realtà virtuale . . . 5
2.2.2 Applicazioni per smartphone . . . 6
2.3 Il profilo dell'utente target . . . 8
2.3.1 L'adulto come apprendente . . . 9
2.3.2 L'anglofono e l'italiano: una distanza linguistica reale . . . 10
2.4 Fondamenti glottodidattici per la progettazione . . . 10
2.4.1 Competenza comunicativa . . . 11
2.4.2 Input comprensibile, filtro affettivo e VR . . . 11
2.4.3 Scaffolding e Zona di Sviluppo Prossimale . . . 12
2.4.4 Task-Based Language Teaching . . . 12
2.4.5 Embodied cognition e Dual Coding Theory . . . 13
2.4.6 Produzione linguistica e noticing . . . 14
2.4.7 Feedback correttivo ed errorful learning . . . 14
2.4.8 Cognitive Load Theory . . . 15
2.5 L'obiettivo didattico: un sillabo per la sopravvivenza linguistica . . . 16

3 Progettazione del sistema **19**
3.1 Indicazioni metodologiche: Design-Based Research . . . 19
3.2 Il sillabo: dal PLI alla sopravvivenza linguistica . . . 20
3.3 Bellavalle: cornice narrativa . . . 21
3.4 Personaggi non giocanti: funzioni linguistiche associate . . . 22
3.5 Le scene e la progressione didattica . . . 24

4 Implementazione tecnica **33**
4.1 L'ambiente di sviluppo: Unity 6 e Meta Quest 2 . . . 33
4.2 Panoramica sulla produzione dei contenuti . . . 34
4.2.1 Costruzione degli ambienti . . . 34

4.2.2 Animator controller . . . 35
4.3 Il sistema di hand-tracking . . . 36
4.4 Il sistema di dialogo . . . 39
4.4.1 Il riconoscimento vocale: Whisper . . . 40
4.4.2 Il sistema listen-and-repeat . . . 44
4.5 Il tutor AI e l'inventario linguistico . . . 46
4.5.1 Implementazione di Ollama/Mistral . . . 50
4.6 Persistenza dello stato di gioco . . . 51
4.7 Il sistema di logging: VoiceRecognitionLogger . . . 52

5 Studio sperimentale **55**
5.1 Le domande di ricerca . . . 55
5.2 Disegno dello studio . . . 57
5.2.1 Metodologia . . . 57
5.2.2 System Usability Scale . . . 58
5.2.3 Presenza e immersività: questionario Slater-Usoh-Steed . . . 59
5.2.4 Valutazione percettiva delle trascrizioni . . . 60
5.2.5 Analisi dei dati . . . 64
5.3 Risultati . . . 67
RQ1. Efficacia acquisizionale . . . 67
RQ2. Tenuta del riconoscimento vocale . . . 68
RQ3. Dati comportamentali come strumento di misurazione . . . 72
RQ4. Usabilità e senso di presenza . . . 73
5.4 Discussione . . . 74

6 Conclusioni **79**
6.1 Limiti del prototipo . . . 80
6.2 Sviluppi futuri . . . 81
6.3 Considerazioni finali . . . 83

APPENDICE A **85**
Dialoghi e confronto con il PLI . . . 85

APPENDICE B **91**

APPENDICE C **93**

APPENDICE D **133**

APPENDICE E **139**
Griglia di valutazione percettiva delle trascrizioni . . . 139

Bibliografia **145**

1 Introduzione

La città di Bellavalle non esiste realmente. Non compare su alcuna mappa, non ha un codice postale, e di certo non è abitata. Eppure, chi arriva alla stazione di Bellavalle deve capire cosa gli chiede il controllore sul treno, deve orientarsi per capire come arrivare effettivamente in città, e presentarsi alla curiosa vicina di casa che si presenta in una lingua ancora incompresa. Chi arriva a Bellavalle deve, in altre parole, *usare* l'italiano per comunicare, prima ancora di studiarlo.

Da questa premessa nasce *ItalianVRExperience*, un ambiente di realtà virtuale immersiva progettato per parlanti anglofoni adulti privi di qualsiasi conoscenza pregressa della lingua italiana. Nell'ambiente, la lingua non è oggetto di esercizio, ma strumento necessario per orientarsi in situazioni comunicative simulate. Sviluppato in Unity 6 per il visore Meta Quest 2, fondato sulle basi linguistiche del Quadro Comune Europeo di Riferimento e sul *Profilo della lingua italiana* (PLI), il prototipo simula quattro scenari di interazione situata. Tre sono dedicati alla comunicazione sociale: la stazione, dove il giocatore arriva; il quartiere residenziale, dove fa le prime conoscenze; il bar, dove porta a termine una vera e propria transazione. Il quarto, l'abitazione, è dedicato all'esplorazione del lessico descrittivo in ambiente domestico. Alla base del progetto c'è una difficoltà concreta: insegnare l'italiano a un adulto principiante assoluto. La distanza tipologica dall'inglese è reale (morfologica, fonologica, prosodica) e si somma a un profilo motivazionale fragile per definizione. Come esplorato nel corso del §2, l'adulto impara finché percepisce il bisogno, e smette prima di aver raggiunto una competenza stabile. Le soluzioni tecnologiche disponibili hanno delle lacune, dovute al fatto che non affrontano questo problema in modo integrato: o ignorano la produzione orale, o il contesto situato, o persino il profilo dell'apprendente. *ItalianVRExperience* nasce dal tentativo di rispondere a queste mancanze, in un unico ambiente di realtà virtuale, integrando un sistema di dialogo situato, il riconoscimento vocale (Whisper), e un tutor grammaticale e lessicale attivabile su richiesta basato su intelligenza artificiale (Ollama/Mistral), capace di rispondere in inglese a domande sulla lingua incontrata in scena.

Il lavoro si articola attorno a quattro domande di ricerca. La prima riguarda l'efficacia acquisizionale del sistema *in toto*: un ambiente VR che privilegia la produzione orale come modalità primaria di interazione favorisce il riconoscimento di un repertorio

di sopravvivenza in italiano A1 da parte di parlanti anglofoni principianti assoluti? La seconda riguarda specificamente la tenuta tecnica del sistema di riconoscimento vocale Whisper, implementato a supporto della produzione orale. Whisper, in modalità di parlato controllato su frasi prestabilite e in dialogo situato, regge alla variabilità di pronuncia dei partecipanti in misura sufficiente a supportare l'interazione? La terza riguarda la misurazione dell'apprendimento: i dati comportamentali raccolti automaticamente durante il gioco costituiscono uno strumento adeguato, comparabile o complementare rispetto al post-test linguistico somministrato al termine della sessione? La quarta domanda, trasversale alle precedenti, riguarda l'usabilità percepita del sistema e il senso di presenza nell'ambiente virtuale. Si tratta di una condizione di interpretabilità più che di un obiettivo in sé: un punteggio basso al *post-test* o un'alta frequenza di mancati riconoscimenti vocali potrebbero dipendere tanto dall'inefficacia dell'impianto didattico quanto da un'interfaccia faticosa o da un'immersione insufficiente. Senza un dato minimo su queste due dimensioni, il risultato resterebbe aperto a due letture opposte: il sistema non insegna, oppure non si lascia usare. Dato il campione ridotto e il disegno di solo *post-test*, i risultati di tutte e quattro le domande hanno valore esplorativo: indicano direzioni promettenti e punti critici del prototipo, senza pretesa di stabilirne l'efficacia in senso causale.

I capitoli che seguono sviluppano questa proposta in ordine: dal contesto alle scelte, dalla teoria all'implementazione, dalla progettazione alla verifica empirica. Il capitolo 2 colloca il progetto nel suo campo di riferimento, esamina criticamente le soluzioni esistenti, definisce il profilo dell'utente target e ricostruisce i fondamenti glottodidattici che orientano le scelte progettuali. Il capitolo 3 documenta quelle scelte: i criteri di selezione del sillabo, la struttura narrativa di Bellavalle, la progettazione dei personaggi non giocanti e la progressione didattica per scena. Il capitolo 4 descrive l'architettura tecnica del sistema: il motore di sviluppo, l'*hand tracking*, il sistema di dialogo basato su Whisper e il tutor grammaticale e lessicale, con il suo sistema di tracciamento contestuale delle parole incontrate in scena. Il capitolo 5 presenta lo studio sperimentale: le domande di ricerca, il disegno dello studio, i risultati e la loro discussione. Nel capitolo 6 verranno esplorate le conclusioni, in vista di ulteriori sviluppi del prototipo VR-CALL.

2 Stato dell'arte

Questo capitolo colloca il lavoro nel suo campo di riferimento, fornendo le coordinate necessarie a comprendere le scelte alla base dell'impostazione della ricerca. Si parte dal tracciare una traiettoria storica della CALL, dell'arrivo della realtà virtuale nell'apprendimento linguistico, e più specificamente delle sottocorrenti DGBLL e VR-CALL, che vale la pena definire per la comprensione globale dello stato dell'arte (§2.1). Si procede con una rassegna critica delle soluzioni esistenti (§2.2), sia per la realtà virtuale che per smartphone. Il capitolo continua con la definizione del profilo dell'utente target: l'apprendente adulto anglofono principiante assoluto, le sue caratteristiche motivazionali e la distanza tipologica tra la sua lingua madre e l'italiano (§2.3). Il §2.4 tratta dei concetti teorici a fondamento della progettazione stessa del prototipo: dalla *Second Language Acquisition* all'*embodied cognition*, dalla *Cognitive Load Theory* al *feedback* correttivo. Il §2.5, infine, definisce l'obiettivo didattico che ne deriva: un sillabo di sopravvivenza fondato sul linguaggio formulaico.

2.1 Storia e contesto: CALL, VR-CALL, DGBLL

L'uso dei computer nell'insegnamento delle lingue ha radici più profonde di quanto sembri. L'uso dei primi programmi a scopi didattici risale agli anni Sessanta, quando i cosiddetti *drill-and-practice* venivano impiegati per esercizi ripetitivi di tipo comportamentista, fondati sull'idea che l'apprendimento consistesse in un cambiamento osservabile del comportamento prodotto da stimoli esterni.[1] In questa fase, la *CALL comportamentista*, Warschauer e Healey (1998) hanno individuato il primo di tre momenti storici distinti, ciascuno corrispondente a un paradigma tecnologico e pedagogico preciso. Il secondo, definito *CALL comunicativa*, emerge tra la fine degli anni Settanta e i primi anni Ottanta: il *personal computer* apre nuove possibilità di lavoro individuale e autonomo, e l'attenzione si sposta dal comportamento del discente ai suoi processi cognitivi interni, con un approccio che valorizza scoperta, espressione e sviluppo personale.[2] Il terzo momento, descritto come *CALL integrativa*, prende forma negli anni Novanta: le critiche all'uso frammentato e decontestualizzato del computer portano verso approcci

[1] Mustafa Polat, "CALL in Context: A Brief Historical and Theoretical Perspective", *Issues and Trends in Educational Technology* 5, no. 1 (2017): pp. 17–18, citando Warschauer e Healey (1998).

[2] Polat, op. cit., pp. 18–19.

costruttivisti, *task-based* e *content-based*, che cercano, dunque, di integrare le diverse abilità linguistiche in ambienti autentici di apprendimento, avvicinando progressivamente la tecnologia alla comunicazione reale.[3]

In questo contesto di crescente integrazione si inserisce la realtà virtuale. Il termine *virtual reality* viene coniato nel 1989 da Jaron Lanier, fondatore della compagnia *VPL Research*, una delle prime *startup* a vendere software e hardware per la realtà virtuale, e dedita alla raccolta, sotto un'unica etichetta, di una serie di progetti che condividevano l'obiettivo di creare ambienti tridimensionali simulati accessibili tramite visori e guanti sensoriali.[4] Steuer propone di superare questa definizione puramente hardware, spostando il fuoco dall'apparato tecnologico all'esperienza del percettore: la realtà virtuale diventa così la condizione in cui un individuo percepisce di essere presente in un ambiente mediato, fenomeno che Steuer chiama *telepresenza*.[5] Due dimensioni tecnologiche determinano il grado di questa presenza: la *vividness*, ovvero la ricchezza sensoriale dell'ambiente simulato, e l'*interactivity*, ovvero la capacità dell'utente di influenzare forma e contenuto di quell'ambiente in tempo reale.[6]

All'interno della CALL si sono così sviluppati due sottofiloni, che portano questa logica integrativa alle sue conseguenze più radicali. Il primo è il *digital game-based language learning* (DGBLL), che studia l'uso dei giochi digitali come ambienti per l'acquisizione linguistica. Cornillie, Thorne e Desmet distinguono, all'interno del DGBLL, tra *commercial off-the-shelf games* (COTS), giochi commerciali capaci di supportare l'apprendimento linguistico in modo incidentale, e *synthetic immersive environments* (SIEs), ambienti immersivi progettati esplicitamente per obiettivi linguistici specifici, con strategie di valutazione integrate nella struttura stessa del gioco.[7] Il secondo sottofilone è la VR-CALL, che studia l'uso della realtà virtuale nell'apprendimento linguistico. La VR-CALL si distingue dalle forme tradizionali di CALL per il grado di immersione offerto: visori *head-mounted* collocano il discente in un ambiente tridimensionale a 360 gradi, con interazione gestuale diretta.[8] La (più recente) revisione sistematica di Çakır su 46 studi pubblicati tra il 2010 e il 2023 documenta una crescita consistente dell'interesse per questa area di confluenza, con effetti positivi dal punto di vista didattico, in particolare su motivazione, coinvolgimento, acquisizione lessicale, competenza orale e pronuncia degli apprendenti.[9] Un ambiente immersivo colloca il discente dentro la

[3] Ibid., p. 19.

[4] Jonathan Steuer, "Defining Virtual Reality: Dimensions Determining Telepresence", *Journal of Communication* 42, no. 4 (1992): p. 73.

[5] Ibid., pp. 75–76.

[6] Ibid., pp. 80–81.

[7] Frederik Cornillie, Steven L. Thorne e Piet Desmet, Digital Games for Language Learning: From Hype to Insight?", *ReCALL* 24, no. 3 (2012): pp. 246–247.

[8] Regina Kaplan-Rakowski e Alice Gruber, Low-Immersion versus High-Immersion Virtual Reality: Definitions, Classification, and Examples with a Foreign Language Focus", in *Proceedings of the Innovation in Language Learning International Conference 2019* (Firenze: Pixel, 2019): p. 552.

[9] Nilay Çakır, "Virtual Reality Games in Foreign Language Education: A Systematic Review", *Language Education and Technology (LET Journal)* 4, no. 1 (2024): pp. 26–29.

situazione comunicativa, che si ritrova ad essere una presenza incarnata in un contesto simulato; le applicazioni per smartphone e i materiali didattici, più tradizionali, non sono strutturalmente in grado di offrire un'esperienza simile.

2.2 Soluzioni esistenti: applicazioni VR e per smartphone

La sezione offre una panoramica delle soluzioni esistenti per l'apprendimento linguistico mediato dalla tecnologia, che risultano numerose ed eterogenee. La ricognizione che segue esamina un campione rappresentativo di applicazioni in realtà virtuale e per smartphone, selezionate in base alla diffusione e alla rilevanza rispetto alla letteratura di riferimento, con l'obiettivo di identificarne punti di forza e limiti rispetto alle dimensioni di *vividness*, interattività e produzione orale discusse al §2.1. Tale ricognizione non segue un protocollo sistematico e i suoi limiti metodologici sono dichiarati al §2.2.2.

2.2.1 Applicazioni in realtà virtuale

Language Lab VR[10] propone l'apprendimento del lessico attraverso il collocamento delle parole in un contesto visivo tridimensionale: gli oggetti dell'ambiente sono etichettati con il loro nome nella lingua target, e il discente interagisce con essi per memorizzarne il significato. L'applicazione non offre alcuno spazio per l'esercizio del parlato né per la riflessione grammaticale; nella versione esaminata sono inoltre emerse associazioni lessicali prive di coerenza logica e un modello di pronuncia non sempre corretto.

Mondly VR[11] è orientata alla pratica del parlato tramite roleplay in ambiente immersivo, con risposte gestite attraverso un sistema a opzioni multiple. L'esperienza è articolata in quattro modalità: *immersive conversations*, *immersive vocabulary*, *extensive learning* e *daily lesson*. Nella versione esaminata, la progressione è interamente affidata all'intuizione dell'utente: non è presente alcuna spiegazione grammaticale esplicita né alcuno strumento di supporto metalinguistico.

Noun Town[12] adotta un meccanismo di ripetizione dilazionata (*spaced repetition*) che ripropone la stessa parola fino alla sua memorizzazione, collocando il lessico in ambienti tematici specifici. Il discente esplora un mondo tridimensionale, interagisce con personaggi non giocanti tramite riconoscimento vocale e completa mini-giochi progettati per consolidare vocabolario specifico per contesto. Nella versione italiana esaminata, tuttavia, la resa linguistica risulta poco curata sia sul piano fonologico sia su quello

[10] Language Lab VR, Meta Quest Store, https://www.meta.com/experiences/language-lab/3802077353226534/ (consultato il 28/07/2026).

[11] Mondly VR, Meta Quest Store, https://www.meta.com/experiences/mondly-learn-languages-in-vr/4214902388537196/ (consultato il 28/07/2026).

[12] Noun Town, sito ufficiale, https://noun.town/ (consultato il 28/07/2026).

lessicale, e la produzione orale richiesta al discente resta limitata alla pronuncia di singoli termini o di brevi formule, senza evolvere in uno scambio dialogico strutturato.

ImmerseMe[13] si distingue per l'uso di video reali con parlanti nativi e per una struttura a bivi narrativi in cui ogni risposta condiziona lo sviluppo delle interazioni successive. L'applicazione è articolata in cinque modalità: *pronunciation*, *typing*, *spelling*, *translation* e *immersion mode*. Nella versione italiana esaminata, le conversazioni proposte risultano statiche una volta esaurite le ramificazioni previste, e la dimensione sociale dell'interazione è assente: il discente interagisce con personaggi video preregistrati, senza che questi reagiscano alle sue azioni nell'ambiente.

Immerse[14] fonda l'esperienza su sessioni condivise con altri utenti e insegnanti in tempo reale, integrate da lezioni guidate da intelligenza artificiale e da una comunità globale di apprendenti. L'applicazione è disponibile in inglese, spagnolo, portoghese e francese, e si rivolge prevalentemente a un'utenza con obiettivi professionali o accademici. Il modello presuppone la disponibilità simultanea di altri utenti o istruttori, condizione che lo distingue strutturalmente dalle soluzioni per l'apprendimento individuale e autonomo.

Da questa ricognizione emerge un quadro in cui le funzionalità rilevanti per un apprendimento linguistico fondato sulla produzione orale situata risultano distribuite su prodotti diversi, ciascuno dei quali soddisfa alcuni criteri e ne trascura altri. Le lacune ricorrenti sono almeno quattro. La prima riguarda la produzione orale: quando è prevista, resta vincolata a risposte a scelta multipla o a percorsi predefiniti, e raramente il parlato del discente produce conseguenze reali nell'ambiente. La seconda riguarda il supporto metalinguistico: la progressione è spesso affidata all'intuizione dell'utente, senza strumenti che accompagnino la riflessione sulla lingua. La terza riguarda la qualità dell'input in italiano: nelle versioni esaminate emergono imprecisioni fonologiche e lessicali che ne compromettono l'affidabilità come modello linguistico. La quarta riguarda la reattività dell'ambiente: le interazioni tendono a esaurirsi in sequenze statiche oppure dipendono dalla presenza simultanea di altri utenti, a scapito dell'apprendimento individuale e autonomo. Sulla prima di queste lacune la letteratura offre una conferma esplicita: come documentato da Forero Pataquiva e Klimova, gli studi VR esistenti risultano «limited to the practice of input-related skills (listening and reading) since there is no way to speak or write in those experiments».[15]

2.2.2 Applicazioni per smartphone

Accanto alle soluzioni in realtà virtuale, il panorama delle applicazioni per smartphone resta il termine di confronto più immediato per un pubblico di apprendenti adulti. Va

[13] ImmerseMe, sito ufficiale, https://immerseme.co/ (consultato il 28/07/2026).

[14] Immerse, Meta Quest Store, https://www.meta.com/experiences/immerse-build-real-language-fluency/3181432891940455/ (consultato il 28/07/2026).

[15] Francisco De Paula Forero Pataquiva e Blanka Klimova, "Systematic Review of Virtual Reality in the Acquisition of Second Language", *International Journal of Emerging Technologies in Learning (iJET)* 17, no. 15 (2022): p. 50.

segnalato che, a differenza del dominio delle app mobile, la VR-CALL non dispone ancora di un *framework* valutativo condiviso: come esplicitato da Forero Pataquiva e Klimova, «this lack of standardization makes comparisons challenging to establish».[16] Le applicazioni sono analizzate sulla base di cinque criteri, ricavati e adattati dalla distinzione tra game-based language learning autentico e gamification superficiale elaborata da Deterding[17] e applicata al contesto CALL da Cornillie, Thorne e Desmet:[18].
I criteri adottati sono:

- ancoraggio a un *framework* linguistico riconosciuto;
- presenza di produzione orale integrata e verificata;
- qualità del *feedback* correttivo;
- grado di immersività e *situatedness* del contesto;
- autenticità dell'interazione linguistica.

Criterio	**Duolingo**	**Babbel**
Sillabo linguistico	Tematico (famiglia, cibo, viaggi); nessun ancoraggio esplicito al QCER	Strutturato per livelli; maggiore attenzione alla grammatica esplicita
Produzione orale	Presente ma marginale; prevalenza di traduzione e abilità ricettive	Presente in alcune modalità; non verificata automaticamente in modo sistematico
Feedback correttivo	Risposta corretta/errata con commento breve; assenza di spiegazioni grammaticali	Correzione esplicita con spiegazione; più orientato alla forma
Immersività	Nulla: interfaccia a schermo piatto, nessun contesto situato	Nulla: interfaccia a schermo piatto, nessun contesto situato
GBLL autentico vs. gamification	Gamification superficiale: punti, streak, badge, classifiche	Gamification limitata; maggiore orientamento didattico, minore componente ludica

Tabella 1: Confronto tra Duolingo e Babbel rispetto alle principali caratteristiche didattiche

Duolingo è l'applicazione mobile per l'apprendimento linguistico più diffusa al mondo, con oltre 300 milioni di utenti.[19] La revisione sistematica di Shortt et al. (2021), condotta

[16]Ibid., p. 51.
[17]Sebastian Deterding, Gamification: Designing for Motivation", *Interactions* 19, no. 4 (2012): p. 14, DOI: 10.1145/2212877.2212883.
[18]Frederik Cornillie, Steven L. Thorne e Piet Desmet, Digital Games for Language Learning: From Hype to Insight?", *ReCALL* 24, no. 3 (2012): pp. 243–256, DOI: 10.1017/S0958344012000134.
[19]Mitchell Shortt et al., "Gamification in Mobile-Assisted Language Learning: a Systematic Review of

su 35 studi pubblicati tra il 2012 e il 2020, offre il quadro critico più completo disponibile in letteratura. Le attività proposte si basano prevalentemente su approcci comportamentisti, «without taking meaning or content into consideration»,[20] e privilegiano le abilità ricettive a scapito di quelle produttive. Gli utenti stessi lamentano un'eccessiva dipendenza dalla traduzione e dalla ripetizione, e difficoltà nel trasferire le competenze acquisite a situazioni reali di comunicazione.[21] La componente di gamification (streak, lingot, classifiche) risulta motivante nel breve periodo, ma non compensa l'assenza di interazione linguistica autentica né di *feedback* grammaticale strutturato.

Babbel condivide il formato a schermo piatto e l'assenza di contestualizzazione situata, ma si distingue per una maggiore attenzione alla grammatica esplicita e per un sillabo più strutturato, sebbene anch'esso privo di ancoraggio dichiarato al QCER. Come nota la revisione di Shortt et al., Babbel e Duolingo offrono esperienze «somewhat similar», con Duolingo significativamente più diffuso grazie al modello gratuito.[22]

Dall'analisi complessiva emerge un quadro frammentato, in quanto le funzionalità rilevanti per un apprendimento linguistico fondato sulla produzione orale situata risultano distribuite su prodotti diversi, ciascuno dei quali soddisfa alcuni criteri e ne trascura altri. Questa lacuna vale tanto per le app su smartphone quanto per le soluzioni in realtà virtuale, e costituisce il punto di partenza da cui muove il progetto.

2.3 Il profilo dell'utente target

Definire con precisione il profilo dell'utente target non è un'operazione preliminare né accessoria: è il passaggio che ancora le scelte didattiche a una progettazione linguistica rigorosa. L'artefatto si rivolge a un adulto anglofono privo di qualsiasi conoscenza pregressa dell'italiano, il cui obiettivo è riuscire a comunicare in contesti di vita quotidiana. L'adulto è in questa sede caratterizzato come *viaggiatore*, anticipando la linea teorica di Nation e Crabbe discussa al §2.5, in quanto corrisponde al profilo *goal-oriented* che Knowles individua come tipico di un apprendente che usa l'educazione come strumento per raggiungere obiettivi concreti e immediati.[23] Il §2.3.1 analizza le caratteristiche motivazionali di questo profilo. Il §2.3.2 analizza la distanza tipologica tra inglese e italiano e le sue implicazioni per l'apprendimento.

Duolingo Literature from Public Release of 2012 to Early 2020", *Computer Assisted Language Learning* (2021), DOI: 10.1080/09588221.2021.1933540, p. 2.

[20] Shortt et al., op. cit., p. 14.

[21] Ibid., p. 21.

[22] Ibid., p. 5.

[23] Malcolm S. Knowles, *The Modern Practice of Adult Education: From Pedagogy to Andragogy*, ed. riveduta e aggiornata (New York: The Adult Education Company, 1980), p. 42.

2.3.1 L'adulto come apprendente

L'assunzione per cui l'educazione non si limita alla trasmissione di conoscenze, ma rappresenta un processo che accompagna l'individuo lungo l'intera vita, è alla base della teoria dell'andragogia, concetto sviluppato a partire dal modello proposto da Knowles.[24] Termine coniato in analogia con la pedagogia ma riferito al discente adulto,[25] l'andragogia muove dal presupposto che l'apprendente adulto si distingua dal discente in età scolare per almeno quattro tratti. Il primo è il bisogno di comprendere, prima di impegnarsi nello studio di un contenuto, *perché* valga la pena apprenderlo. Il secondo è la *self-conception* dell'adulto come soggetto autonomo e autodiretto, insofferente rispetto a dinamiche percepite come infantilizzanti o punitive. Il terzo è il ricorso all'esperienza pregressa come risorsa per l'apprendimento: di qui l'efficacia di scenari simulati riconducibili a situazioni di vita reale, rispetto a esercizi decontestualizzati. Il quarto è un orientamento centrato sul problema e sul compito piuttosto che sull'accumulo di contenuti teorici: Knowles osserva che «*individuals are motivated to engage in learning to the extent that they feel a need to learn and perceive a personal goal that learning will help to achieve*».[26]

Secondo il modello tripolare della motivazione, elaborato da Balboni, l'energia che spinge un individuo ad apprendere una lingua viene ricondotta a tre motori fondamentali, ciascuno con effetti distinti sull'acquisizione. Il *dovere*, motivazione dominante nelle situazioni didattiche tradizionali, non produce acquisizione duratura: «inserisce un filtro affettivo che fa restare nella memoria a breve termine le informazioni apprese»,[27] le quali tendono a svanire una volta superata la verifica. Per l'apprendente adulto, la spinta per eccellenza è il *bisogno*: una motivazione «legata primariamente all'emisfero sinistro del cervello, quello razionale e consapevole»,[28] di natura pragmatica e strumentale, quasi parafrasi di ciò che Knowles identifica come l'orientamento verso obiettivi concreti e immediatamente applicabili. Il *piacere* è invece «motivazione essenzialmente legata all'emisfero destro»,[29] alimentata dall'attrattiva culturale, dalla sfida intellettuale e dal coinvolgimento affettivo. Il bisogno presenta però limiti strutturali che Balboni descrive con precisione: «è necessario che il bisogno sia percepito», condizione tutt'altro che scontata; esso «funziona fino a quando lo studente decide che ha soddisfatto il suo bisogno —punto che si colloca di solito ben al di sotto del livello soglia».[30] Una volta che il discente adulto si sente in grado di farsi capire nel contesto quotidiano, la spinta razionale tende a esaurirsi prematuramente, rendendo necessario che la didattica faccia da ponte verso il piacere di apprendere.

---

[24] Knowles, *op. cit.*, pp. 40–43.

[25] Il termine è costruito sul greco *anér* (tema *andr-*), «man, not boy», in parallelo con *pedagogy*, derivato da *paid* («child») e *agogus* («leading»): cfr. Knowles, *op. cit.*, p. 42.

[26] Knowles, *op. cit.*, p. 56.

[27] Paolo E. Balboni, *Le sfide di Babele. Insegnare le lingue nelle società complesse*, nuova ed. (Torino: UTET Università, 2008), p. 38.

[28] Ibid.

[29] Ibid.

[30] Ibid.

Questo limite motivazionale è riscontrato anche nella letteratura sul MALL (*Mobile-Assisted Language Learning*). La revisione sistematica di Shortt et al. su Duolingo, presentata al §2.2, mostra che la maggior parte degli studi disponibili si concentra sul design dell'applicazione piuttosto che sugli esiti dell'apprendimento, lasciando in larga parte non dimostrato il passaggio dall'*engagement* superficiale allo sviluppo di una competenza comunicativa fondata su necessità reali, il che è un limite particolarmente acuto per un pubblico adulto orientato agli strumenti e non all'intrattenimento.

2.3.2 L'anglofono e l'italiano: una distanza linguistica reale

L'inglese e l'italiano sono lingue tipologicamente distanti, e questa distanza produce conseguenze concrete sull'apprendimento. Sul piano **morfologico**, una delle principali insidie per l'apprendente anglofono è il sistema di allocuzione formale: il *Lei*, privo di qualsiasi equivalente in inglese, è una delle aree di errore più sistematiche per questo profilo di apprendente. Sul piano **fonologico**, l'italiano presenta un inventario vocalico più ricco e regolare dell'inglese, con distinzioni vocaliche e consonantiche che l'anglofono tende a non percepire né produrre nella maniera corretta durante le prime fasi di acquisizione. Graham e Roll, analizzando le performance di Whisper su parlanti con diverse L1, documentano che un inventario vocalico ridotto nella lingua madre è associato a un tasso di errore significativamente più alto nei sistemi di *Automatic Speech Recognition*.[31] Sul piano **prosodico**, l'italiano ha un ritmo sillabico mentre l'inglese ha un ritmo accentuale: una differenza strutturale che produce un accento straniero immediatamente riconoscibile, e che gli stessi autori collegano a variazioni significative nelle performance dei sistemi di riconoscimento vocale in funzione della tipologia prosodica della L1 del parlante.[32]

A fronte di queste difficoltà prevedibili, l'anglofono porta con sé un vantaggio non trascurabile: la presenza di un ampio strato di lessico latino riconoscibile per trasparenza. A questo vantaggio si affianca però un limite altrettanto strutturale: chi ascolta l'italiano per la prima volta non dispone di alcun ancoraggio fonologico su cui appoggiarsi per segmentare il parlato e attribuire significato a ciò che ascolta. La curva di acquisizione fonologica dell'anglofono risulta più lenta di quella di un apprendente con L1 romanza, che può appoggiarsi a un sistema vocalico e prosodico più vicino all'italiano.

2.4 Fondamenti glottodidattici per la progettazione

La progettazione di un ambiente di apprendimento linguistico in realtà virtuale non può prescindere da un quadro teorico solido. I costrutti che seguono sono stati selezionati per rilevanza rispetto alle scelte implementative documentate nel capitolo 3: ciascuno orienta una o più decisioni progettuali specifiche, e il loro modo di tradursi in architettura

[31] Calbert Graham e Nathan Roll, "Evaluating OpenAI's Whisper ASR: Performance Analysis across Diverse Accents and Speaker Traits", *JASA Express Letters*, vol. 4, n. 2 (2024), p. 025206-4.

[32] Ibid.

didattica è documentato in dettaglio nelle sezioni §3.2, §3.3, §3.4 e §3.5. Le sottosezioni che seguono presentano ciascun concetto nella sua formulazione teorica; il lettore troverà le relative applicazioni progettuali nel capitolo successivo.

2.4.1 Competenza comunicativa

L'idea che la competenza linguistica non si riduca alla correttezza grammaticale è oggi così consolidata da sembrare ovvia. Non lo era affatto quando Canale e Swain la formularono nel 1980, in un articolo destinato a ridefinire il modo in cui la ricerca e la didattica pensavano all'obiettivo dell'apprendimento linguistico. Il loro modello, nella versione ampliata da Canale nel 1983,[33] articola la competenza comunicativa in quattro dimensioni distinte: quella grammaticale, che riguarda la padronanza del sistema formale della lingua; quella sociolinguistica, che regola l'appropriatezza degli enunciati rispetto al contesto d'uso; quella discorsiva, che governa coerenza e coesione del testo; e quella strategica, che consente di compensare le lacune nelle altre tre attraverso strategie di comunicazione.[34] È un modello che il QCER eredita e articola ulteriormente nei suoi descrittori per livello, e che il Profilo della Lingua Italiana declina in inventari di funzioni comunicative, atti linguistici e repertori lessicali specifici per l'italiano.[35]

Ciò che interessa in questa sede è un'implicazione del modello spesso rimasta in secondo piano: se la competenza comunicativa è multidimensionale, può anche essere sviluppata selettivamente. Pertanto, le componenti sono distinte e sviluppabili in contesti diversi. Un ambiente narrativo strutturato può distribuirle deliberatamente su interlocutori e situazioni diverse, ciascuno progettato per sollecitare una dimensione specifica.

2.4.2 Input comprensibile, filtro affettivo e VR

Tra i costrutti della *Second Language Acquisition*, pochi hanno avuto un impatto pratico sull'insegnamento linguistico paragonabile a quello dell'*Input Hypothesis* di Stephen Krashen. La tesi centrale è disarmante nella sua semplicità: l'acquisizione avviene quando si comprendono messaggi, non quando si studiano regole grammaticali o si memorizza vocabolario.[36] Il nucleo della proposta krasheniana risiede nella formula *i+1*: l'acquisizione progredisce quando il discente è esposto a strutture leggermente oltre il suo livello attuale di competenza, rese accessibili grazie al contesto extralinguistico, alla conoscenza del mondo e alle risorse già acquisite.[37] L'input non deve essere né troppo

[33]Michael Canale, "From Communicative Competence to Communicative Language Pedagogy", in *Language and Communication*, a cura di Jack C. Richards e Richard W. Schmidt (London: Longman, 1983), pp. 2–27.

[34]Michael Canale e Merrill Swain, "Theoretical Bases of Communicative Approaches to Second Language Teaching and Testing", *Applied Linguistics* 1, no. 1 (1980): pp. 28–31.

[35]Barbara Spinelli, Fiorenza Parizzi e Joseph Sheils, *Profilo della lingua italiana: livelli di riferimento del QCER A1, A2, B1, B2* (Firenze: La Nuova Italia, 2010).

[36]Stephen D. Krashen, *The Input Hypothesis: Issues and Implications* (London: Longman, 1992), p. vii.

[37]Ibid., p. 2.

semplice né troppo distante dalla competenza attuale: nel primo caso non offrirebbe stimolo acquisizionale, nel secondo produrrebbe incomprensione e blocco cognitivo.

È qui che entra in gioco la nozione di filtro affettivo:[38] uno stato psicologico che, quando è elevato —tra le cause rilevate: ansia, scarsa motivazione, bassa autostima— ostacola l'elaborazione dell'input anche quando questo è comprensibile. Horwitz, Horwitz e Cope hanno operazionalizzato questa dimensione nel concetto di *Foreign Language Classroom Anxiety* (FLCA), dimostrandone la correlazione inversa con le misure di rendimento: più alta l'ansia, più basse le *performance*.[39]

La VR aggiunge a questo schema una variabile strutturale rilevante. Kaplan-Rakowski e Gruber hanno documentato empiricamente che il gruppo che praticava il parlato in VR ad alta immersione mostrava punteggi di FLCA significativamente più bassi rispetto al gruppo di controllo su videochiamata, con un effetto circa dieci volte superiore.[40] Gli autori attribuiscono il risultato alla mediazione dell'*avatar*, che protegge l'identità reale dell'apprendente e crea una *comfort zone* per l'errore.[41]

2.4.3 Scaffolding e Zona di Sviluppo Prossimale

Il contributo di Vygotskij all'apprendimento linguistico è complementare a quello di Krashen e si connette direttamente alla logica dell'input calibrato. Il concetto di *Zona di Sviluppo Prossimale* (ZSP), la distanza tra ciò che il discente è in grado di fare autonomamente e ciò che può fare con il supporto di un interlocutore più esperto, introduce la nozione di *scaffolding*: il sostegno temporaneo offerto per permettere al discente di operare oltre le proprie capacità attuali è da ritirarsi progressivamente man mano che la competenza cresce.[42] La ZSP non è distante dall'i+1 di Krashen: entrambi descrivono la zona ottimale di apprendimento come quella in cui il discente è esposto a qualcosa di leggermente superiore alle proprie risorse attuali, con il supporto necessario a renderlo accessibile. La differenza è che Krashen pensa all'input come variabile centrale, Vygotskij all'interazione con un esperto.

2.4.4 Task-Based Language Teaching

Il *Task-Based Language Teaching* (TBLT) è un *framework* educativo per l'insegnamento e l'apprendimento delle lingue seconde e straniere in cui i compiti comunicativi costituiscono il fulcro dell'istruzione, con il sillabo organizzato attorno a compiti comunicativi,

[38] Ibid., pp. 3–4.

[39] Elaine K. Horwitz, Michael B. Horwitz e Joann Cope, "Foreign Language Classroom Anxiety", *The Modern Language Journal* 70, no. 2 (1986): pp. 127–128.

[40] Regina Kaplan-Rakowski e Alice Gruber, "The Impact of High-Immersion Virtual Reality on Foreign Language Anxiety", *Smart Learning Environments* 10, no. 46 (2023): p. 11, Table 1.

[41] Ibid., p. 4.

[42] Lev S. Vygotskij, *Mind in Society: The Development of Higher Psychological Processes*, a cura di Michael Cole et al. (Cambridge, MA: Harvard University Press, 1978), pp. 86–87.

piuttosto che alla grammatica o al vocabolario.[43] I compiti a cui il TBLT fa riferimento sono le stesse attività che le persone svolgono nella vita reale —quelle che Long definisce «the hundred and one things people do in everyday life, at work, at play and in between»[44]. L'acquisizione avviene come effetto collaterale del tentativo di portarli a termine, non come fine esplicito dell'attività. Ellis precisa che la qualità di un task dipende dalla misura in cui vincola il discente a usare la lingua per uno scopo, senza permettergli di limitarsi a manipolarla formalmente.[45]

Peterson ha mostrato, in una meta-analisi sull'uso di giochi e simulazioni nel CALL, come gli ambienti digitali simulati favoriscano l'interazione comunicativa autentica e la motivazione dell'apprendente, con effetti positivi documentati su motivazione, partecipazione e coinvolgimento.[46] Leto, in un contesto più vicino al presente lavoro, ha applicato il *framework* TBLT all'insegnamento dell'italiano come lingua straniera a studenti sinofoni, documentando come l'organizzazione per compiti favorisca l'acquisizione delle sequenze verbali del verbo italiano nonostante la forte distanza tipologica tra le due lingue.[47]

2.4.5 Embodied cognition e Dual Coding Theory

C'è una domanda che la psicologia cognitiva ha a lungo rimosso e che la prospettiva dell'*embodied cognition* ha riportato al centro del dibattito: cosa ha a che fare il corpo con il pensiero? La risposta che Barsalou offre nella sua teoria della *Grounded Cognition* è radicale: i processi cognitivi, compresi quelli linguistici, sono radicati nell'esperienza sensorimotoria del corpo e nell'interazione con l'ambiente.[48] Comprendere il significato di una parola non è recuperare un'etichetta astratta in un dizionario mentale: è simulare l'esperienza percettiva e motoria associata a quella parola. Glenberg e Kaschak hanno fornito evidenza empirica diretta di questo meccanismo, dimostrando che la comprensione di frasi che descrivono azioni fisiche attiva il sistema motorio in modo misurabile.[49] Macedonia, Lehner e Repetto hanno esteso questo framework all'apprendimento linguistico in VR, riscontrando che la condizione con *grasping* virtuale produceva performance mnemoniche significativamente superiori rispetto alla sola esposizione audio-visiva, con effetto misurabile a trenta giorni dall'apprendimento.[50]

---

[43] Ali Shehadeh, Task-Based Language Teaching", in *Reference Collection in Social Sciences* (Elsevier, 2025): p. 2, DOI: 10.1016/B978-0-323-95504-1.01011-5.

[44] Long (1985, p. 89), citato in Shehadeh, Task-Based Language Teaching", p. 3.

[45] Rod Ellis, *Task-Based Language Learning and Teaching* (Oxford: Oxford University Press, 2003): p. 9.

[46] Mark Peterson, Computerized Games and Simulations in Computer Assisted Language Learning: A Meta-Analysis of Research", *Simulation and Gaming* 41, no. 1 (2010): p. 74.

[47] Fabrizio Leto, Costruire e analizzare un corpus di interlingua all'interno di un corso di italiano LS a studenti sinofoni", *Italiano LinguaDue* 15, no. 2 (2023): pp. 84–85, DOI: 10.54103/2037-3597/21940.

[48] Lawrence W. Barsalou, Grounded Cognition", *Annual Review of Psychology* 59 (2008): pp. 617–618.

[49] Arthur M. Glenberg e Michael P. Kaschak, Grounding Language in Action", *Psychonomic Bulletin & Review* 9, no. 3 (2002): pp. 558–559.

[50] M. Macedonia, A. E. Lehner e C. Repetto, "Positive Effects of Grasping Virtual Objects on Memory for Novel Words in a Second Language", *Scientific Reports* 10, no. 1 (2020): p. 6, tavola 2, condizione AVOG. L'effetto a 30 giorni è robusto nel *Free Recall German*, non significativo negli altri test.

A questo si affianca il contributo della *Dual Coding Theory* di Clark e Paivio: le informazioni elaborate attraverso canali multipli —simultaneamente verbale e non verbale— vengono codificate in modo più ricco e recuperate più facilmente rispetto a quelle elaborate attraverso un canale solo.[51]

2.4.6 Produzione linguistica e noticing

Krashen sostiene che l'input comprensibile sia condizione sufficiente per l'acquisizione, e che la produzione linguistica non svolga un ruolo causale in questo processo.[52] Merrill Swain ha argomentato la posizione contraria con la stessa intensità, sostenendo che l'input da solo non basta e che la produzione linguistica svolge funzioni acquisizionali proprie. La sua *Output Hypothesis* ne individua tre: il *noticing*, ovvero la consapevolezza della differenza tra ciò che si vuole dire e ciò che si riesce a dire; la funzione metalinguistica, che spinge a riflettere sulla forma; e l'*hypothesis testing*, ovvero la verifica tacita delle proprie ipotesi grammaticali attraverso la risposta dell'interlocutore.[53] Schmidt ha formalizzato la prima funzione nella sua *Noticing Hypothesis*: la condizione necessaria per la conversione dall'input all'*intake* non è la semplice esposizione alle forme, ma la loro registrazione consapevole.[54] Il meccanismo del *noticing* così inteso trova applicazione diretta nel sistema di listen-and-repeat della scena del bar, discusso al §3.5.

2.4.7 Feedback correttivo ed errorful learning

Non tutti i *feedback* correttivi funzionano allo stesso modo. Lyster e Ranta, in uno studio condotto su quattro classi di immersione francese con 18,3 ore di interazione trascritte e analizzate, hanno identificato sei tipi di *feedback* correttivo e ne hanno misurato l'efficacia in termini di *uptake*[55]. Il risultato più rilevante riguarda il *recast*, che è il tipo di *feedback* più usato dagli insegnanti e consiste nella riformulazione corretta dell'enunciato errato senza segnalazione esplicita dell'errore. Tuttavia, esso risulta il meno efficace per produrre *repair* autonomo, con uno 0 percento di *student-generated repair* rispetto al 46 percento prodotto dall'elicitazione.[56] I tipi che generano *uptake* più consistente sono l'elicitazione, il *feedback* metalinguistico e la richiesta di chiarimento: meccanismi che restituiscono al discente la responsabilità della correzione.

---

[51] James M. Clark e Allan Paivio, "Dual Coding Theory and Education", *Educational Psychology Review* 3, no. 3 (1991): p. 166.

[52] Krashen, *The Input Hypothesis*, p. 2.

[53] Ali Shehadeh, Output Hypothesis", in *Reference Collection in Social Sciences* (Elsevier, 2026), pp. 820–822, DOI: 10.1016/B978-0-323-95504-1.01159-5.

[54] Richard Schmidt, The Role of Consciousness in Second Language Learning", *Applied Linguistics* 11, no. 2 (1990): pp. 129–131.

[55] Indica la risposta immediata del discente al *feedback* ricevuto dall'insegnante o dall'interlocutore: in pratica, cosa fa lo studente subito dopo aver ricevuto una correzione. L'*uptake* può essere: *repair*, se lo studente corregge effettivamente l'errore; *acknowledgement*, se lo studente riconosce la correzione senza correggersi («ah sì»); nessuna risposta, se lo studente ignora il *feedback* e va avanti

[56] Roy Lyster e Leila Ranta, "Corrective Feedback and Learner Uptake: Negotiation of Form in Communicative Classrooms", *Studies in Second Language Acquisition* 19, no. 1 (1997): pp. 46–49, 55.

La ricerca di Metcalfe offre un fondamento empirico complementare. La sua revisione della letteratura sull'*errorful learning*, infatti, mostra che commettere errori seguiti da *feedback* correttivo immediato in un contesto semanticamente ricco migliora la ritenzione della risposta corretta rispetto all'apprendimento privo di errori, con un effetto particolarmente marcato per gli errori commessi con alta confidenza.[57] Questo effetto si produce perché condizionato dalla qualità degli errori stessi: affinché la confidenza sia alta, i distrattori devono essere genuinamente plausibili e opportunamente costruiti. Rodriguez, Kettler ed Elliott precisano che i distrattori efficaci sono quelli che attraggono gli studenti portatori delle misconcezioni specifiche di quel contenuto, non quelli genericamente sbagliati.[58]

2.4.8 Cognitive Load Theory

La *Cognitive Load Theory* di Sweller aggiunge la prospettiva dell'apprendente, occupandosi di quante risorse cognitive il discente ha a disposizione per elaborare nuove informazioni, e di come la modalità di presentazione dei contenuti possa ridurle o preservarle.[59] Sweller distingue tra carico cognitivo *intrinseco*, ossia la complessità ineliminabile del contenuto da apprendere, e carico *estraneo*, generato cioè dalla modalità di presentazione dell'informazione più che del contenuto. Il carico estraneo dovrebbe essere sempre semplificato o ridotto al minimo, perché sottrae risorse di elaborazione al carico intrinseco senza contribuire all'apprendimento.[60] La distinzione è rilevante per qualsiasi ambiente didattico, ma diventa critica in un contesto VR: il discente è già impegnato a gestire simultaneamente l'orientamento spaziale, i gesti di interazione, l'input linguistico e la produzione orale. In queste condizioni, qualsiasi complessità aggiuntiva, dall'uso di un *controller* fisico a istruzioni grammaticali non richieste, fino a informazioni presentate fuori contesto, entra in competizione diretta con le risorse cognitive destinate alla lingua. Ridurre il carico estraneo è quindi una condizione necessaria per rendere efficace l'apprendimento linguistico in un ambiente già cognitivamente denso. Sweller, van Merrienboer e Paas identificano alcune strategie concrete per ridurlo. L'effetto di *split-attention* riguarda l'integrazione mentale di fonti di informazione fisicamente separate. Dal testo, *«such mental integration likely imposes a heavy, extraneous cognitive load»*, e la soluzione è l'integrazione fisica degli elementi nello stesso spazio percettivo. [61] L'*effetto di modalità* mostra che presentare informazioni attraverso canali sensoriali diversi simultaneamente aumenta la capacità effettiva della memoria di lavoro rispetto all'uso di un solo canale.[62]

[57] Janet Metcalfe, Learning from Errors", *Annual Review of Psychology* 68 (2017): pp. 468–470, 472–474.

[58] Michael C. Rodriguez, Ryan J. Kettler e Stephen N. Elliott, Distractor Functioning in Modified Items for Test Accessibility", *Sage Open* 4, no. 4 (2014), DOI: 10.1177/2158244014553586.

[59] John Sweller, "Cognitive Load Theory", in *Psychology of Learning and Motivation*, vol. 55 (Academic Press, 2011): p. 57, DOI: 10.1016/B978-0-12-387691-1.00002-8.

[60] Ibid., p. 63.

[61] John Sweller, Jeroen J. G. van Merrienboer e Fred G. W. C. Paas, "Cognitive Architecture and Instructional Design", *Educational Psychology Review* 10, no. 3 (1998): p. 277.

[62] Ibid., p. 282.

L'*effetto di ridondanza*, infine, citando testualmente, *«occurs when multiple sources of information are self-contained and can be used without reference to each other.»*: presentare la stessa informazione in formati multipli può paradossalmente aumentare il carico cognitivo anziché ridurlo, nella misura in cui ciascuna fonte è già autonomamente comprensibile.[63]

2.5 L'obiettivo didattico: un sillabo per la sopravvivenza linguistica

Il §2.3 ha descritto l'apprendente target come un adulto *goal-oriented*, che nei termini di Knowles si impegna nello studio nella misura in cui vi riconosce uno strumento per raggiungere obiettivi concreti e immediati,[64] e lo ha caratterizzato per questo come *viaggiatore*. I costrutti discussi al §2.4 indicano come sostenerne l'apprendimento; resta da stabilire che cosa, al termine dell'esperienza, questo viaggiatore debba saper fare. Definire l'obiettivo didattico significa, prima di tutto, resistere alla tentazione dell'esaustività. Il *target* del prototipo non è la copertura sistematica del livello A1 del QCER, ma la costruzione di un repertorio minimo funzionale alla sopravvivenza linguistica in situazioni di vita quotidiana: nei termini di Nation e Crabbe, un *survival syllabus* «*reductionist in nature*».[65] Gli autori individuano un sottoinsieme prioritario di funzioni comunicative (salutare, presentarsi, chiedere informazioni, ordinare, gestire transazioni semplici), selezionate in base alla frequenza d'uso e all'urgenza comunicativa del *viaggiatore* straniero, e stimano in circa 120 gli elementi lessicali ed espressivi minimi sufficienti a coprire i bisogni turistici primari. Si tratta di una soglia pari a circa un quarto delle dimensioni del Waystage del Consiglio d'Europa,[66] il livello di accesso progettato per consentire al discente di «*satisfy their most urgent communication needs*» entro un carico di apprendimento «*generally acceptable and also manageable within a comparatively short time*».[67]

L'organizzazione per funzioni non nasce con il sillabo di sopravvivenza. Si inscrive nella tradizione dei sillabi nozional-funzionali inaugurata da Wilkins, secondo cui l'insegnamento linguistico dovrebbe muovere dalle funzioni che l'apprendente deve saper esprimere piuttosto che da un inventario di strutture astratte.[68] Il Profilo della Lingua Italiana eredita questa impostazione e organizza le competenze A1 in categorie funzionali (interagire su informazioni, asserire, descrivere, valutare, esprimere stati emotivi), ciascu-

[63] Ibid., p. 283.

[64] Knowles, *op. cit.*, p. 56.

[65] Paul Nation e David Crabbe, "A Survival Language Learning Syllabus for Foreign Travel", *System* 19, no. 3 (1991): p. 200.

[66] Nella versione 1990, che ridefinisce il Waystage come obiettivo autonomo rispetto all'edizione originale del 1977.

[67] Jan Ate van Ek e John Leslie Melville Trim, *Waystage 1990* (Cambridge: Cambridge University Press, 1998), p. 4.

[68] David Wilkins, *Notional Syllabuses* (Oxford: Oxford University Press, 1976).

na associata a inventari lessicali e grammaticali specifici.[69] Il sillabo di sopravvivenza opera all'interno di questo quadro, ma ne seleziona drasticamente il contenuto: i criteri con cui la selezione è stata condotta per *ItalianVRExperience* sono documentati al §3.2.

La drasticità della selezione ha una conseguenza diretta sul modo in cui la lingua viene presentata. Rinunciare alla scomposizione grammaticale astratta, incompatibile con i tempi ristretti del viaggiatore e con l'orientamento pragmatico dell'apprendente adulto, significa affidarsi a blocchi di linguaggio prefabbricati, i cosiddetti *lexical chunks*: in termini più comuni, le "frasi fatte". Non si tratta di una scorciatoia didattica, ma di un tratto del funzionamento stesso della lingua. Wray ha mostrato che anche i parlanti nativi fanno ampio ricorso a sequenze lessicali precostituite, memorizzate e recuperate come unità globali piuttosto che generate di volta in volta tramite regole sintattiche.[70] Nattinger e DeCarrico ne traggono una conseguenza per la didattica: la fluenza si baserebbe non tanto sulla generazione sintattica di enunciati inediti quanto sul recupero di un repertorio di sequenze ricorrenti, associate a funzioni pragmatiche specifiche.[71]

Per l'apprendente elementare, il ricorso alle sequenze formulaiche è ancora più vantaggioso. Lewis osserva che le espressioni istituzionalizzate, acquisite come interi funzionali anziché come strutture da analizzare, «*provide a way of increasing the elementary student's communicative resources rapidly, and at the same time provide accurate and natural data against which other novel utterances may be [...] a valuable resource contributing to the acquisition of competence*».[72] Lo stesso autore riconosce come consolidata l'idea che «*an important part of language acquisition is the ability to produce lexical phrases as unanalysed wholes or 'chunks', and that these chunks become the raw data by which the learner begins to perceive patterns, morphology, and those other features of language traditionally thought of as 'grammar'*».[73] Skehan conferma il quadro dal versante delle differenze individuali: nelle fasi iniziali gli apprendenti operano su un principio di «*analyse only if you have to*»,[74] e memorizzazione e linguaggio formulaico costituiscono invariabilmente le prime strategie a emergere nel loro repertorio.[75] Espressioni come *vorrei un caffè* o *dov'è l'uscita?* non vengono scomposte nelle loro componenti morfosintattiche, ma acquisite come unità pronte all'uso: chi ha memorizzato *vorrei un caffè, per favore* è in grado di usarla correttamente molto prima di aver compreso la struttura del condizionale italiano.

---

[69] Spinelli, Parizzi e Sheils, *op. cit.*

[70] Alison Wray, *Formulaic Language and the Lexicon* (Cambridge: Cambridge University Press, 2002), p. 9.

[71] James R. Nattinger e Jeanette S. DeCarrico, *Lexical Phrases and Language Teaching* (Oxford: Oxford University Press, 1992), citati in Pierre S. L. Arnaud, recensione, *Meta* 38, no. 3 (1993): p. 567, DOI: 10.7202/001902ar.

[72] Michael Lewis, *The Lexical Approach: The State of ELT and a Way Forward* (Hove: Language Teaching Publications, 1993), p. 95.

[73] Ibid.

[74] Peter Skehan, *Individual Differences in Second Language Learning* (London: Edward Arnold, 1989), p. 37.

[75] Ibid., p. 94.

Il rovescio della medaglia è altrettanto chiaro. Un sillabo formulaico garantisce fluenza funzionale, ma non la comprensione della struttura sottostante: chi sa dire *vorrei un caffè* potrebbe non essere in grado di generalizzare la forma a un contesto diverso. È il *trade-off* costitutivo del sillabo di sopravvivenza, accettabile al livello A1, dove l'urgenza comunicativa prevale sulla sistematicità, e da riconsiderare nel passaggio ai livelli successivi.

Il sillabo di sopravvivenza è infine, nei termini di Nation e Crabbe, «*clearly oriented towards production*»:[76] il viaggiatore ha bisogno di prendere l'iniziativa comunicativa e dispone di poche possibilità di svolgere un ruolo puramente ricettivo nelle prime interazioni reali. È un'indicazione che, pur nata da esigenze pratiche, incontra sul piano teorico l'*Output Hypothesis* di Swain discussa al §2.4.6: la produzione non è soltanto ciò di cui il viaggiatore ha bisogno, ma uno dei meccanismi attraverso cui l'acquisizione avviene. Privilegiare produzione orale e ascolto a sfavore della dimensione scritta non è dunque una semplificazione, ma una definizione precisa e metodologicamente fondata dell'obiettivo.

[76] Nation e Crabbe, *op. cit.*, p. 200.

3 Progettazione del sistema

I capitoli precedenti hanno delineato il problema, giustificato la scelta della realtà virtuale come contesto didattico e ricostruito il quadro teorico che sorregge le scelte implementative. Questo capitolo entra nel merito dei criteri con cui è stato progettato il prototipo, con il sillabo di base, la logica narrativa che organizza l'esperienza di Bellavalle, e la progressione didattica che struttura l'interazione del discente con il sistema. È il primo capitolo del contributo originale, e come tale cambia registro. Si passa qui dallo stato dell'arte alla documentazione progettuale effettiva.

Il terzo capitolo è organizzato come segue. Il §3.1 illustra il *framework* metodologico adottato per lo sviluppo, la *Design-Based Research*, e ne chiarisce le implicazioni per la trasparenza sulle revisioni in itinere. Il §3.2 esplicita i tre criteri con cui le funzioni comunicative sono state selezionate dal Profilo della Lingua Italiana, dichiarando le tensioni tra esaustività del *framework* di riferimento e riduzionismo del sillabo di sopravvivenza. Il §3.3 descrive la cornice narrativa di Bellavalle e la logica della progressione tra le scene. Il §3.4 documenta la progettazione dei personaggi non giocanti in termini di registro, relazione sociale e funzioni linguistiche. Il §3.5, infine, presenta le scene una per una e le fasi della progressione didattica al loro interno, dall'esposizione guidata alla produzione orale.

3.1 Indicazioni metodologiche: Design-Based Research

La scelta di un approccio metodologico è, prima di tutto, una scelta su quale tipo di domanda vale la pena porre. Se in un esperimento randomizzato controllato si è soliti chiedere, "questa versione funziona meglio di quella?", la *Design-Based Research* riformula la domanda: come funziona questo artefatto nell'uso reale, e cosa impariamo ogni volta che lo modifichiamo?

È la seconda domanda quella pertinente per questo lavoro. La Design-Based Research (DBR) è un paradigma metodologico consolidato nella ricerca educativa che privilegia lo sviluppo e il perfezionamento progressivo di artefatti didattici concreti come modalità di indagine, in alternativa alla verifica a posteriori di un progetto già definito. Il

Design-Based Research Collective la caratterizza attraverso cinque tratti fondamentali: l'intreccio tra progettazione di ambienti di apprendimento e sviluppo teorico; la ciclicità del processo, articolato in fasi continue di design, implementazione, analisi e *redesign*; la produzione di teorie condivisibili e trasferibili ad altri progettisti; l'attenzione a come il design funziona in contesti autentici, non in condizioni controllate; e l'uso di metodi capaci di connettere i processi di implementazione agli esiti osservati.[77] Ogni revisione dell'artefatto produce conoscenza nuova, quella conoscenza contestualizzata che emerge solo dall'incontro tra il design e chi lo usa, e che distingue la DBR dalla verifica a posteriori di un progetto già definito.

Nel corso dello sviluppo di *ItalianVRExperience* questo ciclo si è realizzato in cinque momenti precisi. Il primo riguarda il sistema di dialogo: la versione iniziale prevedeva la selezione di risposte tramite bottoni virtuali, senza produzione orale. L'implementazione ha reso evidente la contraddizione con l'obiettivo centrale del progetto, e ha portato all'introduzione del riconoscimento vocale tramite Whisper nella scena del quartiere. Il secondo riguarda il modello linguistico: l'ipotesi di affidare a Mistral la generazione di dialogo libero con gli NPC è stata abbandonata dopo i test preliminari, che hanno mostrato l'impossibilità di controllare la complessità lessicale dell'output al livello A1. Da quella valutazione è emersa la scelta di circoscrivere Mistral al tutor grammaticale. Il terzo riguarda l'interazione fisica: eliminare i controller in favore del hand tracking ha richiesto una revisione dell'intera architettura di interazione, motivata dalla necessità di ridurre il carico cognitivo estraneo dell'interfaccia, come discusso al §2.4.8. Gli ultimi due, più circoscritti, sono avvenuti durante lo studio stesso, dopo le due sessioni pilota. Il quarto riguarda le istruzioni: i soli pannelli scritti di *onboarding* si sono rivelati insufficienti a guidare i partecipanti, e a ciascuno è stata associata una clip audio che ne legge il testo. Il quinto riguarda il turno di parola: i partecipanti faticavano a capire quando cominciare a parlare, e per questo è stato introdotto un segnale sonoro di inizio turno.

È questa natura ciclica a distinguere la DBR dalla ricerca sperimentale classica: la DBR produce conoscenza contestualizzata. I risultati di questo lavoro valgono per questo prototipo, questi partecipanti, queste condizioni di test: estenderli richiederà ulteriori cicli di ricerca e sviluppi futuri, cui si dedica attenzione nelle conclusioni (§6). La revisione dell'artefatto non è un fallimento da correggere, ma il momento in cui la ricerca produce conoscenza nuova.[78]

3.2 Il sillabo: dal PLI alla sopravvivenza linguistica

Il Profilo della Lingua Italiana documenta sessantadue funzioni comunicative per il livello A1. La selezione di quelle da includere nel sillabo ha seguito tre criteri applicati in ordine

[77] The Design-Based Research Collective, "Design-Based Research: An Emerging Paradigm for Educational Inquiry", *Educational Researcher* 32, no. 1 (2003): pp. 5–6.

[78] Ibid.

gerarchico: urgenza comunicativa, orientamento alla produzione orale, distribuibilità nelle scene.

Il primo ha filtrato le funzioni sulla base di quanto siano immediatamente necessarie per un anglofono adulto alle prime interazioni in italiano: salutare, presentarsi, chiedere informazioni, ordinare, pagare, congedarsi. Il secondo ha privilegiato, tra quelle di pari urgenza, le funzioni che richiedono un contributo attivo del player: un sillabo centrato su funzioni ricettive avrebbe contraddetto la scelta del riconoscimento vocale come modalità primaria di interazione. Il terzo ha distribuito le funzioni selezionate sui quattro NPC in modo da rispettare il gradiente di rischio comunicativo descritto al §3.3, con la variazione sociolinguistica tra registro formale e informale incontrata in contesto, senza spiegazioni esplicite.

In coerenza con l'impostazione formulaica discussa al §2.5, ciascuna funzione selezionata è resa nel gioco attraverso uno o più *chunks* situati, e non attraverso le strutture grammaticali che la compongono. Lo stesso principio governa l'inventario linguistico del gioco: le frasi salvate automaticamente nelle schede *Domande* e *Risposte* (§4.5) non formano un glossario di voci isolate, ma un repertorio di sequenze pronte all'uso, ciascuna legata all'NPC e alla situazione in cui è stata incontrata.

Va dichiarata con onestà la conseguenza di questa scelta sul prototipo. Il *trade-off* descritto al §2.5, tra fluenza funzionale e comprensione della struttura, è accettato consapevolmente: *ItalianVRExperience* non offre al giocatore strumenti per generalizzare le formule apprese a contesti nuovi, se non in forma embrionale attraverso il tutor grammaticale, e questo resta uno degli elementi da affrontare in un eventuale sviluppo a livelli superiori.

Il risultato della selezione è un insieme ristretto di funzioni comunicative, distribuite tra registro formale e informale e ancorate a situazioni che un visitatore straniero incontra nelle prime ore di permanenza in Italia: interagire con un interlocutore istituzionale, chiedere informazioni sul luogo, presentarsi ed esprimere soddisfazione, gestire numeri e denaro in uno scambio reale, formulare una richiesta in un contesto di servizio e rifiutare con cortesia, congedarsi nel registro appropriato. La distribuzione di queste funzioni tra i personaggi del gioco è illustrata al §3.4. Adottare il PLI come *framework* e rinunciare alla maggior parte delle sue funzioni è il prezzo di qualsiasi sillabo di sopravvivenza: al termine dell'esperienza il giocatore dispone di un repertorio funzionale minimo, non di una competenza A1 completa. I limiti di questa scelta sono ripresi nelle conclusioni (§6).

3.3 Bellavalle: cornice narrativa

Bellavalle è il borgo immaginario in cui è ambientata l'esperienza. Il giocatore vi arriva in treno come visitatore straniero e ne attraversa quattro luoghi in sequenza: la stazione, il quartiere, il bar e la casa. Bellavalle, tuttavia, non si esplora liberamente. Ogni scena è

associata a un insieme definito di funzioni comunicative, e la progressione tra loro segue un gradiente di rischio comunicativo crescente, inteso come il grado di imprevedibilità e libertà interattiva che il player deve affrontare per portare a termine il compito. Questa architettura narrativa trova il suo fondamento nel *Task-Based Language Teaching* (§2.4.4) e nel principio dell'apprendimento situato: la lingua si impara usandola in situazione, in un contesto che genera pressione comunicativa autentica. I principi della progressione incrementale e del sottoinsieme identificati da Gee giustificano la scelta di ordinare le scene non per difficoltà grammaticale ma per grado crescente di rischio comunicativo, replicando la logica dello scaffolding decrescente discussa al §2.4.3.

Il percorso narrativo ha un punto di partenza fisso: la stazione ferroviaria, dove il controllore attende con il biglietto. È la prima scena vincolata dell'ambiente virtuale, una transazione in cui le mosse comunicative sono predeterminate e il margine di variazione pragmatica è minimo. La scena in stazione serve da *onboarding*: prima che il dialogo con il controllore abbia inizio, il player può esplorare liberamente lo spazio, provare i gesti che attivano le componenti interattive del gioco (spiegati da pannelli interattivi) per cominciare a orientarsi nell'ambiente virtuale; scopre come attivare il movimento, l'inventario e come usare le stesse mani per la manipolazione degli oggetti. È proprio quest'ultima a sbloccare il primo dialogo: approcciato dal controllore, per avanzare, il player deve consegnargli fisicamente il biglietto. Da lì il percorso si apre nel quartiere residenziale, dove Lina chiede al player di presentarsi e di dire dove è diretto, aprendo un primo spazio di scelta reale. Esplorando il quartiere, il player può incontrare un chitarrista di strada: un personaggio opzionale, con cui può scegliere di fermarsi a contare le monete o proseguire verso casa. La scena del bar è invece a ingresso discrezionale: il player può decidere se varcare la soglia del locale e ritrovarsi in un contesto di interazione di servizio.

Questa struttura produce un gradiente di libertà interattiva che rispecchia quello del rischio comunicativo. Alla stazione il player risponde a stimoli vincolati in un registro obbligato. Nel quartiere le risposte disponibili sono tutte corrette sul piano pragmatico: la pressione si sposta dalla forma alla scelta. Il chitarrista aggiunge una variabile di partecipazione senza conseguenze narrative permanenti. La casa allenta tutto: è uno spazio di esplorazione lessicale e grammaticale in cui il player interagisce con l'ambiente al proprio ritmo. Il filo narrativo rende questa progressione didattica percepibile come esperienza coerente. All'interno di ciascuna scena, prima che il dialogo abbia inizio, il player incontra pannelli con le opzioni di risposta in forma scritta e audio: una fase di esposizione guidata la cui funzione è discussa nel dettaglio al §3.5.

3.4 Personaggi non giocanti: funzioni linguistiche associate

Ciascun NPC incarna una configurazione diversa delle componenti della competenza comunicativa di Canale e Swain (§2.4.1), assegnata in funzione del contesto e della

relazione sociale. La variazione tra gli NPC è il meccanismo attraverso cui il sillabo espone il player a configurazioni pragmatiche diverse senza richiedere spiegazioni esplicite.

Il controllore opera in un registro neutro e funzionale, con mosse comunicative predeterminate: segue un copione fisso, e per ogni nodo è disponibile una sola risposta corretta, senza spazio per alternative equivalenti. L'uso del *lei* è obbligato. La componente strategica è sollecitata dall'opzione *Scusi?*, con cui il giocatore può chiedere al controllore di ripetere: l'opzione replica in forma digitale la negoziazione del significato, restituendo al player la responsabilità di segnalare l'incomprensione.

L'interazione con la vicina, Lina, funziona in modo diverso. Il registro è informale, il *tu* è naturale, e tutte le opzioni di risposta disponibili sono corrette sul piano pragmatico: *Sì, sono nuovo* e *Sono appena arrivato* realizzano la stessa funzione in forme equivalenti. È un contrasto deliberato con la rigidità sociolinguistica della scena precedente, e riflette la natura del registro informale, in cui la variazione è attesa. Va dichiarato uno scostamento dal PLI: *Nessun problema*, in risposta alla scusa di Lina (Appendice), compare nel Profilo solo come esponente A2, ma è stato incluso per la sua urgenza comunicativa, coerentemente con il primo criterio enunciato al §3.2, perché si tratta di una formula irrinunciabile in qualsiasi incontro sociale.

Il chitarrista è un personaggio opzionale ed episodico, perché l'utente non è vincolato in alcun modo a interagirci. Il registro che usa è marcatamente informale, dovuto al fatto che la relazione con l'utente è quella dello scambio tra estranei in un contesto pubblico non istituzionale. La missione di conteggio è progettata come compito di apprendimento incarnato: contare monete in uno spazio virtuale, dove è possibile afferrarle con la mano e spostarle nella custodia della chitarra, è pensato per attivare simultaneamente il canale verbale e quello sensorimotorio, secondo il principio dell'*embodied cognition* discusso al §2.4.5, e per far emergere il lessico numerico direttamente dal compito. Nelle sessioni sperimentali, tuttavia, la missione non è stata attivata da alcun partecipante.

La barista opera in registro di servizio, più vincolato di Lina ma meno rigido del controllore. Le funzioni attivate sono la richiesta in contesto di servizio, il rifiuto educato, la transazione economica e il congedo. I distrattori replicano errori sistematici dell'anglofono a livello A1: la confusione tra *prego* e *grazie*, tra una formula di accordo e una di richiesta. Per lo stesso criterio di urgenza comunicativa è stato incluso *Ecco a lei*, in risposta alla richiesta di pagamento, anch'esso non coperto esplicitamente dal PLI A1.

Sul piano del *feedback* correttivo, l'unica scena in cui una risposta può essere scorretta è il bar, dove i distrattori bloccano l'avanzamento. Quando il giocatore sceglie un distrattore, la barista non riformula la risposta corretta, ma segnala di non aver capito e ripropone la domanda (*Non ho capito. Vuole ordinare qualcosa?*): si tratta di una richiesta di chiarimento, che restituisce al giocatore la responsabilità della correzione senza interrompere la finzione narrativa. Secondo la tassonomia di Lyster e Ranta discussa al §2.4.7, è uno dei tipi di *feedback* che generano *uptake* più consistente, a

differenza del *recast*. La scelta di un *feedback* implicito e interno al dialogo, anziché di una correzione esplicita, è consapevole: interrompere il flusso comunicativo con una spiegazione romperebbe l'immersione e segnalerebbe al giocatore che si trova in un contesto di addestramento. La funzione metalinguistica esplicita non scompare del tutto dall'architettura del sistema: viene affidata al tutor AI Clippy, accessibile su richiesta dalla scheda *AI* dell'inventario. La storica mascotte di Microsoft Office interviene quando è il player stesso a cercare chiarimento sulla forma. Il tutor AI restituisce le coniugazioni dei verbi incontrati nei dialoghi del gioco, definizioni lessicali contestualizzate per le parole del vocabolario di scena, e riconosce forme verbali coniugate riconducendole all'infinito corrispondente con coniugazione completa. Il sistema è inoltre in grado di segnalare al player se la parola richiesta è già stata incontrata nel corso della sessione, collegando il supporto metalinguistico al percorso specifico dell'apprendente nell'ambiente. Gli sviluppi futuri potranno estendere questa reattività al riconoscimento automatico degli errori prodotti durante i dialoghi, come esplicitato al §6. Le scene includono anche personaggi di sfondo non interagibili, con un'eccezione: una stessa figura femminile compare sia sul treno sia al bar. Quando il giocatore la tocca con il gesto *pinch*, pronuncia una frase scelta casualmente tra quelle previste per il contesto in cui si trova, mostrata a sua volta accanto al personaggio. Si tratta di frasi di circostanza, che non richiedono risposta: la figura serve ad aumentare la densità comunicativa dell'ambiente e il senso di trovarsi in un luogo abitato, senza aggiungere carico cognitivo.

La tabella che segue riassume la distribuzione delle funzioni comunicative per ciascun NPC con il corrispondente ancoraggio al PLI A1. La mappatura completa delle funzioni comunicative associate agli NPC è consultabile in Appendice A.

3.5 Le scene e la progressione didattica

La progressione didattica all'interno di ogni scena si articola in fasi distinte, che variano per tipo e quantità di supporto offerto al discente. Prima di descriverle, tuttavia, è necessario chiarire un elemento comune a tutte le scene che prevedono dialogo: prima che l'interazione con un NPC abbia inizio, il player può consultare pannelli guida che presentano le opzioni di risposta disponibili in forma scritta, accompagnate da una clip audio che ne riproduce la pronuncia. L'obiettivo è che quando nel dialogo si troverà a leggere le stesse opzioni, il player abbia già interiorizzato come suonano in italiano. Il pannello fa da ponte tra la forma scritta e la sua realizzazione fonetica. È una soluzione pensata per un principiante assoluto che, esposto per la prima volta a forme italiane scritte, non dispone delle risorse fonologiche per dedurne autonomamente la pronuncia, come discusso al §2.3.2. La consultazione dei pannelli guida non è obbligatoria, anzi, il player può liberamente avvicinarsi all'NPC senza averli esplorati. Chi lo fa, però, si trova a dover pronunciare forme che non ha mai sentito. Una difficoltà sufficiente a motivare, al tentativo seguente, la consultazione preventiva dei pannelli. Questa fase di esposizione guidata è lo *scaffolding* che rende possibile il passo successivo: non si elimina la difficoltà, la si rende affrontabile, in coerenza con la logica della *Zona di*

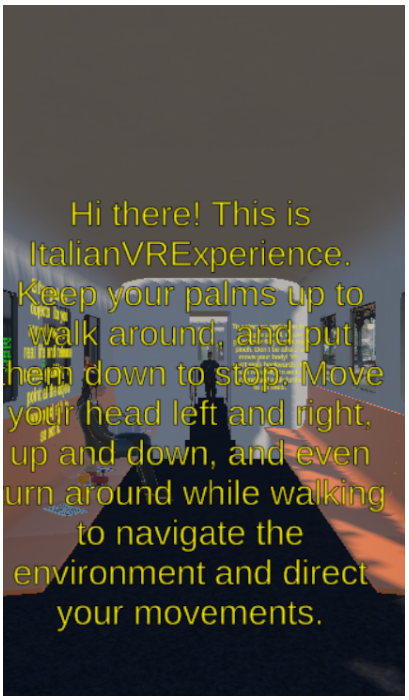
NPC	Funzioni comunicative	Tipo di risposta
Controllore	Rispondere a una richiesta istituzionale; informarsi sul luogo; esprimere gratitudine; attirare l'attenzione; prendere congedo	Vincolata
Lina	Scusarsi/accettare le scuse; presentarsi; informarsi sul modo e sul luogo; esprimere soddisfazione; prendere congedo	Aperta
Chitarrista	Asserire; rispondere accettando; strutturare l'interazione; rispondere a un ringraziamento	Aperta
Barista	Chiedere in contesto di servizio; rispondere rifiutando; gestire una transazione economica; prendere congedo	Vincolata

Tabella 2: Distribuzione delle funzioni comunicative per NPC e tipo di risposta richiesta. La mappatura completa con esponenti PLI e dialoghi è in Appendice .

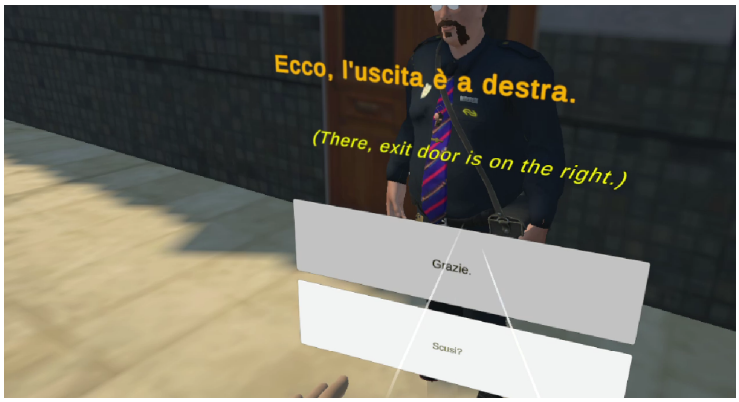
Sviluppo Prossimale discussa al §2.4.3. Segue una presentazione delle scene dell'ambiente immersivo.

01 Stazione: il controllore

La scena di *onboarding* introduce il player all'ambiente e ai gesti di interazione prima che il dialogo abbia inizio. Il primo *spawn* dell'utente è sul treno, dove una serie di pannelli interattivi guidano progressivamente alla scoperta dei gesti disponibili: il movimento, la manipolazione degli oggetti, l'apertura dell'inventario e l'orientamento nello spazio tramite la testa. Questi sono descritti in dettaglio in Appendice B (Figura 1a). Ciascun pannello è accompagnato da una clip audio che ne legge il testo: una soluzione introdotta dopo le sessioni pilota, nelle quali i soli pannelli scritti si erano rivelati insufficienti a guidare partecipanti impegnati contemporaneamente con il visore, i gesti e l'ambiente. Dopo aver preso dimestichezza con il visore e l'ambiente, l'utente arriva in una zona del treno in cui viene raggiunto dal controllore, che chiede il biglietto. Per avanzare, il player deve consegnarlo fisicamente, dunque afferrare l'oggetto fisico e porgerlo al controllore (Figura 2). È la prima istanza di apprendimento incarnato del gioco (§2.4.5): l'azione fisica è il compito comunicativo stesso, e non è decorativa. Il dialogo successivo è interamente a selezione di pannello, senza produzione vocale: il player sceglie la risposta toccando l'opzione con il gesto *pinch*. Le risposte sono vincolate e i distrattori non sono presenti. La battuta italiana è corredata di traduzione inglese.



(a) Pannello di *onboarding* sulla locomozione.

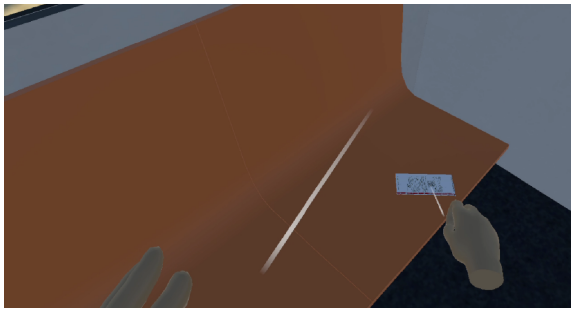


(b) Il controllore indica l'uscita.

Figura 1: La stazione: istruzioni iniziali e primo dialogo.



(a) La richiesta del controllore.



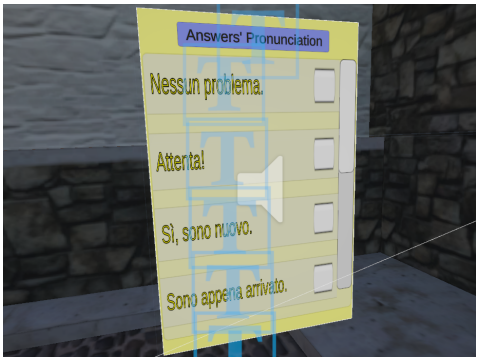
(b) Il biglietto afferrato con la mano.

Figura 2: *Biglietto, prego*: la risposta del giocatore è un'azione fisica.

02 Quartiere: la vicina

È la prima scena con produzione vocale sistematica. Prima del dialogo, il pannello di *onboarding* spiega con quale gesto attivare il microfono. Poi, a comparsa, il pannello guida mostra le opzioni di risposta in italiano con le loro clip audio e i sottotitoli in inglese, a supporto completo per il principiante assoluto (Figura 3). Durante dialogo, le opzioni di risposta, nonostante siano già state presentate all'utente, restano visibili, affinché il player possa leggerle pronunciando la risposta scelta dopo aver attivato il

microfono. Whisper trascrive, il sistema verifica la corrispondenza con i *trigger vocali* attesi e avanza al nodo successivo se e quando corrisponde con il *target*. L'interazione con Lina avanza a prescindere da quale opzione viene pronunciata: tutte le opzioni disponibili sono pragmaticamente corrette, perché le funzioni comunicative inerenti al dialogo con la vicina ammettono variazioni. Non ci sono distrattori, e questa scelta riflette la natura del registro informale discussa al §3.4: la variazione pragmatica non si apprende per regola, ma per esposizione a contesti in cui forme diverse producono effetti equivalenti. I sottotitoli in inglese e la velocità pedagogica dell'audio degli NPC sono le traduzioni concrete del principio dell'input comprensibile (§2.4.2): rendono accessibile un input altrimenti opaco, sfruttando la familiarità con la L1 come leva di comprensione e abbassando il filtro affettivo nelle prime interazioni. La cattura del nome tramite riconoscimento vocale è l'unico nodo in cui la risposta è libera: dopo la richiesta di presentazione della vicina, il player dice il proprio nome e Whisper lo trascrive (senza confronto con trigger attesi). I dati di questa scena costituiscono la *baseline* per RQ2: misurano la tenuta del riconoscimento vocale nelle condizioni di supporto massimo, con sottotitoli presenti e nessuna pressione da distrattori.



(a) Pannello guida con le risposte da ascoltare.



(b) Lina, con sottotitolo e opzioni di risposta.

Figura 3: Il quartiere: supporto massimo, con traduzione inglese.

02 Quartiere: il chitarrista
La missione del chitarrista è opzionale e non prevede produzione vocale. La scena si attiva quando il player deposita una banconota da dieci euro nella custodia della chitarra: il gesto avvia il dialogo, al termine del quale il chitarrista chiede aiuto per contare le monete (Figura 10). Se il player accetta, conta fisicamente le monete afferrandole con le mani, supportato dall'ascolto delle clip audio dei numeri in italiano associate a ciascuna moneta, da 1 a 10. È una scena di ricezione e azione fisica situata, il cui fondamento nell'*embodied cognition* è discusso al §2.4.5. La sua presenza nelle sessioni non era garantita e, di fatto, nessun partecipante ne ha attivato il dialogo (come si vedrà al §5.4): le domande di ricerca non ne risentono, poiché nessuna delle misure dello studio riguarda i contenuti di questa scena.



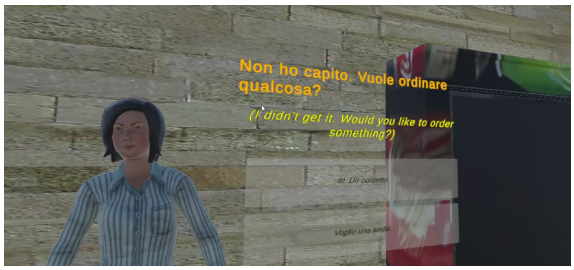
(a) La consegna della banconota. (b) Impalcatura dell'interazione.

Figura 4: La scena del chitarrista: il compito è riconoscibile, il gesto che lo avvia no.

03 Bar: La barista
È la scena centrale per la verifica delle quattro domande di ricerca. Rispetto al quartiere, introduce due modifiche strutturali deliberate. In primo luogo, in questo dialogo viene rimossa la traduzione dalle opzioni di risposta: le battute della barista restano accompagnate dalla traduzione inglese, ma le opzioni di risposta compaiono, sui pannelli e nel dialogo, soltanto in italiano (Figura 5). Il player deve orientarsi senza il supporto della L1, e riconoscere tra le opzioni il lessico già incontrato nelle scene precedenti: *grazie*, *sì*, *no*, *prego*, *scusi*. In secondo luogo, vi è la presenza di distrattori plausibili: non tutte le opzioni di risposta sono pragmaticamente corrette, e solo quella giusta consente di avanzare nel dialogo. I distrattori sono costruiti secondo i principi dell'*errorful learning* e della costruzione tecnica dei distrattori discussi al §2.4.7: sono realizzazioni plausibili della stessa funzione comunicativa in un registro errato, in un ordine sintattico sbagliato, o in un contesto pragmatico inappropriato. Errori, dunque, che un anglofono a livello A1 commetterebbe con una certa confidenza, proprio perché sembrano ragionevoli.



(a) *Desidera?*: le opzioni, senza traduzione, includono i distrattori.



(b) Il nodo di chiarimento raggiunto dopo un distrattore.

Figura 5: Il bar: la traduzione inglese accompagna le battute della barista, ma non le opzioni di risposta.

Questa scena prevede, inoltre, il sistema di *listen-and-repeat* sugli oggetti etichettati. Ciò è in linea con la funzione di *noticing* descritta al §2.4.6: il player pronuncia la forma target, Whisper la trascrive, il sistema verifica la corrispondenza. In caso di corrispondenza il nome dell'oggetto compare sull'etichetta; in caso contrario il giocatore può ritentare senza allontanarsi. Ogni tentativo è registrato da `VoiceRecognitionLogger`: testo trascritto, opzione attesa, esito del match. Questi dati sono la misura principale per la seconda domanda di ricerca e, incrociati con il post-test, anche per la terza. La presentazione dell'informazione in questa scena segue il principio di riduzione del carico estraneo discusso al §2.4.8: le etichette interattive sugli oggetti presentano una sola informazione alla volta nel momento in cui il player interagisce con l'oggetto, senza sovraccarico grammaticale o lessicale fuori contesto. Il confronto tra i risultati del post-test per le frasi del quartiere e quelle del bar è indicativo del contributo dei sottotitoli alla ritenzione mnemonica. Se le risposte date durante il dialogo al bar, pronunciate senza traduzione inglese, risultano meno memorizzate al post-test rispetto a quelle del quartiere, emerge un dato rilevante sul ruolo di supporto della L1, che le domande aperte del questionario integrano con la percezione soggettiva del partecipante. Va dichiarato che questo confronto non è conclusivo, perché le due scene variano simultaneamente per più variabili: NPC, registro, presenza di distrattori, presenza di sottotitoli. È un'indicazione, non una misura controllata.

04 Casa

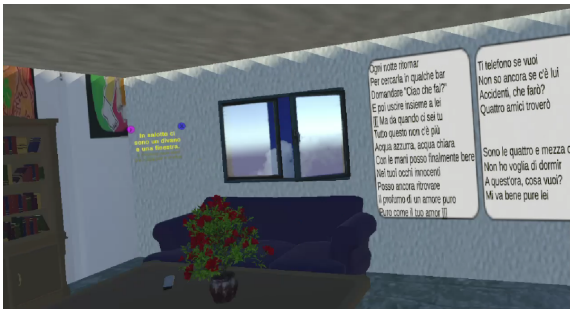
La scena domestica chiude il percorso come spazio di esplorazione ricettiva a bassa pressione comunicativa. Non prevede dialogo, produzione vocale né distrattori: il player si muove liberamente tra i quattro ambienti. Esplora un soggiorno, una camera da letto, la cucina, il bagno, e quando entra in una di queste zone compare un pannello per prossimità, attivato da un collider *trigger*. Ogni pannello presenta una frase descrittiva breve e contestualizzata, selezionata dai descrittori A1 del Profilo della Lingua Italiana per la funzione *Descrivere*, come *C'è una cucina.*, *C'è un letto.* o *In salotto ci sono un divano e una finestra.* (Figura 6). La frase è accompagnata da una clip audio e dalla traduzione della frase in inglese; il pannello scompare quando il player lascia la zona.



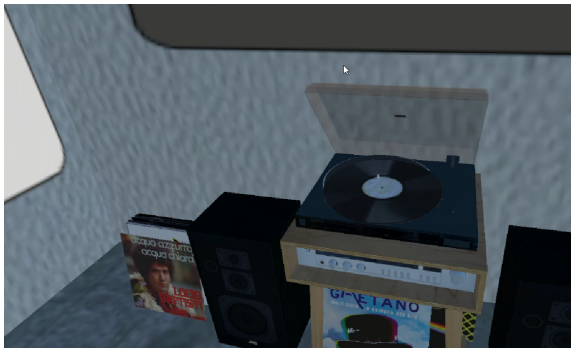
(a) *C'è una cucina.*



(b) *C'è un letto.*



(c) *In salotto ci sono un divano e una finestra.*



(d) Il giradischi.

Figura 6: La casa: pannelli descrittivi per prossimità ed esplorazione libera.

La configurazione multimodale di testo scritto, clip audio e contesto spaziale replica il principio della *Dual Coding Theory* discusso al §2.4.5: tre canali di codifica simultanei che si rinforzano a vicenda. La scena non contribuisce alla misurazione dei dati per le domande di ricerca attraverso dati comportamentali automatici, ma offre un momento di decompressione lessicale dopo la pressione del bar, coerente con il principio dello *scaffolding* decrescente (§2.4.3): ciò che nelle scene precedenti richiedeva produzione, qui viene semplicemente ricevuto. Il giradischi nel soggiorno è l'unico elemento interattivo aggiuntivo: posizionando il disco sul piatto, parte *Acqua azzurra, acqua chiara* di Lucio Battisti, accompagnata da una trascrizione animata che evidenzia strutture specifiche

in sincronia con il canto (Figura 7). La selezione della canzone e la scelta dei passaggi trascritti seguono i criteri di Mezzadri[79] nel *Cantagramma*, che isola le frasi in cui il sistema dei pronomi personali emerge con chiarezza dal contesto musicale. Il giradischi permette di riprodurre anche una serie di canzoni aggiuntive, al solo scopo di aumentare la densità ambientale della scena. La porta può essere aperta, la finestra può chiudersi, la televisione si accende, la padella in cucina è afferrabile: è dunque disponibile una serie di oggetti interagibili non aventi funzione didattica diretta, ma che contribuiscono alla credibilità dello spazio.

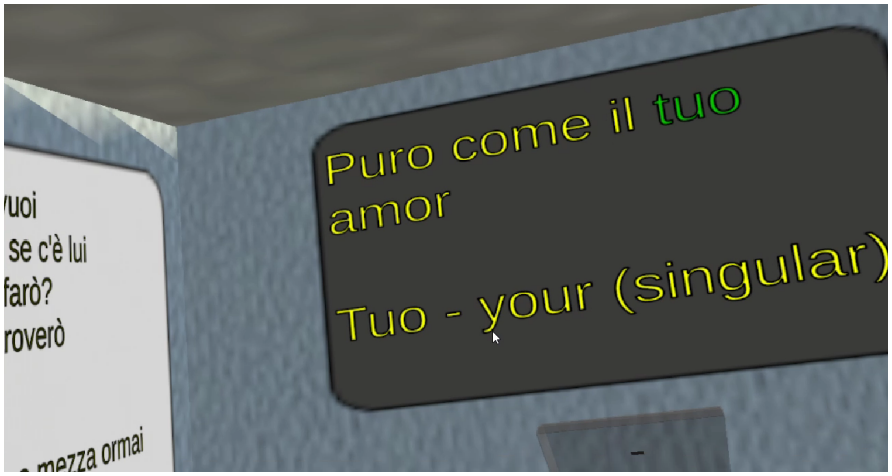


Figura 7: Trascrizione animata di *Acqua azzurra, acqua chiara*, con l'evidenziazione del pronome possessivo.

Accanto alle funzioni didattiche descritte, numerosi elementi dell'ambiente rispondono a un obiettivo trasversale: rafforzare il senso di presenza. Gli oggetti afferrabili distribuiti alla stazione, al bar e in casa, il giradischi e la scena del chitarrista ampliano le possibilità di interazione libera al di fuori dei compiti linguistici, e rendono Bellavalle un luogo da esplorare e non soltanto un percorso da completare. Nei termini di Steuer (§2.1), si tratta di elementi che agiscono sull'*interactivity*, cioè sulla capacità dell'utente di incidere sull'ambiente, accanto alla *vividness* garantita dagli asset e dall'audio (§4.2). Il senso di presenza così costruito non è un fine in sé: è la condizione perché la pressione comunicativa delle scene risulti credibile, e il suo effettivo raggiungimento è verificato al §5.3.

[79] Marco Mezzadri, *Cantagramma: apprendere la grammatica italiana con le canzoni* (Perugia: Guerra Edizioni, 2006): pp. 51–58.

4 Implementazione tecnica

I capitoli precedenti hanno definito il problema, giustificato le scelte didattiche e descritto la struttura dell'esperienza. Questo capitolo entra nell'implementazione: documenta come i sottosistemi di *ItalianVRExperience* sono stati costruiti, quali strumenti sono stati adottati e perché, e come si integrano tra loro. La descrizione fa riferimento agli script C# del progetto, identificati per nome ogni volta che la scelta implementativa è sufficientemente specifica da meritare documentazione diretta. Gli *snippet* presentati nelle sotto-sezioni sono riportati per intero, insieme agli altri script più significativi, in Appendice C[80]. Il §4.1 presenta l'ambiente di sviluppo e il visore. Il §4.2 descrive la produzione dei contenuti: ambienti, modelli e animazioni. Il §4.3 documenta il sistema di *hand tracking* e i gesti implementati. Il §4.4 presenta l'architettura del sistema di dialogo, con il riconoscimento vocale tramite Whisper e il sistema *listen-and-repeat*. Il §4.5 descrive l'inventario linguistico e il tutor grammaticale, basato sul modello linguistico Mistral eseguito localmente tramite Ollama. I §4.6 e §4.7, infine, trattano la persistenza dello stato di gioco e il sistema di *logging*.

4.1 L'ambiente di sviluppo: Unity 6 e Meta Quest 2

ItalianVRExperience è sviluppato in Unity 6 (versione 6000.0.4f1), motore di sviluppo scelto per la sua compatibilità nativa con il visore Meta Quest 2 tramite il *package* XR Plug-in Management con backend OpenXR — lo standard aperto per lo sviluppo di applicazioni in realtà virtuale e aumentata. Unity gestisce il *rendering* stereoscopico dell'ambiente tridimensionale, la simulazione fisica degli oggetti, il sistema di animazione degli NPC e l'intera pipeline di interazione con l'utente. Il codice è scritto in C# e organizzato in classi separate, ciascuna responsabile di un sottosistema specifico: una scelta che facilita la manutenzione e il *testing* incrementale durante lo sviluppo iterativo descritto al §3.1. Le classi sono raggruppate in *namespace* che rispecchiano questa suddivisione, come `Bellavalle.Voice` per il riconoscimento vocale, `Bellavalle.Scene` per le scene e il dialogo, `Bellavalle.UI` per l'interfaccia e `Bellavalle.Data` per i dati dei dialoghi.

[80]Per leggibilità, negli *snippet* sono state omesse le righe non pertinenti alla discussione, come i messaggi di debug e i controlli di errore; dove l'omissione interrompe il flusso del codice, è segnalata da tre punti

Il visore adottato è il Meta Quest 2, un dispositivo *standalone* —non richiede cioè obbligo di connessione a un PC per funzionare— dotato di display stereoscopico a 90Hz e di un sistema di tracciamento spaziale che rileva la posizione e l'orientamento della testa del player nell'ambiente fisico. Quest 2 rende possibile l'*hand tracking* senza controller fisici, condizione che si è allineata perfettamente con l'architettura di interazione discussa al §4.3. Il *build* finale del progetto è un'applicazione Android, uno dei sistemi operativi su cui gira il Quest 2, compilata tramite il modulo Android Build Support di Unity e installata direttamente sul visore utilizzando il software SideQuest. Durante lo sviluppo, il progetto è stato testato tramite Quest Link, una modalità che collega il visore al PC via cavo o wireless, permettendo di eseguire l'applicazione direttamente dall'editor Unity senza *compilare* un *build* completo.

4.2 Panoramica sulla produzione dei contenuti

Gli ambienti di *ItalianVRExperience* sono assemblati a partire da asset tridimensionali scaricati da Unity Asset Store e Sketchfab, selezionati e integrati in scena per costruire spazi coerenti e riconoscibili. La scelta degli asset è stata guidata da un obiettivo preciso: rispecchiare la cultura italiana dove e quando possibile. La stazione ospita un treno Trenitalia, al bar campeggiano una moka e molti altri elementi della «pausa» all'italiana, le facciate della città portano dipinti in stile artistico prettamente italiano. L'autenticità culturale dell'ambiente è una scelta didattica prima ancora che estetica: un anglofono che entra a Bellavalle deve sentire di essere in Italia, perché è quella familiarità visiva e contestuale a rendere credibile la pressione comunicativa che il gioco esercita.

L'audio è prodotto su più livelli. Le clip dei dialoghi degli NPC sono generate con ElevenLabs, un sistema di sintesi vocale che permette di ottenere voci italiane naturali e calibrate per la velocità pedagogica richiesta dal progetto. Le clip che leggono i pannelli di *onboarding*, in inglese, sono invece registrate da un parlante anglofono madrelingua: le istruzioni arrivano così al giocatore nella sua lingua e con una voce naturale, mentre la voce sintetica resta riservata all'input in italiano. La musica di ambiente è un contributo originale composto e registrato appositamente per il gioco, presente in tutte le scene. Alla stazione, l'audio di sottofondo è una registrazione reale di una stazione italiana: un dettaglio che aggiunge autenticità all'ambiente più istituzionale del percorso. Per evitare interferenze con la trascrizione vocale e rispondere alle esigenze di Whisper, tutte le sorgenti audio della scena vengono messe in pausa mentre il microfono è attivo e riprendono al termine della registrazione (§4.3).

4.2.1 Costruzione degli ambienti

Ciascuna delle quattro scene di *ItalianVRExperience* è costruita come una scena Unity separata, cioè come un ambiente autonomo con i propri oggetti, luci e comportamenti. Le scene vengono caricate in sequenza tramite `SceneManager`, il modulo di Unity che gestisce il passaggio da una scena all'altra, al completamento della scena precedente.

In Unity ogni elemento di una scena è un *GameObject*, un contenitore a cui si associano componenti che ne definiscono aspetto e comportamento; i *GameObject* possono essere annidati gli uni negli altri, formando una struttura gerarchica. In ciascuna scena questa gerarchia segue una logica per funzione: gli elementi puramente decorativi sono raggruppati sotto contenitori distinti da quelli con cui il giocatore può interagire, e questi ultimi sono a loro volta suddivisi per ruolo, separando per esempio le zone di attivazione dai componenti che gestiscono i dialoghi. Questa organizzazione facilita la manutenzione e il *testing* incrementale descritto al §3.1.

L'illuminazione è gestita in tempo reale (*real-time lighting*): una luce direzionale (*directional light*), che illumina l'intera scena da un'unica direzione, simula la luce naturale negli esterni, mentre luci puntiformi e a cono (*point* e *spot light*) illuminano gli interni del bar e della casa. L'alternativa sarebbe stata il *baked lighting*, in cui l'illuminazione viene precalcolata e salvata in apposite mappe: una soluzione più leggera in esecuzione, ma che richiede di ricalcolare le mappe a ogni modifica dell'ambiente. L'illuminazione in tempo reale, pur più onerosa, ha permesso di mantenere le scene modificabili durante tutto lo sviluppo iterativo.

In Unity, un *collider trigger* è un volume invisibile associato a un oggetto della scena, che non produce collisioni fisiche ma rileva l'ingresso e l'uscita di altri oggetti, consentendo di associare a questi eventi un comportamento programmato. Nel sistema, i *collider trigger* sono impiegati in due contesti distinti. Il primo è la prossimità narrativa: nelle scene in cui il dialogo non viene avviato da un evento di gioco, è l'ingresso del giocatore in una zona delimitata da un *collider trigger* ad attivarlo. Tale ingresso è rilevato da `SceneDirector`, come descritto al §4.4. Il secondo è la prossimità lessicale nella scena della casa: ciascun ambiente domestico è delimitato da un *collider trigger* che, all'ingresso del giocatore, attiva il pannello descrittivo corrispondente e lo disattiva all'uscita. Questo meccanismo garantisce che il giocatore riceva un'informazione alla volta, nel momento in cui è contestualmente rilevante, in coerenza con il principio di riduzione del carico cognitivo estraneo discusso al §2.4.8.

I modelli tridimensionali provengono dall'Asset Store di Unity e dalla piattaforma Sketchfab, che raccolgono modelli gratuiti e a pagamento distribuiti con licenze d'uso specifiche. Alcuni modelli hanno richiesto interventi in Blender, software libero di modellazione tridimensionale, per uniformarne la scala e l'orientamento degli assi a quelli degli altri oggetti in scena, o per sostituire materiali non supportati dalla *pipeline* di rendering adottata in Unity. I modelli così corretti sono stati esportati in formato `.fbx`, il formato di interscambio più diffuso per l'importazione di modelli tridimensionali in Unity.

4.2.2 Animator controller

I modelli degli NPC sono stati realizzati principalmente con Adobe Fuse CC, un software per la creazione di personaggi tridimensionali, e in minima parte sono stati importati in Unity come modelli pre-esistenti. Tutti gli NPC possiedono un *rig* umanoide standard,

cioè con uno scheletro virtuale che ne consente l'animazione, compatibile con Mecanim, il sistema di animazione di Unity. Ciascun NPC è dotato di un *Animator Controller*, una macchina a stati che definisce le animazioni disponibili e le condizioni di passaggio dall'una all'altra. Gli stati previsti sono `Idle`, la posa di attesa quando il dialogo non è attivo; `Talking`, l'animazione di parlato riprodotta insieme alle clip audio; e, dove la scena lo richiede, `Walking`, per i personaggi di sfondo in movimento.

Le animazioni provengono dalla libreria gratuita Adobe Mixamo e dall'Asset Store di Unity, e sono state applicate ai modelli tramite il *retargeting* di Mecanim: un meccanismo che adatta un'animazione realizzata per uno scheletro generico alle proporzioni del singolo personaggio, rendendola riutilizzabile su modelli diversi senza doverla ricreare. Il passaggio da uno stato all'altro è regolato da parametri dell'*Animator Controller*, impostati da `DialogueManager` in corrispondenza degli eventi del dialogo. All'avvio della clip audio dell'NPC viene attivato il parametro booleano `isTalking`, che porta il personaggio dallo stato `Idle` allo stato `Talking`; al termine della clip il parametro viene disattivato e l'NPC torna alla posa di attesa. Per rendere possibile questo collegamento, all'avvio di ciascuna scena narrativa `SceneDirector` passa a `DialogueManager` un riferimento diretto all'*Animator Controller* dell'NPC coinvolto (si veda, per esempio, `LinaSequence.cs`).

4.3 Il sistema di hand-tracking

Il sistema di *hand tracking* di ItalianVRExperience è gestito tramite il package XR Hands di Unity, che espone i dati di posizione e rotazione di ogni singola articolazione della mano rilevata dal Quest 2: ventisette joint per mano, dal polso alle punte delle dita. È su questi dati che si appoggiano tutti i gesti del gioco.

La locomozione è gestita da `FistDynamicMoveProvider`, una classe custom che estende `DynamicMoveProvider` del toolkit XR. Il principio è semplice: se almeno uno dei due palmi è rivolto verso l'alto, il player avanza nella direzione in cui guarda. La rilevazione avviene leggendo la rotazione del joint del palmo e calcolando il prodotto scalare con il vettore `Vector3.up`. Se il valore supera una soglia configurabile in Inspector, il gesto è considerato attivo. Rivolgere entrambi i palmi verso il basso ferma il movimento.

L'attivazione del microfono è affidata a `GestureSpeechInput`, componente persistente tra le scene. Poiché `FistDynamicMoveProvider` e `InventoryUI` non sono persistenti ma vengono ricreati a ogni scena, `GestureSpeechInput` aggiorna automaticamente i propri riferimenti a ogni cambio scena tramite `SceneManager.sceneLoaded`, recuperandoli con `FindFirstObjectByType`. Il gesto adottato è `PuntaSu`: l'indice puntato verso l'alto con le altre dita raccolte, rilevabile su entrambe le mani. La rilevazione (Listato 1) verifica due condizioni: che la direzione dell'indice, calcolata tra la base e la punta del dito, formi con `Vector3.up` un prodotto scalare superiore a una soglia configurabile in Inspector, e che medio, anulare e mignolo siano chiusi, cioè che la loro punta disti dal centro del palmo meno di sei centimetri.

```
bool IsPointingUp(XRHand hand)
{
        if (!hand.isTracked) return false;
        var indexTip = hand.GetJoint(XRHandJointID.IndexTip);
        var indexProximal = hand.GetJoint(XRHandJointID.IndexProximal);
        if (!indexTip.TryGetPose(out Pose tipPose)) return false;
        if (!indexProximal.TryGetPose(out Pose proxPose)) return false;

        // L'indice deve puntare verso l'alto...
        Vector3 indexDir = (tipPose.position - proxPose.position).normalized;
        if (Vector3.Dot(indexDir, Vector3.up) <= pointUpThreshold) return
        ↪  false;

        // ...e medio, anulare e mignolo devono essere chiusi verso il palmo
        var palmJoint = hand.GetJoint(XRHandJointID.Palm);
        if (!palmJoint.TryGetPose(out Pose palmPose)) return false;
        return IsFingerCurled(hand, XRHandJointID.MiddleTip,
        ↪  palmPose.position)
        && IsFingerCurled(hand, XRHandJointID.RingTip, palmPose.position)
        && IsFingerCurled(hand, XRHandJointID.LittleTip, palmPose.position);
}
```

Listato 1: Rilevamento del gesto `PuntaSu` in `GestureSpeechInput`

Il riconoscimento del gesto non apre subito il microfono (Listato 2). Quando la posa viene rilevata, `GestureSpeechInput` avvia un'attesa di un secondo (`micStartDelay`); se la mano abbandona la posa prima dello scadere, l'attivazione viene annullata. L'attesa svolge una doppia funzione: impedisce che un gesto accidentale e fugace apra il microfono, e fa sì che la registrazione non catturi ciò che il giocatore dice prima della frase italiana, come commenti o esitazioni. Allo scadere, `StartListening` mette in pausa tutte le sorgenti audio della scena, così che musica e voci degli NPC non interferiscano con la trascrizione, disabilita la locomozione per evitare che il player si muova involontariamente mentre parla e avvia la registrazione tramite `PushToTalkRecorder`. Al termine di ogni battuta dell'NPC, un segnale sonoro indica al giocatore che è il suo turno. Il segnale è stato introdotto dopo le due sessioni pilota (§5.2.1), nelle quali i partecipanti avevano mostrato difficoltà a capire quando cominciare a parlare. Nella versione impiegata durante lo studio, il segnale marca l'inizio del turno e non l'apertura del microfono: dopo averlo udito, il giocatore esegue il gesto, e la registrazione parte soltanto allo scadere dell'attesa di un secondo.

```
if (!_listening)
{
        if (IsTargetGesture(_handSubsystem.leftHand) ||
        ↪  IsTargetGesture(_handSubsystem.rightHand))
        {
                if (_startDelayCoroutine == null)
                _startDelayCoroutine =
                ↪  StartCoroutine(StartListeningDelayed());
        }
        else if (_startDelayCoroutine != null)
        {
                // Gesto rilasciato durante l'attesa: il microfono non si
                ↪  apre
                StopCoroutine(_startDelayCoroutine);
                _startDelayCoroutine = null;
        }
}
else if (IsPalmDown(_handSubsystem.leftHand) &&
↪  IsPalmDown(_handSubsystem.rightHand)
&& Time.time - _recordingStartTime >= minRecordingTime)
{
        StopListening();
}
```

Listato 2: Apertura ritardata e chiusura manuale del microfono in `GestureSpeechInput.Update`

La registrazione si chiude in due modi. Il primo è manuale: il giocatore rivolge entrambi i palmi verso il basso, gesto accettato solo dopo almeno due secondi di registrazione (`minRecordingTime`), così che un movimento involontario delle mani non tronchi l'enunciato appena iniziato. Il secondo è automatico: `PushToTalkRecorder` interrompe da solo l'acquisizione quando rileva silenzio. In entrambi i casi il ripristino dello stato è affidato a un unico metodo, `HandleRecordingStopped`, che riprende le sorgenti audio e riattiva la locomozione solo se l'inventario non è aperto. La posizione con i palmi in giù ferma anche il movimento, poiché `FistDynamicMoveProvider` non rileva più il gesto di locomozione: i due effetti derivano da due rilevazioni distinte sulla medesima postura.

La visualizzazione delle mani in scena è affidata all'Hand Visualizer del package XR Hands, che renderizza in tempo reale un modello tridimensionale delle mani, animato dai dati del tracciamento, senza richiedere script custom. Nel corso dello sviluppo sono stati invece dismessi cinque script che tentavano di collegare le mani tracciate a un modello umanoide completo. L'obiettivo era offrire al giocatore un corpo intero, visibile riflettendosi in uno specchio della scena, e sfruttare proprio lo specchio come punto di partenza di una missione dedicata ai gesti della comunicazione non verbale italiana.

L'idea è stata accantonata insieme agli script, e nella versione attuale il giocatore dispone soltanto delle proprie mani: un caso esemplare del ciclo iterativo.

4.4 Il sistema di dialogo

Il sistema di dialogo di *ItalianVRExperience* è costruito attorno a tre componenti principali, ciascuno con una responsabilità precisa e ben separata dalle altre: `SceneDirector` decide quando un dialogo comincia, `DialogueManager` ne governa lo svolgimento, `DialogueUI` lo rende visibile al giocatore.

Il punto di ingresso di ogni scena narrativa è `SceneDirector`. Il componente monitora la prossimità del player tramite un *collider trigger* e, quando le condizioni sono soddisfatte, cioè quando i flag di gioco richiesti sono attivi e la scena non è già stata completata, avvia il dialogo passando a `DialogueManager` l'albero di dialogo corretto e il nodo di ingresso. Allo stesso componente spetta la chiusura della scena: la progressione tra scene è gestita tramite un `EventBus` statico, e quando un dialogo termina `SceneDirector` riceve l'evento `DialogueEnded` e chiede a `GameManager` di registrare la missione come completata.

Una volta avviato il dialogo, il controllo passa a `DialogueManager`, che ne è il motore. Per ogni nodo, il componente riproduce la clip audio dell'NPC nella versione pedagogica lenta, attende la fine dell'audio e solo allora mostra le opzioni di risposta. Quando il player sceglie un'opzione, `DialogueManager` registra la risposta in `GameManager` e avanza al nodo successivo. L'avanzamento può essere comandato anche dall'esterno: un metodo pubblico, `GoToNodePublic`, consente ad altri sistemi di portare il dialogo a un nodo specifico senza passare per la selezione delle opzioni. Su questo meccanismo si basa la consegna del biglietto nella scena della stazione: nessun bottone fa avanzare il dialogo, perché a farlo è il gesto stesso di porgere il biglietto al controllore, rilevato da un *handler* separato che chiama questo metodo.

La parte visibile di questo processo è affidata a `DialogueUI`. Il suo metodo `ShowOptions`, invocato da `DialogueManager` a ogni nodo, aggiorna in sequenza la battuta dell'NPC in italiano, il sottotitolo inglese quando l'asset lo prevede e i bottoni delle opzioni di risposta, generati dinamicamente (Appendice C). Non tutte le opzioni hanno lo stesso statuto: quelle con cui il giocatore chiede di ripetere o segnala di non aver capito (*Scusi?* con il controllore, *Non ho capito* con il chitarrista, *Prego?* con Lina) vengono rese visivamente distinte tramite `StyleAsHelpButton`, che le mostra in grigio e a dimensione ridotta, segnalando al player che si tratta di un'opzione di supporto e non di una risposta comunicativa.

La differenza di comportamento tra la scena del quartiere e quella del bar non è accidentale: risponde a un disegno sperimentale preciso, discusso nel dettaglio al §5.1. Nel quartiere tutte le opzioni di risposta sono pragmaticamente corrette e il dialogo avanza a prescindere dalla scelta effettuata; al bar i distrattori plausibili bloccano

l'avanzamento in caso di selezione errata. Sul piano tecnico, questa distinzione è implementata nella struttura degli alberi di dialogo, definiti negli asset `NPCData`: nel quartiere tutte le opzioni conducono al nodo successivo, mentre al bar ciascun distrattore conduce a un nodo di chiarimento, in cui la barista segnala di non aver capito e propone nuove opzioni di risposta, a loro volta comprensive di un distrattore (Appendice). Se invece la trascrizione non corrisponde ad alcuna opzione, il dialogo resta sul nodo corrente e il giocatore può ripetere la risposta. Anche la presenza della traduzione inglese è configurata per asset: essa compare solo quando il nodo ne contiene il testo, e nei nodi del bar il campo è stato lasciato vuoto di proposito, in coerenza con il gradiente di supporto discusso al §3.5. Entrambe le variabili, presenza dei distrattori e/o della traduzione, contribuiscono alla struttura dei dati raccolti dal log di sessione e alle analisi descritte al §5.2.

La cattura del nome del player è gestita da `NameInputHandler`, che si attiva su qualsiasi nodo con `isNameCapture = true` nell'asset `NPCData` tramite il sistema di eventi: quando `DialogueManager` emette `NodeEntered`, `NameInputHandler` intercetta il nodo, mostra al giocatore l'invito a pronunciare il proprio nome e si mette in attesa della trascrizione. La trascrizione prodotta da Whisper viene passata a `CleanTranscribedName`, che rimuove la punteggiatura finale, individua e taglia i prefissi formulaici più comuni in italiano e in inglese, come *mi chiamo* o *my name is*, e capitalizza il risultato (Appendice C).

Se il testo risultante è vuoto, troppo lungo o inizia con una parentesi quadra (segnale tipico di un errore di trascrizione Whisper), il sistema chiede di ripetere. Altrimenti il nome viene salvato in `GameState.playerName` e da quel momento tutti gli NPC possono recuperarlo tramite il template `{playerName}` nelle battute successive.

4.4.1 Il riconoscimento vocale: Whisper

Il riconoscimento vocale è affidato a Whisper di OpenAI, eseguito interamente in locale tramite il modello `ggml-small` senza dipendenza da connessioni di rete. Il GameObject che ospita `WhisperManager`, `MicrophoneRecord` e `PushToTalkRecorder` è reso persistente tra le scene tramite `PersistentWhisper`, che implementa il pattern singleton con `DontDestroyOnLoad`, la stessa architettura del `GameManager`.

La pipeline di registrazione è gestita da `PushToTalkRecorder`. Quando `GestureSpeechInput` conferma il gesto dell'indice puntato verso l'alto (§4.3), avvia la registrazione tramite `MicrophoneRecord`. Prima di avviarla, `PushToTalkRecorder` verifica che `WhisperManager` abbia completato il caricamento del modello tramite la guardia `whisperManager.IsLoaded`: nei primi secondi di esecuzione, prima che il modello sia pronto, il gesto viene ignorato. Quando la registrazione si chiude, con i palmi rivolti verso il basso oppure per silenzio, l'audio viene passato in modo asincrono a `WhisperManager` (Listato 3).

```
async void OnRecordStopHandler(AudioChunk recordedAudio)
{
        if (recordedAudio.Length < 0.3f) return;

        var result = await whisperManager.GetTextAsync(
        recordedAudio.Data, recordedAudio.Frequency, recordedAudio.Channels);

        if (result == null || string.IsNullOrWhiteSpace(result.Result))
        ↪  return;

        string text = result.Result.Trim();
        OnTranscriptionReady?.Invoke(text);
}
```

Listato 3: Invio dell'audio registrato a Whisper in `PushToTalkRecorder`.

Le registrazioni più brevi di 0,3 secondi vengono scartate: il filtro elimina le attivazioni brevissime, come un gesto riconosciuto per un istante, ma non quelle involontarie che aprono il microfono per un tempo più lungo, che restano documentate nella categoria MIC (§5.3).

La trascrizione prodotta da Whisper viene poi confrontata con le opzioni di risposta attese dal nodo di dialogo tramite `VoiceMatchUtils`. È qui che si gioca la partita più delicata del sistema. Un principiante anglofono che pronuncia italiano per la prima volta produrrà quasi certamente una forma fonetica distante dalla norma: accento anglofono marcato, vocali ridotte, consonanti geminate assenti. Richiedere una corrispondenza esatta tra trascrizione e risposta attesa significherebbe bloccare sistematicamente il player su errori di pronuncia normali e attesi nelle prime fasi di acquisizione.

La soluzione adottata è un *matching* tollerante basato su normalizzazione testuale. `VoiceMatchUtils.Normalize` (Listato 4) converte tutto in minuscolo, rimuove gli accenti grafici e sostituisce la punteggiatura con spazi: così *sì* combacia con *si*, *perché* con *perche*, e le varianti trascritte da Whisper con accenti ridotti o assenti trovano comunque corrispondenza. La punteggiatura diventa uno spazio anziché sparire, per evitare che due parole separate da una virgola vengano fuse in una sola.

```
public static string Normalize(string s)
{
        if (string.IsNullOrWhiteSpace(s)) return string.Empty;

        string lower = s.ToLowerInvariant();
        lower = lower
        .Replace('à', 'a').Replace('è', 'e').Replace('é', 'e')
        .Replace('ì', 'i').Replace('ò', 'o').Replace('ù', 'u');

        // La punteggiatura diventa spazio, non sparisce
        // (evita di fondere due parole)
        ...
        // Gli spazi multipli vengono compressi in uno solo
        return collapsed.ToString();
}
```

Listato 4: Normalizzazione del testo in `VoiceMatchUtils`

Il *matching* vero e proprio (Listato 5) cerca il *trigger* come sotto-sequenza di parole intere nella trascrizione, aggiungendo uno spazio ai due estremi di entrambe le stringhe: in questo modo *si* non viene riconosciuto dentro *così*.

```
public static bool Matches(string transcribedRaw, string triggerRaw)
{
        string transcribed = Normalize(transcribedRaw);
        string trigger = Normalize(triggerRaw);
        if (transcribed.Length == 0 || trigger.Length == 0) return false;

        // Gli spazi ai bordi impediscono match parziali dentro altre parole
        return (" " + transcribed + " ").Contains(" " + trigger + " ")
        || transcribed == trigger;
}
```

Listato 5: Confronto tra trascrizione e *trigger* in `VoiceMatchUtils`

Ogni opzione di risposta nel nodo di dialogo ha uno o più `voiceTriggers`, un array di stringhe definito nell'asset `NPCData` per ciascuna `PlayerOption`. Quando Whisper restituisce la trascrizione, `VoiceMatchUtils.FindMatchingOption` scorre le opzioni del nodo corrente e, per ciascuna, verifica tramite `Matches` se almeno uno dei suoi *trigger* compare nel testo trascritto come sequenza di parole intere. La prima corrispondenza trovata determina quale opzione è stata scelta dal giocatore e, di conseguenza, quale nodo di dialogo viene raggiunto. Nella maggior parte dei casi i *trigger* coincidono con le frasi mostrate nei pannelli, e il sistema accetta quindi ciò che il giocatore ha letto e ascoltato prima di avvicinarsi all'NPC; per alcune opzioni sono state aggiunte come

trigger anche singole parole chiave, come *grazie* per *Ciao, grazie!* o *arrivederci* per *No. Arrivederci!*. Il confronto non prevede alcuna misura di somiglianza tra stringhe: tutta la tolleranza fonetica è affidata alla normalizzazione descritta sopra. I log confermano questo funzionamento: dei 199 tentativi accettati nei dialoghi, 196 contengono per intero il testo di un'opzione del nodo, e i restanti tre sono stati riconosciuti tramite una parola chiave. Se Whisper trascrive la pronuncia del giocatore in una forma che, una volta normalizzata, contiene uno dei *trigger*, il riconoscimento riesce; in caso contrario, fallisce. `VoiceRecognitionLogger` registra entrambi i casi, producendo i dati su cui si costruisce l'analisi della seconda domanda di ricerca al §5.3.

L'ordine di scansione ha una conseguenza che emerge nei risultati (Listato 6): `FindMatchingOption` restituisce la prima opzione il cui *trigger* compare nella trascrizione, senza distinguere tra risposta corretta e distrattore. Se una parola chiave è presente in più opzioni dello stesso nodo, il sistema può quindi selezionare un distrattore anche quando il giocatore intendeva pronunciare la risposta corretta, come discusso al §.

```
public static int FindMatchingOption(string transcribed, PlayerOption[]
↪  options)
{
        if (options == null) return -1;

        for (int i = 0; i < options.Length; i++)
        {
                var triggers = options[i].voiceTriggers;
                if (triggers == null) continue;

                foreach (var trig in triggers)
                if (Matches(transcribed, trig))
                return i;   // vince la prima opzione che combacia
        }
        return -1;
}
```

Listato 6: Selezione dell'opzione riconosciuta in `VoiceMatchUtils`.

Questa architettura introduce però una tensione metodologica che è al centro della seconda domanda di ricerca: quanto la tolleranza del *matching* compensa le difficoltà fonetiche dell'anglofono, e quanto invece le maschera? Un sistema troppo permissivo accetta trascrizioni che non corrispondono a una produzione orale accettabile; uno troppo rigido penalizza errori normali e attesi. `VoiceRecognitionLogger` è lo strumento che permette di esplorare questa tensione empiricamente: registra per ogni tentativo il testo trascritto, le frasi attese e l'esito del match, consentendo di identificare i pattern di errore ricorrenti. Sono questi i dati su cui si costruisce l'analisi del §5.

4.4.2 Il sistema listen-and-repeat

La scena del bar introduce una modalità di interazione vocale distinta dal dialogo con gli NPC: il *listen-and-repeat*. Il sistema descritto al §4.4 gestisce scambi conversazionali su nodi predefiniti con trigger multipli; qui l'obiettivo è la produzione orale isolata di una singola parola associata a un oggetto dell'ambiente. Il giocatore si avvicina a un oggetto etichettato, ascolta la pronuncia target tramite clip audio e la ripete ad alta voce; il sistema verifica la corrispondenza e restituisce un feedback visivo immediato. Come la missione del chitarrista, l'esercizio è facoltativo: il giocatore sceglie liberamente quali oggetti avvicinare e quante parole pronunciare, senza che questo condizioni l'avanzamento nella scena.

Componenti e responsabilità

Il sistema è distribuito su tre componenti. `ObjectCanvasActivator` gestisce apertura e chiusura del canvas informativo: l'hover della mano sull'oggetto, rilevato tramite `XRBaseInteractable`, attiva il canvas; l'uscita dall'hover lo disattiva. `ListenAndRepeatActivator` risiede sullo stesso GameObject e controlla la logica vocale: all'apertura, nasconde il testo del nome dell'oggetto in modo che il player incontri prima la forma sonora, e si mette in ascolto della trascrizione. `GestureSpeechInput` coordina il gesto PuntaSu con `PushToTalkRecorder`: quando la registrazione parte, notifica `ListenAndRepeatActivator` tramite `NotifyRecordingStarted()`, che imposta il flag interno `_waitingForTranscription` a `true`.

Per garantire che un solo oggetto alla volta sia in stato attivo, `ListenAndRepeatActivator` mantiene un riferimento statico `_activeInstance` all'istanza correntemente aperta: a ogni nuovo hover, l'istanza precedente viene chiusa prima di aprire quella corrente, senza alcun coordinatore esterno.

Il punto di convergenza: la *callback* di trascrizione

Il momento in cui i tre componenti si incrociano è la *callback* abbonata all'evento `OnTranscriptionResultReady` di `PushToTalkRecorder` (Listato 7).

```
void OnTranscriptionResultReady(WhisperResult result)
{
        if (!_isOpen) return;
        if (dialogueManager != null && dialogueManager.IsActive) return;
        if (!_waitingForTranscription) return;

        _waitingForTranscription = false;
        _attemptNumber++;

        string transcribed = (result.Result ?? "").Trim().ToLowerInvariant();
        bool isMatch = transcribed.Contains(targetWord.ToLowerInvariant());

        logger?.LogListenAndRepeat(targetWord, _attemptNumber, isMatch,
        ↪  result);

        if (_feedbackCoroutine != null) StopCoroutine(_feedbackCoroutine);
        _feedbackCoroutine = StartCoroutine(ShowFeedback(isMatch));
}
```

Listato 7: Verifica della parola pronunciata in `ListenAndRepeatActivator`.

Le tre guardie in apertura definiscono le condizioni di attivazione: il canvas deve essere aperto (`_isOpen`), nessun dialogo NPC deve essere in corso (`dialogueManager.IsActive`), e il gesto deve essere stato eseguito (`_waitingForTranscription`). L'ultima condizione impedisce che trascrizioni prodotte durante un dialogo NPC vengano elaborate dal sistema listen-and-repeat, che condivide la stessa pipeline vocale. Superati i controlli, il contatore di tentativi viene incrementato, la trascrizione confrontata con la parola target e il risultato passato al logger prima di avviare il *feedback* visivo.

Matching e feedback

Il confronto non passa per `VoiceMatchUtils`: la trascrizione viene soltanto convertita in minuscolo, e il sistema verifica con un `Contains` che la parola target vi compaia come sottostringa. Si tratta di un criterio più permissivo di quello dei dialoghi, perché non richiede che la parola sia isolata: *tazza* viene riconosciuta anche all'interno di una trascrizione come *la tazza*. Per la produzione di una singola parola lessicale, la scelta è sufficiente; l'assenza della normalizzazione degli accenti, d'altra parte, non ha effetti pratici, poiché nessuna delle parole target contiene vocali accentate.

Sul piano del *feedback*, un match rivela il nome dell'oggetto nel canvas con il colore verde, che poi si stabilizza sul bianco: l'oggetto è acquisito. Un mismatch mostra il testo brevemente in rosso e lo nasconde di nuovo, lasciando al player la possibilità di ritentare senza allontanarsi. La durata del *feedback* è configurabile nell'Inspector tramite `feedbackDuration`.

Logging
Ogni tentativo viene registrato nel CSV tramite `VoiceRecognitionLogger.LogListenAndRepeat`, con la parola target, il numero progressivo del tentativo e l'esito. In fase di analisi, i record dell'esercizio sono distinguibili dagli altri tentativi vocali della stessa scena grazie al campo `InteractionType`, che assume il valore `listen_repeat` anziché `dialogo`. Il campo `Scene`, aggiornato automaticamente da `VoiceRecognitionLogger` a ogni cambio di scena, colloca invece ciascun record nella scena del bar.

4.5 Il tutor AI e l'inventario linguistico

L'inventario è un pannello di interfaccia collocato nello spazio tridimensionale della scena (*world-space canvas*), anziché sovrapposto alla visuale come un menu tradizionale. Si apre con un gesto e, a ogni apertura, `InventoryUI` lo riposiziona davanti al giocatore, a una distanza configurabile e orientato nella direzione dello sguardo. L'orientamento considera solo la componente orizzontale dello sguardo, così che il pannello resti verticale e leggibile anche quando il giocatore guarda verso l'alto o verso il basso. Il contenuto è organizzato in tre schede: *Domande*, *Risposte* e *AI*.

Le prime due raccolgono le frasi incontrate durante i dialoghi, secondo un criterio selettivo: una frase viene registrata solo se il giocatore ha scelto l'opzione corretta. Per ogni scambio completato con successo, `DialogueManager` salva due voci, la battuta dell'NPC nella scheda *Domande* e la risposta pronunciata dal giocatore nella scheda *Risposte*, ciascuna con il testo italiano, la traduzione inglese e l'NPC di provenienza (Figura 8). Le risposte errate non vengono registrate. L'inventario non conserva dati propri: a ogni apertura di una scheda legge le frasi accumulate da `GameManager`, il componente che mantiene lo stato della sessione (§), e le visualizza. In questo modo il contenuto rispecchia automaticamente il percorso del giocatore attraverso le scene, senza bisogno di sincronizzazione esplicita.

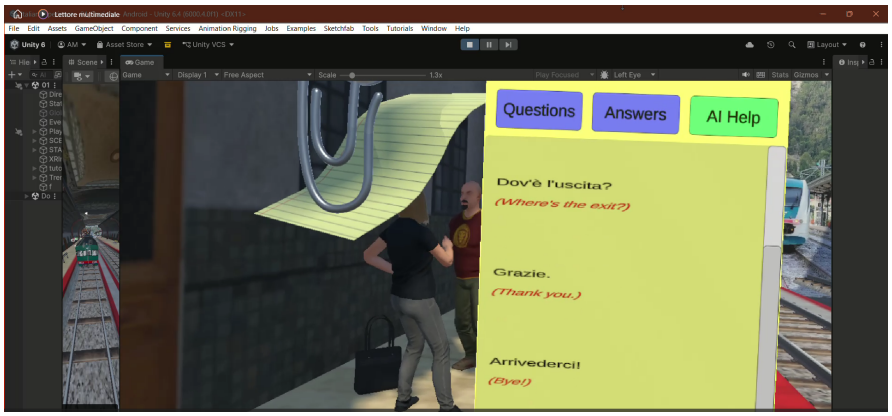
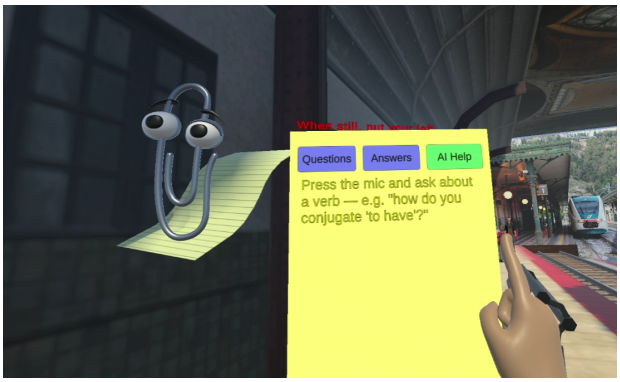


Figura 8: La scheda *Answers* dell'inventario, con le risposte pronunciate alla stazione e la loro traduzione inglese.

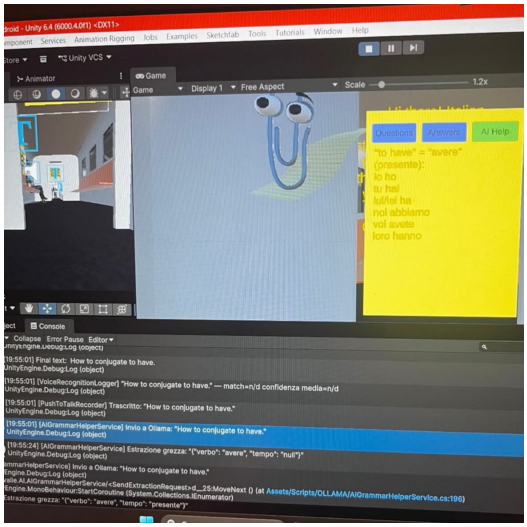
La scheda AI

La scheda *AI* ospita il tutor grammaticale, rappresentato da Clippy e gestito da `AIHelperPanelUI`. Poiché il giocatore è un principiante assoluto, le richieste al tutor vengono formulate in inglese: all'apertura della scheda, `AIHelperPanelUI` imposta Whisper sulla lingua inglese, e alla chiusura ripristina l'italiano. Il cambio di lingua svolge anche una funzione di smistamento: la trascrizione prodotta da Whisper viene inviata a tutti i componenti in ascolto, e la lingua attiva indica che è destinata al tutor e non al sistema di dialogo, peraltro inattivo mentre l'inventario è aperto.

Una richiesta come *how do you conjugate "to have"?* viene elaborata in due passi. Nel primo, `AIGrammarHelperService` la inoltra al modello linguistico Mistral, eseguito localmente tramite Ollama (§), con un *prompt* di sola estrazione: il modello restituisce un oggetto JSON che indica il verbo o la parola italiana a cui la domanda si riferisce, senza mai produrre forme coniugate o traduzioni. Nel secondo passo, questa indicazione è usata come chiave di ricerca in due database compilati e verificati a mano, da cui proviene per intero la risposta mostrata al giocatore (Figura 9). La separazione tra riconoscimento dell'intento e produzione della risposta è la scelta progettuale centrale del tutor, discussa al §.



(a) La scheda *AI* all'apertura.



(b) La risposta a *how do you conjugate "to have"?*

Figura 9: Il tutor grammaticale nell'inventario: la domanda è posta in inglese, la risposta proviene dal database dei verbi.

I database verificati

Il primo database, `VerbConjugationDatabase`, copre i verbi attestati nei dialoghi del gioco (*essere*, *avere*, *andare*, *volere*, *venire*, *potere*, *capire*, *suonare*, *contare*, *aiutare*, *spendere*), ciascuno con l'infinito italiano, una glossa inglese e le sei forme del presente indicativo. Il testo della risposta, costruito dal metodo `BuildPresenteText`, riporta la

coniugazione completa e una nota sull'uso di *Lei* come forma di cortesia, rilevante per le interazioni con la barista. Per consentirgli di associare il verbo inglese a quello italiano, il *prompt* inviato al modello include una tabella di corrispondenze generata da questo database,[81] nella forma "to have" → "avere", che delimita al contempo l'insieme dei verbi riconoscibili. Un indice di tutte le forme di ciascun verbo, infinito e sei persone, consente inoltre di riconoscere il verbo anche quando il giocatore chiede il significato di una forma coniugata letta nei pannelli, come in *what does spendi mean?*: in questo caso il sistema risale all'infinito e restituisce la coniugazione completa.

Il secondo database, `VocabularyDatabase`, raccoglie circa cento voci di lessico non verbale: parole incontrate nei dialoghi delle quattro scene, pronomi personali e possessivi, formule comunicative, lessico domestico e di servizio. Ogni voce riporta la traduzione inglese, la categoria grammaticale e una definizione facoltativa in inglese. Formule come *nessun problema* o *va bene* vi compaiono come voci unitarie.

La cascata di risoluzione

La ricerca nei due database segue una cascata a tre livelli, riportata nel Listato 8. Il primo livello tenta il riconoscimento come verbo, a partire dall'infinito estratto dal modello. Il secondo, attivato quando la domanda non riguarda un verbo, cerca la parola o l'espressione richiesta prima nel `VocabularyDatabase` e poi nell'indice delle forme coniugate. Il terzo gestisce le domande su più parole, come *what does lei spende mean?*, scomponendo l'espressione e spiegando ciascun termine separatamente. Se nessun livello produce un risultato, il giocatore riceve un messaggio fisso in inglese che lo invita a riformulare la domanda: anche in questo caso, il testo non è generato dal modello.

[81] Metodo `BuildEnglishToItalianMappingBlock()` della classe `VerbConjugationDatabase`.

```
void ResolveAnswer(string rawGenerated)
{
        string verbo  = ExtractJsonField(rawGenerated, "verbo");
        string parola = ExtractJsonField(rawGenerated, "parola");

        // 1. Verbo: coniugazione dal database
        if (!string.IsNullOrEmpty(verbo))
        {
                var verbEntry = verbDatabase.Get(verbo);
                if (verbEntry != null)
                {
                        OnAnswerReady?.Invoke(verbEntry.BuildPresenteText());
                        return;
                }
        }

        if (!string.IsNullOrEmpty(parola))
        {
                // 2. Parola o espressione unitaria
                string exactAnswer = TryResolveSingleTerm(parola);
                if (exactAnswer != null) {
                ↪  OnAnswerReady?.Invoke(exactAnswer); return; }

                // 3. Più parole: spiegazione termine per termine
                var words = parola.Split(new[] { ' ' },
                StringSplitOptions.RemoveEmptyEntries);
                var explanations = new List<string>();
                foreach (var w in words)
                {
                        string wordAnswer = TryResolveSingleTerm(w);
                        if (wordAnswer != null) explanations.Add(wordAnswer);
                }
                if (explanations.Count > 0)
                {
                        OnAnswerReady?.Invoke(string.Join("\n\n---\n\n",
                        ↪  explanations));
                        return;
                }
        }

        // Nessuna corrispondenza: messaggio fisso
        OnHelpNeeded?.Invoke("I didn't understand your question. ...");
}
```

Listato 8: Cascata di risoluzione in `AIGrammarHelperService`

Il tracciamento delle parole incontrate
Il tracciamento contestuale è affidato a `DialogueWordTracker`, un componente persistente tra le scene. Ogni volta che il giocatore completa con successo un nodo di dialogo, le parole contenute nella battuta dell'NPC e nell'opzione scelta vengono salvate nello stato di gioco. Quando il tutor restituisce la spiegazione di un verbo o una parola già incontrati, la risposta si chiude con la nota *You already saw this word in the game!*. Il meccanismo conferisce al tutor una reattività minima rispetto al percorso dell'apprendente: senza generare contenuto linguistico, collega la domanda del giocatore a ciò che ha già attraversato nell'ambiente, integrando il supporto metalinguistico con la memoria contestuale della sessione.

4.5.1 Implementazione di Ollama/Mistral

Ollama è una soluzione per l'esecuzione locale di modelli linguistici generativi, senza dipendenza da servizi esterni o connessioni di rete. Il modello impiegato è Mistral, eseguito interamente sul PC di sviluppo durante le sessioni di test. La scelta dell'esecuzione locale risponde a due esigenze concrete: la tutela dei dati dei partecipanti durante lo studio sperimentale e la possibilità di vincolare il comportamento del modello tramite *system prompt* senza dipendere da API esterne. L'indirizzo di Ollama è configurabile tramite un file `ollama_config.txt` in `Application.persistentDataPath`: se presente all'avvio, sovrascrive l'indirizzo predefinito `localhost`. Questa soluzione consente di eseguire Ollama su una macchina separata dal visore durante le sessioni sperimentali, senza modificare il *build*.

L'uso del modello linguistico nel progetto è circoscritto e deliberato. Durante lo sviluppo era stata valutata l'ipotesi di affidargli la generazione di dialogo libero con gli NPC tramite `AIDialogueService`, uno script che costruisce un *system prompt* con vincoli espliciti di livello A1, invia al modello la trascrizione vocale del giocatore e ne riceve la risposta dell'NPC. I test preliminari hanno mostrato che, anche con vincoli stringenti, le risposte generate eccedevano il livello A1 sul piano lessicale: a una domanda come *Come stai?*, il modello rispondeva *Assolutamente in forma*, una formulazione corretta e naturale, ma che al posto di una formula di base come *Sto bene* ricorre a un avverbio d'intensità e a un'espressione idiomatica estranei al repertorio di un principiante assoluto. Un input di questo tipo, per quanto autentico, risulterebbe incomprensibile per l'utente target. `AIDialogueService` è stato quindi disattivato, ma conservato nel progetto come traccia della revisione progettuale descritta al §3.1.

L'architettura anti-allucinazione L'esperienza con `AIDialogueService` ha orientato la progettazione del tutor verso un principio netto: il modello non genera mai contenuto linguistico italiano. Né le coniugazioni né le definizioni provengono da Mistral, ma soltanto dai due database descritti al §, compilati e verificati a mano. Al modello è affidato un unico compito: riconoscere a quale voce dei database si riferisce la domanda del giocatore. Il rischio di allucinazione è così eliminato alla radice: il modello non può inventare una coniugazione o una definizione, perché non gli viene mai chiesto di

produrne una, e la sua risposta non raggiunge mai il giocatore direttamente, ma viene usata soltanto come chiave di ricerca.

Il prompt di estrazione Il *prompt*, costruito dal metodo `BuildExtractionPrompt`, descrive al modello il suo ruolo di strumento di estrazione, include la tabella di corrispondenze dei verbi riconoscibili e impone un formato di output rigido: un solo oggetto JSON su una riga, senza commenti né formattazione, con tre campi (Listato 9). Il campo `verbo` è compilato con l'infinito italiano quando la domanda riguarda un verbo, il campo `parola` con il termine italiano quando la domanda riguarda il significato di una parola; i campi non pertinenti restano nulli. Il *prompt* prevede anche un campo `tempo`, che il modello compila con *presente* quando la domanda non specifica un altro tempo verbale. Poiché il database copre soltanto il presente indicativo, il campo svolge una funzione di filtro: se il giocatore chiede un tempo diverso, il tutor risponde con un messaggio fisso che ne segnala il limite, anziché con una coniugazione non pertinente. La struttura resta predisposta per un'estensione futura ad altri tempi verbali per altri livelli del PLI.

```
// ...
"{\"verbo\": \"<Italian verb infinitive or null>\", " +
       "\"parola\": \"<Italian word or null>\", " +
       "\"tempo\": \"<tense or null>\"}\n\n" +
"Rules:\n" +
// ...
"4. NEVER output conjugated forms or translations yourself. " +
"NEVER add commentary.\n" +
"5. Output ONLY the JSON object, on a single line. " +
"Set unused fields to null.\n\n" +
$"Question: \"{playerQuestionEN}\"";
```

Listato 9: Formato di output e regole del *prompt* di estrazione.

La risposta del modello viene letta dal metodo `ExtractJsonField`, che isola il valore di ciascun campo tramite un'espressione regolare in grado di catturare anche valori composti da più parole, come *lei spende*.

4.6 Persistenza dello stato di gioco

La persistenza dello stato di gioco tra le scene è affidata a `GameManager`, un singleton implementato con il pattern `DontDestroyOnLoad` che sopravvive ai cambi di scena per tutta la durata della sessione. `GameManager` tiene traccia delle missioni completate, delle risposte scelte dal player nei nodi di dialogo e del nome catturato tramite riconoscimento vocale nella scena del quartiere. Questi dati sono accessibili da qualsiasi script in qualsiasi scena tramite riferimento statico all'istanza singleton, senza necessità di passarli esplicitamente tra le scene.

L'esperienza è progettata come sessione singola continua: non esiste un menu di salvataggio né un meccanismo di ripresa. Lo stato persiste esclusivamente in memoria per tutta la durata della sessione. Il codice include un'infrastruttura di salvataggio su disco tramite `SaveSystem`, ma non è una funzionalità attiva nell'esperienza attuale.

Il completamento di ciascuna missione viene registrato in `GameManager` tramite `CompleteMission()`, che aggiunge l'identificativo della scena all'insieme delle missioni completate. Questo costituisce la base del sistema di sblocco progressivo delle interazioni: ogni `SceneDirector` controlla all'avvio se la scena risulta già completata tramite `IsMissionDone()` e, in caso affermativo, la salta senza attendere la prossimità del player. Se la scena non è ancora completata, il dialogo viene avviato solo quando il player entra nel collider trigger di prossimità dell'NPC e tutte le condizioni di accesso sono soddisfatte, verificate da `ConditionsMet()` tramite un array di flag narrativi configurabili in Inspector.

4.7 Il sistema di logging: VoiceRecognitionLogger

`VoiceRecognitionLogger` è il componente responsabile della raccolta automatica dei dati vocali durante le sessioni sperimentali. È collocato sullo stesso GameObject di `PushToTalkRecorder` e si abbona tramite `OnEnable` a tre eventi dell'`EventBus`: `DialogueStarted`, `NodeEntered` e `DialogueEnded`. Questi eventi gli consentono di tenere traccia in ogni momento del dialogo attivo e del nodo corrente, così che ogni trascrizione ricevuta possa essere annotata con il contesto narrativo preciso in cui è stata prodotta.

Per ogni tentativo vocale del player, il logger riceve dalla pipeline vocale il risultato completo della trascrizione, da cui estrae il testo trascritto. Nei dialoghi, il logger verifica se la trascrizione corrisponde a una delle opzioni attese nel nodo corrente tramite `VoiceMatchUtils.FindMatchingOption`, lo stesso meccanismo usato dal sistema di dialogo (Listato 6); nel *listen-and-repeat*, invece, l'esito viene passato direttamente da `ListenAndRepeatActivator` (§4.4.2). Il campo `IsMatch` può assumere cinque valori distinti, che distinguono situazioni analiticamente diverse invece di collassarle in un unico indicatore generico:

- `1`: match riuscito, la trascrizione corrisponde a uno dei *trigger* attesi nel nodo;
- `0`: match fallito, la trascrizione non corrisponde ad alcun *trigger*;
- `n/d_nome`: nodo di cattura del nome (`isNameCapture = true`), per il quale non esiste un insieme di *trigger* attesi e il match non è calcolabile per design; il campo `ExpectedPhrases` viene marcato come `[nome libero]`;
- `n/d_vuoto`: nodo senza opzioni vocali (pannello a soli bottoni), in cui la trascrizione arriva al logger ma il nodo non prevede una produzione vocale valutabile;

- `n/d_audio`: Whisper non ha ricevuto un segnale utile, e la trascrizione è un marcatore di audio vuoto prodotto dal modello (per esempio `[BLANK_AUDIO]`, `[Silence]`, `[Musica]`).

Solo le righe con `IsMatch` pari a `1` o `0` contribuiscono al conteggio di `AttemptNumber`, che registra quanti tentativi vocali valutabili il player ha effettuato su ciascun nodo prima di completarlo o abbandonarlo; i casi `n/d` non incrementano il contatore, perché non costituiscono tentativi di risposta. La logica completa di assegnazione è riportata in Appendice C.

Per ogni tentativo viene scritto in *append* un record CSV che comprende, tra gli altri, i campi `InteractionType`, `AttemptNumber`, `Timestamp`, `ParticipantId`, `AudioFile`, `Scene`, `TreeId`, `NodeId`, `NpcLineIT`, `ExpectedPhrases`, `TranscribedText`, `MatchedOption` e `IsMatch`. I campi `Timestamp` e `AudioFile` servono a collegare ciascun tentativo alla registrazione audio della sessione e sono stati rimossi al termine della valutazione percettiva (§5.2.4). Ogni sessione produce un file distinto, il cui nome include il `ParticipantId` (per esempio `VoiceLog_P01_.csv`); il file viene creato all'avvio della sessione, a condizione che il `ParticipantId` sia stato impostato nell'Inspector, e nessuna riga viene scritta prima di quel momento. I log sono stati poi importati in un foglio di calcolo, uno per partecipante, in cui è stata condotta la valutazione percettiva descritta in Appendice E.

Il logger non salva alcun file audio: registra esclusivamente la trascrizione testuale e i metadati di ciascun tentativo. Questa scelta risponde a un criterio di minimizzazione dei dati personali: il log non contiene la voce del partecipante né il suo nome, ma soltanto il `ParticipantId`. La registrazione audio necessaria alla valutazione percettiva delle trascrizioni è effettuata separatamente dalla ricercatrice, con il consenso esplicito dei partecipanti, ed è stata collegata al log solo temporaneamente, per la durata della valutazione (§5.2.4). Il `ParticipantId` è quindi l'unico elemento che collega tra loro log, registrazione audio e questionario anagrafico. La corrispondenza tra identificativo e nome reale è conservata su un foglio cartaceo separato, mai in formato digitale, e sarà eliminata al termine dello studio insieme alle registrazioni audio, rendendo a quel punto i dati pienamente anonimi. Le considerazioni etiche relative alla raccolta dei dati sono riportate in Appendice D.

5 Studio sperimentale

Un prototipo resta un'ipotesi finché qualcuno non lo usa. Questo capitolo presenta la ricerca sperimentale condotta con quindici parlanti anglofoni principianti assoluti: ne enuncia le direzioni e la metodologia, nonché gli strumenti di raccolta dati, i risultati e la loro discussione. Il §5.1 richiama le quattro domande di ricerca e ne esplicita l'intreccio, chiarendo come lo stesso set di dati risponda a domande diverse a seconda della prospettiva adottata. Il §5.2 descrive il disegno dello studio: il campione, la procedura, gli strumenti di misurazione quantitativi e qualitativi. Il §5.3 presenta i risultati per ciascuna domanda. Il §5.4 ne discute le implicazioni, dichiara i limiti e indica le direzioni aperte.

5.1 Le domande di ricerca

Le domande di ricerca anticipate al §1 si rispondono in sequenza, in quanto intrecciate dalla condivisione degli stessi dati raccolti durante le sessioni di gioco. Rendere esplicito questo intreccio prima di descrivere il disegno dello studio serve anche a giustificare ulteriormente la natura delle scene. Questa sezione risponde, di conseguenza, all'esigenza di spiegare le intenzioni per cui sono stati selezionati gli strumenti di raccolta dati impiegati per questa analisi.

RQ1 è la domanda di ricerca prettamente glottodidattica: indaga in quale misura un ambiente VR strutturato attorno a compiti comunicativi reali, in cui la produzione orale è la modalità primaria di interazione, favorisce il riconoscimento di un repertorio di sopravvivenza in italiano A1 da parte di parlanti anglofoni principianti assoluti. La verifica è affidata al post-test somministrato dopo la sessione, che misura il riconoscimento fuori contesto delle funzioni comunicative e del lessico incontrati nel gioco. I dati comportamentali raccolti automaticamente durante il gioco offrono una misura complementare, il cui rapporto con il *post-test* è oggetto di RQ3. Se consideriamo che ogni risposta vocale trascritta da Whisper rappresenta un match riuscito o fallito, ogni nodo di dialogo completato o ripetuto è allora una traccia dell'interazione del discente con il sistema. Il post-test misura cosa il player sa riconoscere *fuori contesto* dopo la sessione gioco; il log misura come si è comportato *dentro* il gioco. Il confronto tra i risultati del post-test per gli item relativi al quartiere e quelli relativi al bar risponde a

RQ1 e ad un'altra implicazione ipotetica. Le due scene differiscono per la presenza o assenza della traduzione inglese nelle opzioni di risposta, dunque, un'analisi incrociata dei risultati permette di formulare un'ipotesi indicativa sul contributo dei sottotitoli alla ritenzione mnemonica.

RQ2 indaga se Whisper, nella modalità di parlato controllato adottata dal prototipo,[82] regge alla variabilità di pronuncia dei partecipanti in misura sufficiente a supportare l'interazione. Le misure principali, ricavate dal log, sono la quota di tentativi riconosciuti, in particolare al primo tentativo, e il numero di tentativi necessari per completare ciascun nodo vocale. La valutazione percettiva delle trascrizioni (§5.2.4) consente poi di stabilire in quali casi l'esito del riconoscimento automatico corrisponde al giudizio di un ascoltatore madrelingua. La scena del quartiere con Lina, che prevede i sottotitoli in inglese, costituisce la *baseline*: misura la tenuta di Whisper nelle condizioni di supporto massimo e in assenza di distrattori. La scena del bar ne costituisce la misura principale, in quanto unica scena con opzioni di risposta prive di traduzione e dotata di distrattori, in cui il riconoscimento avviene nelle condizioni di pressione comunicativa più vicine a una conversazione reale. Data la dimensione del campione, i risultati hanno valore esplorativo. RQ2 è una condizione necessaria per RQ1: se Whisper non riconosce l'output vocale del discente in misura sufficiente, i dati sulla produzione orale non sono affidabili e RQ1 non può essere verificata con i dati comportamentali.

RQ3 indaga se i dati comportamentali raccolti automaticamente durante il gioco, in forma di trascrizioni vocali ed esiti dei match, costituiscano uno strumento di misurazione adeguato, comparabile o complementare al post-test tradizionale. RQ3 usa RQ1 come termine di confronto. Se i risultati del *post-test* e il tasso di riconoscimento in gioco convergono sulle stesse formule, il test aggiunge poca informazione rispetto a ciò che il log già misura. Se differiscono, il test cattura qualcosa che il log non vede, e quella divergenza, seppur eventualmente infima, è di per sé un risultato. RQ3 è la più originale delle quattro domande di ricerca, e la più esposta metodologicamente: non si è riscontrato ancora in letteratura un corpo consolidato di evidenze sulla validità dei dati comportamentali come proxy di competenza linguistica in ambienti VR. Le sue conclusioni avranno necessariamente valore esplorativo.

RQ4 indaga l'usabilità percepita del sistema e il senso di presenza nell'ambiente virtuale, misurati rispettivamente tramite la System Usability Scale e il questionario Slater-Usoh-Steed somministrati al termine della sessione. Correlata all'indagine sull'efficacia didattica dell'ambiente, ne costituisce la condizione di leggibilità: senza un dato minimo sull'usabilità e sul senso di presenza, un risultato basso al *post-test* resterebbe aperto a due letture opposte: il sistema non insegna, oppure non si lascia usare. In questo senso RQ4 è strumentale alle altre tre, pur misurando un costrutto distinto.

[82] Vincolato dunque da frasi prestabilite da leggere ad alta voce su *prompt* visivo, con o senza sottotitoli inglesi a supporto.

5.2 Disegno dello studio

Lo studio esplorativo adotta un disegno sperimentale di tipo *post-test* condotto su un unico gruppo di partecipanti. Il campione è costituito da apprendenti classificati come principianti assoluti sulla base di un criterio di selezione prestabilito: l'assenza di conoscenze pregresse della lingua italiana, ad eccezione di eventuali parole isolate. Tale condizione è stata verificata mediante autovalutazione attraverso il questionario anagrafico somministrato ai partecipanti. La valutazione dell'acquisizione linguistica si basa esclusivamente sui risultati ottenuti al *post-test* e sui dati comportamentali raccolti automaticamente nel corso delle sessioni sperimentali.

5.2.1 Metodologia

Il campione è composto da quindici parlanti nativi di inglese britannico, otto donne e sette uomini, di età compresa tra 22 e 60 anni (mediana 28), reclutati tramite campionamento di convenienza. Al momento dello studio tutti i partecipanti risiedevano a Stamford, nel Lincolnshire: undici vi sono nati, uno proviene dalla vicina Ketton e tre da altre città dell'Inghilterra centrale e settentrionale (Nottingham, Sheffield e Manchester). Tutti hanno indicato una conoscenza pregressa dell'italiano pari a 1 su 5, il valore minimo della scala, e undici non avevano mai utilizzato la realtà virtuale. Le prime due sessioni (P01, P02) hanno svolto anche funzione di pilota e sono incluse nel campione. A seguito di queste sessioni sono stati introdotti nel sistema due supporti, la lettura audio dei pannelli di *onboarding* e un segnale sonoro di inizio turno: le due sessioni pilota si sono quindi svolte senza di essi. Le varietà di inglese dei partecipanti sono tutte non rotiche: la resa della /r/ in posizione postvocalica costituisce quindi una variabile comune all'intero gruppo, con effetti attesi sul riconoscimento delle parole con /r/ del corpus (*arrivederci*, *frigorifero*, *prego*, *cornetto*). L'omogeneità del campione, per provenienza e luogo di residenza, rende invece impraticabile un confronto tra varietà di inglese: l'analisi di RQ2 si concentra perciò sui fenomeni fonologici condivisi dal gruppo.

Ogni sessione si svolge in presenza con la ricercatrice come moderatrice, per una durata stimata di trenta minuti. La sessione si articola in tre fasi distinte, ciascuna con strumenti propri.

Prima della sessione
Il partecipante firma il consenso informato e compila il questionario anagrafico (Modulo 1, Appendice D), che raccoglie dati su età, background linguistico e varietà di inglese parlata. Questi dati non costituiscono una misurazione delle competenze, ma forniscono le variabili di contesto utili all'interpretazione dei risultati di RQ2.

Durante la sessione
Il partecipante svolge la sessione di gioco, della durata di circa trenta minuti, alla presenza della ricercatrice, che interviene soltanto in caso di difficoltà tecniche. Durante la sessione il gioco registra automaticamente il log dei tentativi vocali, mentre la ricercatrice registra l'audio della sessione tramite dispositivo mobile (§5.2.4). Le informazioni che il log non

cattura, come i problemi di attivazione del microfono e l'uso dei pannelli di supporto, sono state annotate in seguito, nella griglia di valutazione percettiva.

Dopo la sessione
Immediatamente dopo il gioco, prima di qualsiasi debriefing, il partecipante compila in sequenza tre strumenti: il post-test linguistico (Modulo 2.1), il questionario di presenza (Modulo 2.2) e il questionario di usabilità (Modulo 2.3). L'ordine è fisso: il post-test linguistico viene somministrato per primo per evitare che la riflessione sull'esperienza contamini la misurazione del lessico acquisito; il questionario di presenza precede quello di usabilità perché le sue domande richiedono un ricordo sensoriale fresco dell'ambiente virtuale, che un questionario tecnico sull'interfaccia rischierebbe di alterare.

Gli strumenti di raccolta dati sono descritti nelle sottosezioni seguenti, ciascuno in relazione alle domande di ricerca cui risponde.

Il **post-test a risposta multipla** (Modulo 2.1) misura direttamente cosa i partecipanti sanno fare dopo il gioco in termini di funzioni comunicative e lessico del sillabo, fuori dal contesto situato in cui li hanno incontrati. È lo strumento primario per RQ1. Il confronto tra gli item relativi al quartiere e quelli relativi al bar permette inoltre di formulare un'ipotesi indicativa sul contributo dei sottotitoli alla ritenzione mnemonica.

Il **log CSV** generato automaticamente da `VoiceRecognitionLogger` registra, per ogni tentativo vocale, il testo trascritto da Whisper, le frasi attese dal nodo di dialogo, l'esito del match, e il contesto della scena. È lo strumento primario per RQ2 e secondario per RQ3: i dati del quartiere costituiscono la baseline per la tenuta di Whisper con supporto massimo; i dati del bar costituiscono la misura principale in condizioni di pressione comunicativa.

Il `ParticipantId`, assegnato dalla ricercatrice prima di ogni sessione, collega il log alla griglia di valutazione percettiva. Il modulo post-sessione, invece, non ha registrato l'identificativo: le risposte al *post-test* e ai questionari non possono quindi essere associate ai singoli log, anche perché anonime. Per questa ragione, il confronto tra *post-test* e dati comportamentali previsto per RQ3 è condotto a livello di gruppo, item per item, anziché per singolo partecipante (§5.3 RQ3). Le procedure di analisi adottate per ciascuna domanda di ricerca, insieme alla definizione delle misure impiegate, sono descritte al §5.2.5.

5.2.2 System Usability Scale

L'usabilità percepita del sistema è misurata tramite la *System Usability Scale* (SUS), uno strumento a dieci item sviluppato da John Brooke nel 1986 nell'ambito del programma di ingegneria dell'usabilità di Digital Equipment Co. Ltd.[83] Ogni item è formulato come un'affermazione, riferita in questo studio al gioco, da valutare su una scala Likert a

[83]John Brooke, "SUS: A Quick and Dirty Usability Scale", in *Usability Evaluation in Industry*, a cura di Patrick W. Jordan et al. (London: Taylor & Francis, 1996), pp. 189–194.

cinque punti (da *strongly disagree* a *strongly agree*). Gli item a formulazione positiva e negativa si alternano, così da limitare la tendenza a rispondere in modo uniforme senza considerare il contenuto delle domande. Ciascuna risposta viene convertita in un contributo compreso tra 0 e 4; la somma dei contributi, moltiplicata per 2,5, produce un punteggio da 0 a 100. Secondo la scala interpretativa proposta da Bangor, Kortum e Miller, punteggi superiori a 70 indicano un'usabilità accettabile.[84] La SUS è stata scelta per la sua robustezza, la sua brevità e la sua ampia diffusione nella letteratura sui sistemi interattivi, che consente il confronto con valori di riferimento consolidati. La SUS si presta inoltre a campioni ridotti: in un confronto tra questionari di usabilità, Tullis e Stetson hanno osservato che produce risultati affidabili già a partire da 12-14 partecipanti.[85] Trattandosi di uno strumento generico, la SUS non rileva aspetti specifici dell'interazione in realtà virtuale, come i gesti non riconosciuti dal sistema di *hand tracking*: per l'attivazione del microfono, l'unico gesto per cui i dati lo consentono, questi problemi sono documentati dalla categoria MIC della griglia di valutazione percettiva (§5.2.4). Le domande aperte dei questionari registrano qualsiasi altra osservazione sul riconoscimento dei gesti o sugli aspetti interattivi.

5.2.3 Presenza e immersività: questionario Slater-Usoh-Steed

La misurazione del senso di presenza, distinta e complementare rispetto all'usabilità, si basa sul questionario sviluppato da Mel Slater, Martin Usoh e Anthony Steed nel corso di una serie di studi sulla realtà virtuale immersiva, nella formulazione impiegata in Usoh et al. (2000).[86] La scelta di questo strumento risponde a due esigenze. La prima è pratica: con pochi item, il questionario si somministra in pochi minuti e non appesantisce una sessione già articolata. La seconda è teorica: Usoh et al. (2000) lo sottopongono a un *reality test*, verificando se produca punteggi più alti per esperienze reali rispetto a esperienze virtuali, condizione minima che qualsiasi misura della presenza dovrebbe soddisfare. Il questionario Slater-Usoh-Steed supera il test, seppur marginalmente, a differenza del Witmer-Singer Presence Questionnaire a 32 item. Gli autori segnalano la tendenza dei partecipanti a relativizzare le risposte al dominio dell'esperienza vissuta, che rende problematici i confronti tra ambienti diversi; in questo studio, in cui tutti i partecipanti vivono la stessa esperienza, il limite non compromette la validità interna della misurazione.

Il questionario somministrato si compone di cinque item, che fanno riferimento esplicito a ItalianVRExperience e coprono tre dimensioni della presenza. La prima è il senso di trovarsi fisicamente nell'ambiente virtuale, rilevato dall'item *I had a sense of "being*

[84] Aaron Bangor, Philip T. Kortum e James T. Miller, "An Empirical Evaluation of the System Usability Scale", *International Journal of Human-Computer Interaction* 24, no. 6 (2008): pp. 574–594.

[85] Thomas S. Tullis e Jacqueline N. Stetson, "A Comparison of Questionnaires for Assessing Website Usability", in *Proceedings of the Usability Professionals Association (UPA) Conference* (Minneapolis, 2004).

[86] Martin Usoh et al., "Using Presence Questionnaires in Reality", *Presence: Teleoperators and Virtual Environments* 9, no. 5 (2000): pp. 497–503.

there" in ItalianVRExperience. La seconda è la misura in cui l'ambiente virtuale prevale sulla realtà circostante durante l'esperienza, rilevata da due item: uno chiede se ci siano stati momenti in cui ItalianVRExperience è stato percepito come reale, l'altro chiede di collocarsi tra il senso di trovarsi altrove e quello di trovarsi in un luogo italiano. La terza è la natura del ricordo che il partecipante conserva dell'ambiente, anch'essa rilevata da due item: uno chiede se ItalianVRExperience sia ricordato più come un insieme di immagini viste o come un luogo visitato, l'altro se venga pensato come un luogo analogo agli altri in cui il partecipante è stato.

Ogni item è valutato su una scala Likert a cinque punti, la stessa della System Usability Scale somministrata nella stessa sessione, i cui estremi sono ancorati a descrittori verbali specifici per ciascuna domanda (per esempio *Images that I saw* e *Somewhere that I visited*). Il punteggio di presenza di ciascun partecipante è dato dalla media delle risposte ai cinque item, su una scala da 1 a 5, dove un valore più alto indica un senso di presenza più marcato; come dato complementare si riporta la percentuale di risposte pari a 4 o 5. Il punteggio così costruito va letto in chiave descrittiva, come misura interna allo studio e non come termine di confronto diretto con i valori riportati in letteratura. Il testo completo degli item è riportato in Appendice D.

Va segnalata e chiarita, infine, una potenziale fonte di confusione terminologica: l'acronimo SUS è condiviso da questo questionario (Slater-Usoh-Steed) e dalla System Usability Scale di Brooke. I due strumenti misurano costrutti distinti e non hanno alcuna relazione tra loro. Nel resto di questo lavoro, il termine SUS senza qualificazione si riferisce alla System Usability Scale; il questionario di presenza è indicato esplicitamente come questionario Slater-Usoh-Steed o questionario di presenza.

5.2.4 Valutazione percettiva delle trascrizioni

Il log automatico di `VoiceRecognitionLogger` registra l'esito del riconoscimento vocale in termini binari: riconoscimento riuscito o mancato. Questo dato offre una misura delle prestazioni del sistema, ma non consente di interpretarle. Un mancato riconoscimento può dipendere da una pronuncia fonologicamente lontana dalla norma, da una trascrizione infedele di Whisper o da un limite del meccanismo di confronto; un riconoscimento riuscito, simmetricamente, può riflettere una pronuncia corretta oppure una normalizzazione operata dal modello linguistico interno di Whisper, che riconduce l'output alla forma attesa indipendentemente da ciò che il partecipante ha prodotto. Per distinguere questi casi, il dato automatico è stato affiancato da una valutazione percettiva condotta sulle registrazioni audio delle sessioni.

Poiché il gioco non salva alcun file audio (§4.7), ciascuna sessione è stata registrata separatamente dalla ricercatrice tramite dispositivo mobile, con il consenso esplicito dei partecipanti (Appendice D). Le registrazioni sono impiegate al solo scopo della valutazione percettiva, non sono condivise con terzi e saranno eliminate al termine dell'analisi. Durante la fase di valutazione, ogni riga del log è stata collegata al file audio corrispondente tramite due colonne di servizio, contenenti il nome del file e l'istante di

registrazione del tentativo, così da individuare rapidamente il segmento da ascoltare. Completata la compilazione delle griglie di valutazione, le due colonne sono state rimosse dal log.

Per ogni tentativo vocale registrato nel log, la ricercatrice ha ascoltato il segmento audio corrispondente e ha risposto a due domande indipendenti. La prima riguarda la fedeltà della trascrizione: il testo prodotto da Whisper riflette ciò che il partecipante ha effettivamente pronunciato? La seconda riguarda l'accettabilità della pronuncia: ciò che il partecipante ha prodotto sarebbe stato comprensibile per un madrelingua nel contesto comunicativo della frase? Le due domande sono deliberatamente separate, perché Whisper può trascrivere fedelmente una pronuncia insufficiente, oppure produrre una trascrizione corretta, grazie al proprio modello linguistico interno, a partire da una pronuncia lontana dalla norma. La valutazione è stata condotta dalla ricercatrice, madrelingua italiana, in qualità di unica valutatrice: per contenerne la componente soggettiva, i criteri sono stati definiti a priori e calibrati sulle prime due sessioni, le quali hanno funzione di pilota e sono incluse nel campione. L'assenza di un secondo valutatore indipendente costituisce un limite dello studio, ripreso nella discussione (§).

Codifica dei fenomeni fonologici
I codici di annotazione sono stati definiti a partire da due fonti: i fenomeni attesi per apprendenti anglofoni di italiano, sulla base delle differenze tra i due sistemi fonologici discusse al §2.3.2, e l'ascolto delle due sessioni iniziali (P01, P02), confrontato con le trascrizioni di Whisper. In queste due sessioni sono stati valutati 69 tentativi vocali, in 36 dei quali la pronuncia si discosta in modo udibile dalla norma, per un totale di 48 occorrenze di fenomeni. La Tabella 3 riporta, per ciascun fenomeno osservato, un esempio tratto dai log, il numero di occorrenze e l'esito del confronto tra riconoscimento automatico e valutazione percettiva, secondo le categorie definite di seguito.[87]

Tabella 3: Fenomeni fonologici osservati nelle sessioni iniziali (P01, P02; 69 tentativi valutati): esempi, occorrenze e categorie di valutazione.

Cod.	**Fenomeno**	**Esempio (forma attesa → trascrizione)**	**Occorr.**	**Categorie**
RHO	Resa non vibrante di /r/	*arrivederci* con /r/ approssimante (solo all'ascolto)	9	RC 3, MI 3, OC 2, TI 1
GEM+	Ipercorrettismo geminato	*arrivato* → "arrivatto"	8	RC 7, FN 1
FUS	Segmentazione anomala dei confini di parola	*a domani* → "Ado mani"	7	RC 4, FN 2, TI 1

[87] I conteggi si riferiscono alla codifica definitiva delle due sessioni, compilata con gli stessi criteri applicati alle sessioni successive.

Cod.	Fenomeno	Esempio (forma attesa → trascrizione)	Occorr.	Categorie
GEM	Scempiamento	*donna* → “dona”	6	RC 5, TI 1
ELO	Allungamento vocalico	*uomo* → “ooomo”	6	TI 3, RC 2, MG 1
VOC-d	Dittongazione o timbro vocalico inglese	*persona* → “pearsona”	4	TI 2, RC 1, MI 1
SUB	Sostituzione consonantica	*tazza* → “tassa”	3	RC 2, FN 1
VOC-a	Chiusura di /a/ verso /o/	*acqua* → “ocqua”	1	RC 1
VOC-e	Alterazione di /e/ finale	*mille* → “millie”	1	MI 1
ALTRO	Fenomeno non previsto, descritto nelle note	/	3	TI 2, MI 1
Totale			**48**	RC 25, TI 10, MI 6, FN 4, OC 2, MG 1

Già nelle sessioni iniziali emergono i due comportamenti di Whisper che l’analisi dell’intero campione avrebbe poi confermato. Da un lato, i fenomeni che alterano la forma della parola in modo visibile nel testo, come l’ipercorrettismo geminato, lo scempiamento e la segmentazione anomala, vengono trascritti fedelmente e portano per lo più a un rifiuto corretto (16 casi su 21). Dall’altro, la resa non vibrante di /r/, il fenomeno più frequente, produce l’esito più disperso: in cinque casi su nove il sistema accetta comunque la risposta (OC o MI), perché Whisper riconduce la /r/ inglese alla grafia italiana e il fenomeno resta udibile soltanto all’ascolto.

Nelle sessioni successive l’insieme dei codici è rimasto stabile, con due eccezioni: sono emersi due fenomeni non osservati nelle sessioni iniziali, ma previsti tra quelli attesi per l’utente target. Il primo è la semplificazione del nesso consonantico iniziale (GR; *sgabello* → “scabello”), comparso a partire dalla quarta sessione; il secondo è la lettura di *ch* come /tʃ/ per interferenza dell’ortografia inglese (PAL; *che bello* → “ce bello”), comparso a partire dalla settima. L’insieme finale comprende quindi dodici codici, definiti per esteso in Appendice E (§E.3). La frequenza di ciascun fenomeno nell’intero campione e la sua distribuzione nelle categorie di valutazione sono presentate nei risultati relativi a RQ2 (§5.3).

Categorie di valutazione
Ogni tentativo vocale è classificato in una di sei categorie, ottenute incrociando le due risposte della valutazione percettiva, cioè la fedeltà della trascrizione (T) e l'accettabilità della pronuncia (P), con l'esito del riconoscimento automatico registrato nel campo `IsMatch`. Le categorie sono mutuamente esclusive e sono definite come segue.

MG, match genuino (`IsMatch` = 1, T Y, P Y). Whisper trascrive fedelmente una pronuncia accettabile, e il sistema la riconosce correttamente.

MI, match su pronuncia insufficiente (`IsMatch` = 1, T Y, P N). Whisper trascrive fedelmente una pronuncia che un madrelingua non avrebbe compreso, e il sistema la accetta comunque: il riconoscimento risulta troppo permissivo.

OC, *over-correction* (`IsMatch` = 1, T N). Whisper produce un testo diverso da quanto pronunciato, riconducendolo alla forma attesa per effetto del proprio modello linguistico interno: il riconoscimento dipende dal modello più che dalla produzione del partecipante.

FN, falso negativo (`IsMatch` = 0, T Y, P Y). Whisper trascrive fedelmente una pronuncia accettabile, ma il sistema non la riconosce: il limite risiede nel meccanismo di confronto.

RC, rifiuto corretto (`IsMatch` = 0, T Y, P N). Whisper trascrive fedelmente una pronuncia non accettabile, e il sistema la rifiuta correttamente.

TI, trascrizione infedele (`IsMatch` = 0, T N). Whisper fraintende la pronuncia e il riconoscimento fallisce: l'errore risiede nella trascrizione.

Accanto alle sei categorie, la griglia prevede la categoria MIC per i tentativi in cui l'errore non dipende né dalla pronuncia né dalla trascrizione, ma dall'acquisizione del segnale: attivazione accidentale del microfono, attivazione in ritardo rispetto all'inizio dell'enunciato, interruzione brusca della registrazione. Questi tentativi sono esclusi dal calcolo delle sei categorie. Per ogni tentativo la griglia registra inoltre se il partecipante stava consultando i pannelli di supporto (pannelli di pronuncia preliminari ai dialoghi, pannelli del *listen and repeat*) (*Pannelli*) e la modalità di emissione (*Modalità vocale*: neutra, voce alta, oppure scandita, con parole separate e articolate con enfasi).

Ai fini di RQ2, le categorie più informative sono OC e FN, che rappresentano i due modi opposti in cui il riconoscimento automatico può divergere dal giudizio umano. OC individua i casi in cui il riconoscimento è prodotto dal modello linguistico di Whisper più che dalla pronuncia del partecipante, e rende il riconoscimento un indicatore poco affidabile della qualità della produzione. FN individua invece i casi in cui il sistema è più rigido dell'ascolto umano e penalizza produzioni comunicativamente accettabili. MI segnala un terzo profilo: la difficoltà del sistema a distinguere una pronuncia adeguata da una insufficiente, anche quando la trascrizione è fedele. La griglia completa e le istruzioni di compilazione sono riportate in Appendice E.

5.2.5 Analisi dei dati

Data la dimensione del campione ($N = 15$), l'analisi è esclusivamente descrittiva: i risultati sono espressi in frequenze, percentuali, medie con deviazione standard, mediane e intervalli, senza ricorso a test inferenziali. Le associazioni tra variabili, come quella tra uso dei pannelli ed esito del riconoscimento, sono riportate come tali e non come effetti causali, non disponendo di dati numerici a sostegno o confutazione di eventuali ipotesi a riguardo.

L'analisi dei dati vocali ha seguito una procedura in cinque fasi. Nella prima, i log dei quindici partecipanti sono stati verificati e ripuliti: sono state ripristinate le lettere accentate nei file salvati con una codifica errata, riallineate le righe malformate e corretti gli identificativi dei partecipanti registrati col proprio nome[88], attribuendo ciascuna riga in base al nome del file di log. Nella seconda, i dati sono stati importati in un foglio di calcolo, con un foglio per partecipante e una riga per tentativo vocale, a formare la griglia di valutazione descritta al §5.2.4. Nella terza, ciascun tentativo è stato ascoltato e annotato secondo i criteri dell'Appendice E. Nella quarta, conclusa la compilazione, l'intero corpus è stato sottoposto a una revisione di coerenza, per applicare in modo uniforme a tutte le sessioni la regola dei tentativi MIC e i criteri di codifica della segmentazione anomala, precisati nel corso della valutazione. Nella quinta, la categoria di ciascun tentativo è stata calcolata automaticamente dal foglio di lavoro, incrociando T, P e `IsMatch` secondo la legenda dell'Appendice E (§E.4), e i dati sono stati aggregati per partecipante, scena, tipo di interazione, uso dei pannelli e modalità vocale. Per ciascun partecipante sono state conteggiate le occorrenze corrispondenti a ciascuna delle variabili analizzate, sulla base dei criteri definiti per la codifica dei dati. Le percentuali sono state calcolate dividendo il numero di occorrenze rilevate per il totale delle osservazioni pertinenti alla specifica variabile e moltiplicando il risultato per 100. Le misure riferite ai nodi, cioè il primo tentativo valido, i nodi completati e il numero medio di tentativi per nodo, sono state calcolate manualmente a partire dai dati della griglia, così come i punteggi del *post-test*, della System Usability Scale e del questionario di presenza a partire dalle risposte al modulo post-sessione.

Dal totale dei tentativi registrati sono state escluse anzitutto le righe senza una risposta attesa valutabile, cioè la cattura del nome e le domande al tutor grammaticale (88 righe). Dei 497 tentativi restanti, 42 sono stati classificati come MIC, perché compromessi dall'acquisizione del segnale, e uno è stato escluso in sede di valutazione: alla battuta di Lina che attendeva *Ciao, grazie!* o *A domani!*, il partecipante ha risposto *prego*, una replica pertinente ma estranea alle opzioni del nodo, che non costituisce un tentativo di produrle. L'analisi di RQ2 si basa quindi su 454 tentativi.

Le misure ricavate dal log sono definite come segue. Il *primo tentativo valido* è, per ciascun partecipante, il primo tentativo non classificato come MIC su ogni nodo di dialogo e su ogni parola del *listen-and-repeat* (306 casi). Un nodo è considerato *completato*

[88] I nomi propri sono stati sostituiti dagli identificativi per partecipante, per mantenere l'anonimato.

quando almeno uno dei tentativi del partecipante su quel nodo è stato riconosciuto. Il numero medio di *tentativi per nodo* è il rapporto tra i tentativi validi e i nodi affrontati. L'*accordo* tra riconoscimento automatico e valutazione percettiva è la quota di tentativi in cui l'esito del sistema coincide con il giudizio sulla pronuncia: tentativi accettati con pronuncia accettabile (`IsMatch` = 1, P = Y) e tentativi rifiutati con pronuncia non accettabile (`IsMatch` = 0, P = N).

In forma sintetica, indicando con $n(\cdot)$ il numero di tentativi che soddisfano la condizione tra parentesi e con N il totale di riferimento, le misure sono calcolate come segue:

$$R_1 = \frac{n(\text{primi tentativi validi riconosciuti})}{n(\text{primi tentativi validi})} \qquad C = \frac{n(\text{nodi completati})}{n(\text{nodi affrontati})} \qquad \bar{t} = \frac{n(\text{tentativi validi})}{n(\text{nodi affrontati})}$$

$$A = \frac{n(\texttt{IsMatch} = 1,\ P = Y) + n(\texttt{IsMatch} = 0,\ P = N)}{N}$$

dove R_1 è il tasso di riconoscimento al primo tentativo, C il tasso di completamento, $\bar{t}$ il numero medio di tentativi per nodo e A l'accordo tra riconoscimento automatico e valutazione percettiva. Il punteggio della System Usability Scale di ciascun partecipante è calcolato come

$$SUS = 2{,}5 \times \left[\sum_{i \text{ dispari}} (x_i - 1) + \sum_{i \text{ pari}} (5 - x_i) \right]$$

dove x_i è la risposta all'item i su scala da 1 a 5, e il punteggio di presenza come media aritmetica delle risposte ai cinque item del questionario Slater-Usoh-Steed.

Per RQ1, le risposte al *post-test* sono analizzate per singolo item e aggregate per scena, attribuendo ciascun item alla scena in cui la formula è incontrata; gli item relativi a formule presenti in più scene, come la risposta a *Grazie* e *Arrivederci*, sono trattati come trasversali. Per ogni item è inoltre individuata la risposta errata più frequente, e le percentuali degli item del quartiere e del bar sono confrontate per esplorare il contributo dei sottotitoli. Per RQ2, le misure ricavate dal log sono calcolate sull'intero gruppo, per scena, per partecipante e, nel *listen-and-repeat*, per singola parola; le categorie della valutazione percettiva sono contate in totale e per scena, e il disaccordo tra sistema e ascolto è scomposto nelle sue due direzioni, cioè tentativi accettati con pronuncia non accettabile e tentativi rifiutati con pronuncia accettabile. I fenomeni fonologici annotati sono infine incrociati con le categorie, e l'esito del riconoscimento è confrontato in base all'uso dei pannelli e alla modalità vocale. Per RQ3, a ciascuna formula comune ai due strumenti è associato il nodo di dialogo in cui la formula è incontrata o prodotta, e la percentuale di risposte corrette al *post-test* è affiancata alla percentuale di riconoscimento al primo tentativo valido su quel nodo. Il denominatore di quest'ultima corrisponde ai partecipanti che hanno effettivamente raggiunto il nodo: il nodo di chiarimento del bar, per esempio, è raggiunto soltanto da chi ha scelto un distrattore. Per RQ4, il punteggio della System Usability Scale è calcolato secondo la procedura standard (§5.2.2) e quello di presenza come media dei cinque item (§5.2.3); entrambi sono descritti tramite media, deviazione standard, mediana e intervallo.

Le risposte aperte del questionario (item 32, 51, 52, 53 e 54) sono state analizzate tramite una codifica tematica: le risposte di ciascuna domanda sono state raggruppate per temi ricorrenti, contando per ciascun tema il numero di partecipanti che lo menzionano. Nei risultati, i temi sono illustrati con citazioni testuali, riportate in lingua originale e accompagnate dall'identificativo del partecipante.

5.3 Risultati

I risultati sono presentati per ciascuna domanda di ricerca nell'ordine in cui sono state enunciate al §5.1. Per ciascuna si riportano prima i dati quantitativi principali, poi la triangolazione con gli strumenti secondari. Data la dimensione del campione ($N = 15$), tutti i risultati hanno valore descrittivo ed esplorativo.

RQ1. Efficacia acquisizionale

Il *post-test* comprende quattordici item a scelta singola. I partecipanti hanno risposto correttamente, in media, a 12,1 item su 14 (86%; deviazione standard 1,3), con un intervallo compreso tra 10 e 14; nessun partecipante è sceso sotto il 71% di risposte corrette. La Tabella 4 riporta i risultati per item.

Item	Contenuto	Scena	Corrette	Errore più frequente
17	*Biglietto, prego* (significato)	Stazione	15/15	/
22	*Biglietto, prego* (intenzione del controllore)	Stazione	15/15	/
16	Risposta a *Come ti chiami?*	Quartiere	14/15	*Sto bene* (1)
24	*Sei nuovo qui?*	Quartiere	10/15	*Where you are going* (5)
25	*Piacere!*	Quartiere	12/15	*You're welcome* (3)
28	*Nessun problema*	Quartiere	15/15	/
29	*Scusa*	Quartiere	15/15	/
18	*Per favore*	Bar	15/15	/
19	Parola che non indica una bevanda	Bar	14/15	*Acqua* (1)
21	*Cornetto*	Bar	13/15	*Tazza* (2)
26	*C'è un divano*	Casa	8/15	*A table* (5)
27	*La cucina*	Casa	12/15	*In the living room* (3)
20	Risposta a *Grazie*	Trasversale	9/15	*Per favore* (5)
23	*Arrivederci*	Trasversale	14/15	*You say thank you* (1)

Tabella 4: Risultati del *post-test* per item ($N = 15$).

Aggregando gli item per scena, le percentuali di risposte corrette sono del 100% per la stazione (30/30), del 93% per il bar (42/45), dell'88% per il quartiere (66/75) e

del 67% per la casa (20/30); gli item trasversali si attestano al 77% (23/30). Gli item 17 e 22 verificano la stessa formula e hanno ottenuto entrambi il punteggio pieno: considerandoli come un unico item, la media scende a 11,1 su 13, senza variazioni nel quadro complessivo.

Il dato più solido riguarda le formule incontrate nelle interazioni situate a più alta frequenza: *Biglietto, prego*, *per favore*, *Nessun problema*, *Scusa* e *Arrivederci* sono state riconosciute da tutti o quasi tutti i partecipanti. Gli errori si concentrano invece in tre punti. Il primo è la scena della casa, l'unica priva di interazione e di produzione orale: *C'è un divano* è l'item con il punteggio più basso (8/15). Il secondo è la risposta a *Grazie*: cinque partecipanti scelgono *Per favore* anziché *Prego*, confermando la confusione tra le due formule che i distrattori del bar erano stati progettati per sollecitare (§3.4). Il terzo è la sovrapposizione tra domande strutturalmente simili dello stesso dialogo: cinque partecipanti interpretano *Sei nuovo qui?* come *Dove vai?*, che Lina pone due battute dopo.

Il confronto tra quartiere e bar, previsto per esplorare il contributo dei sottotitoli alla ritenzione, non mostra un vantaggio della scena sottotitolata: gli item del bar, privi di traduzione inglese, ottengono una percentuale leggermente superiore. Il dato va letto con la cautela già dichiarata al §3.5, perché le due scene differiscono per più variabili contemporaneamente e gli item non sono di difficoltà equivalente. Le risposte aperte del questionario offrono una lettura soggettiva dello stesso confronto. Motivando la scelta della scena in cui ritenevano di aver imparato di più (item 31 e 32), quattro partecipanti citano esplicitamente i sottotitoli come supporto alla comprensione (P03, P10, P13, P15) e altri due la chiarezza delle situazioni (P04, P09). Chi indica il bar, invece, lo motiva con la pratica più che con la comprensione: *the bar got me practising my pronounce* (P02). Nessun partecipante segnala l'assenza della traduzione nelle opzioni del bar come un ostacolo. Il quadro suggerisce che i due livelli di supporto siano stati percepiti come funzioni diverse.

L'autovalutazione è coerente con questo quadro: alla domanda su quanto ritengano di aver imparato, undici partecipanti rispondono 4 su 5, uno 5 e tre 3 (media 3,9). Come parte dell'esperienza più utile all'apprendimento viene indicata con maggiore frequenza la scena del controllore (9 menzioni), seguita dalla guida grammaticale nell'inventario (5), dal bar (4), dalla casa (3) e dall'incontro con Lina (3); tre partecipanti non indicano alcuna parte in particolare.

RQ2. Tenuta del riconoscimento vocale

Riconoscimento automatico Escluse la cattura del nome e le domande al tutor, il log registra 497 tentativi vocali contando i dialoghi e l'esercizio di *listen-and-repeat*. Di questi, 42 (8%) sono stati classificati come MIC, perché dovuti a problemi di acquisizione del segnale: attivazione del microfono in ritardo (24), accidentale (12) o interruzione brusca (6). Le tre voci hanno origini diverse, riconducibili alla gestione del microfono descritta al §4.3. Le attivazioni in ritardo, le più numerose, dipendono dall'attesa di

un secondo tra il riconoscimento del gesto e l'apertura del microfono: i partecipanti che iniziavano a parlare nel momento stesso in cui alzavano l'indice perdevano la parte iniziale dell'enunciato. Il segnale sonoro introdotto dopo le sessioni pilota non elimina il problema, perché indica l'inizio del turno e non l'istante in cui il microfono si apre. Le attivazioni in ritardo risultano comunque meno frequenti dopo la sua introduzione: 7 nelle due sessioni pilota, in media 3,5 per sessione, contro 17 nelle tredici successive, in media 1,3 per sessione, di cui 9 concentrate in un solo partecipante. Il segnale era stato pensato per aiutare l'apprendente a capire quando fosse il suo turno, non per indicare l'istante in cui il microfono inizia a registrare, che dipende comunque dal gesto. Si può dire che le attivazioni accidentali dipendono invece dal criterio di rilevamento della posa, che considera soltanto la direzione dell'indice e la chiusura delle altre dita: in alcuni casi l'indice veniva rilevato come puntato verso l'alto anche con la mano abbassata, avviando la registrazione senza che il partecipante intendesse parlare. Parte delle interruzioni brusche, infine, è compatibile con la chiusura automatica per silenzio, che può scattare durante una pausa di esitazione a metà frase. Un altro tentativo è stato escluso in sede di valutazione. I tentativi analizzati sono quindi 454. Considerando per ciascun partecipante il primo tentativo valido su ogni nodo di dialogo e su ogni parola dell'esercizio (306 casi), il riconoscimento riesce nel 62% dei casi: 66% nei dialoghi del quartiere, 67% nei dialoghi del bar, 56% nel *listen-and-repeat*. Il dato individuale varia molto, dal 21% all'85% (mediana 70%). Nonostante un primo tentativo fallito in quasi quattro casi su dieci, l'interazione procede: il 98% dei nodi del quartiere, il 97% di quelli del bar e il 95% delle parole del *listen-and-repeat* vengono completati, con una media di 1,5 tentativi per nodo. Whisper, in altri termini, rallenta l'interazione ma non la blocca. Nel bar, l'esito positivo del riconoscimento indica che la trascrizione corrisponde a una delle opzioni del nodo, non necessariamente a quella corretta: in alcuni casi è stato riconosciuto un distrattore, per esempio *Va bene, per favore* in risposta a *Desidera?*. Il riconoscimento misura quindi la tenuta tecnica del sistema, non l'appropriatezza pragmatica della scelta.

Nel *listen-and-repeat*, la riuscita al primo tentativo varia sensibilmente da parola a parola (Tabella 5). Le parole con il punteggio più basso sono quelle che contengono i fenomeni più lontani dall'inglese: il nesso iniziale /zg/ di *sgabello*, le consonanti doppie di *cappuccino*, *tazza* e *donna*, l'affricata /tts/ di *tazza*. Poiché l'esercizio è facoltativo, le percentuali si riferiscono a gruppi di partecipanti parzialmente diversi.

Parola	**Riuscita al primo tentativo**
sgabello	3/13 (23%)
cappuccino	3/11 (27%)
tazza	6/14 (43%)
frigorifero	5/11 (45%)
donna	5/10 (50%)
tavolino	5/9 (56%)
persona	9/14 (64%)
acqua	10/13 (77%)
uomo	9/11 (82%)
sedia	11/12 (92%)

Tabella 5: *Listen-and-repeat*: riuscita al primo tentativo valido per parola. L'esercizio è facoltativo: il denominatore indica quanti partecipanti hanno provato ciascuna parola.

Confronto con la valutazione percettiva La Tabella 6 riporta la distribuzione dei 454 tentativi nelle sei categorie della griglia.

Categoria	**Totale**	**%**	**Quartiere**	**Bar**
MG, match genuino	240	52,9	97	143
MI, match su pronuncia insufficiente	50	11,0	16	34
OC, *over-correction*	22	4,8	12	10
FN, falso negativo	20	4,4	5	15
RC, rifiuto corretto	89	19,6	38	51
TI, trascrizione infedele	33	7,3	8	25
Totale	454	100	176	278

Tabella 6: Distribuzione dei tentativi nelle categorie della valutazione percettiva.

Whisper ha trascritto fedelmente l'88% dei tentativi (399 su 454). Il confronto tra l'esito del riconoscimento e il giudizio sulla pronuncia mostra un accordo nell'81% dei casi: il sistema accetta ciò che un ascoltatore madrelingua avrebbe compreso e rifiuta ciò che non avrebbe compreso. Il disaccordo, circa un caso su cinque, si distribuisce in modo pressoché simmetrico nelle due direzioni. Da un lato, il 19% dei tentativi accettati (60 su 312) corrisponde a una pronuncia giudicata non accettabile: sono le categorie MI e parte

di OC, in cui il sistema risulta troppo permissivo. Dall'altro, il 18% dei tentativi rifiutati (26 su 142) corrisponde a una pronuncia giudicata accettabile: sono i falsi negativi e parte delle trascrizioni infedeli, in cui il sistema risulta più rigido dell'orecchio umano. Le due categorie indicate come più rilevanti al §5.2.4, OC e FN, hanno dimensioni analoghe e contenute (rispettivamente 4,8% e 4,4%): il riconoscimento automatico, nel complesso, rispecchia il giudizio umano, ma non in modo abbastanza fine da poter sostituire la valutazione percettiva. Una parte di questa permissività dipende dalla configurazione dei *trigger*: quando un'opzione è riconosciuta anche attraverso una sola parola chiave, il sistema accetta risposte incomplete o parzialmente errate, purché contengano quella parola.

Fenomeni fonologici Nei 454 tentativi dell'intero campione sono state annotate 256 occorrenze di fenomeni fonologici, distribuite su 198 tentativi (Tabella 7). Rispetto alle sessioni iniziali (Tabella 3), l'ordine di frequenza cambia: il fenomeno più diffuso è lo scempiamento delle doppie (GEM, 51 occorrenze in 13 partecipanti su 15), seguito dalla resa non vibrante di /r/ (RHO, 38), dalla sostituzione consonantica (SUB, 31), dal timbro vocalico inglese (VOC-d, 29), dall'ipercorrettismo geminato (GEM+, 28) e dalla segmentazione anomala (FUS, 27).

Codice	**Occorrenze**	**MG**	**MI**	**OC**	**FN**	**RC**	**TI**
GEM	51	2	3	4	5	29	8
RHO	38	3	7	8	1	13	6
SUB	31	0	0	2	2	22	5
VOC-d	29	2	6	3	3	7	8
GEM+	28	2	6	3	1	14	2
FUS	27	0	1	1	2	16	7
VOC-e	12	2	5	1	0	4	0
ELO	12	1	1	1	0	6	3
GR	10	0	2	0	1	5	2
ALTRO	7	1	1	0	1	1	3
VOC-a	6	0	0	0	1	4	1
PAL	5	0	0	0	1	2	2
Totale	256	13	32	23	18	123	47

Tabella 7: Fenomeni fonologici per categoria di valutazione (Fenomeno 1 e Fenomeno 2, tutti i partecipanti).

L'incrocio tra fenomeni e categorie distingue due comportamenti di Whisper. I fenomeni che alterano la forma della parola in modo visibile nella trascrizione, come lo scempiamento (GEM), la sostituzione consonantica (SUB) e la segmentazione anomala (FUS),

portano per lo più a un rifiuto corretto: Whisper li trascrive fedelmente e il confronto con la forma attesa fallisce, come ci si aspetterebbe. La resa non vibrante di /r/ (RHO) segue invece un percorso diverso: è il fenomeno con il maggior numero di casi OC (8 su 38), perché Whisper tende a normalizzare la /r/ inglese verso la grafia italiana, rendendo il fenomeno invisibile nel log e udibile soltanto all'ascolto. È la conferma più diretta della necessità di affiancare al log una valutazione percettiva: sulla base del solo testo trascritto, la difficoltà più diffusa nella pronuncia dei partecipanti resterebbe in larga parte nascosta.

Supporti, modalità di emissione, percezione dei partecipanti Nei tentativi in cui il partecipante consultava i pannelli di supporto il riconoscimento riesce nel 71% dei casi, contro il 56% dei tentativi senza pannelli; i tentativi scanditi, con parole separate e articolate con enfasi, sono riconosciuti meno spesso di quelli in modalità neutra (62% contro 71%). Entrambi i dati vanno letti come associazioni descrittive e non come effetti causali: la scansione, in particolare, compare spesso come reazione a un tentativo già fallito. Il questionario conferma il quadro dei log. Tutti i partecipanti dichiarano di aver dovuto ripetere almeno una frase: con Lina tutti e quindici, con la barista dodici, con gli oggetti del bar sei. Undici partecipanti su quindici dichiarano di aver sentito il bisogno di parlare più lentamente o più chiaramente per essere compresi (item 50), soprattutto nei dialoghi con Lina e con la barista (item 51). Per alcuni, tuttavia, l'adattamento è stato vissuto come un aiuto più che come un ostacolo: *Speaking slower got me being more clear* (P03); *speaking slower helped my confidence especially after the first dialogue* (P15). La stessa tendenza emerge dalla griglia di valutazione, dove 103 tentativi sono stati pronunciati in modalità scandita. Nonostante lo sforzo richiesto, otto partecipanti trovano più naturale parlare italiano ad alta voce nel gioco, sei non notano differenze e soltanto uno lo trova più difficile.

RQ3. Dati comportamentali come strumento di misurazione

Poiché il modulo post-sessione non consente di collegare le risposte ai singoli log (§5.2.1), il confronto è condotto a livello di gruppo, affiancando per ciascuna formula la percentuale di risposte corrette al *post-test* e la percentuale di riconoscimento, al primo tentativo, del nodo in cui la formula è stata incontrata o prodotta (Tabella 8).

Formula	***Post-test***	**Nodo di gioco**	**Primo tentativo**
Nessun problema	15/15 (100%)	Lina, *Ops, scusa!*	8/15 (53%)
Sei nuovo qui?	10/15 (67%)	Lina, *Sei nuovo qui?*	10/15 (67%)
Piacere!	12/15 (80%)	Lina, *bel nome!*	9/15 (60%)
Per favore	15/15 (100%)	Barista, *Desidera?*	13/14 (93%)
Cornetto	13/15 (87%)	Barista, *Vuole ordinare qualcosa?*	4/9 (44%)
Arrivederci	14/15 (93%)	Barista, congedo	9/15 (60%)

Tabella 8: Confronto tra *post-test* e riconoscimento in gioco per le formule comuni ai due strumenti.

I due indicatori divergono in modo sistematico: le percentuali del *post-test* sono pari o superiori a quelle del riconoscimento in gioco in tutti i casi, con lo scarto più ampio, di 47 punti percentuali, nel caso di *Nessun problema*. La divergenza ha una spiegazione strutturale: il *post-test* misura il riconoscimento del significato, una competenza ricettiva, mentre il log misura la produzione orale, filtrata a sua volta dalla tenuta tecnica di Whisper. Un partecipante può comprendere perfettamente *Arrivederci* e non riuscire, al primo tentativo, a pronunciarlo in modo che il sistema lo riconosca. L'unico caso di convergenza, *Sei nuovo qui?* (67% in entrambi gli strumenti), coincide con la formula che nel *post-test* ha generato più confusione.

Ne segue che, in questo prototipo, i dati comportamentali non possono sostituire il *post-test*: sottostimano la conoscenza ricettiva dei partecipanti e mescolano la difficoltà linguistica con quella tecnica del riconoscimento. Possono però integrarlo, perché registrano una dimensione che il *post-test* a scelta multipla non osserva: la capacità di produrre oralmente la formula nel momento in cui serve. La risposta a RQ3 è quindi di complementarità, non di equivalenza, con il limite di un confronto condotto sul gruppo e su un numero ridotto di formule.

RQ4. Usabilità e senso di presenza

Il punteggio medio della System Usability Scale è di 86,7 su 100 (deviazione standard 10,2; mediana 90; intervallo 60–100). Tredici partecipanti su quindici superano la soglia di 70 indicata da Bangor, Kortum e Miller come limite dell'usabilità accettabile; i due punteggi inferiori sono 67,5 e 60. Gli item con i valori più favorevoli riguardano la coerenza del sistema (*too much inconsistency*, media 1,0 su 5) e l'integrazione delle funzioni (*well integrated*, 4,9); i meno favorevoli, pur restando positivi, riguardano la facilità d'uso (*easy to use*, 3,7) e la sicurezza percepita (*confident using the game*, 3,9),

coerenti con le difficoltà nell'attivazione del microfono documentate dalla categoria MIC.

Il punteggio medio di presenza è di 4,36 su 5 (deviazione standard 0,47; intervallo 3,6–5,0), e il 92% delle risposte si colloca sui valori 4 o 5. L'item con il valore più alto riguarda il senso di trovarsi fisicamente nell'ambiente (*being there*, 4,53), insieme al ricordo dell'ambiente come luogo visitato piuttosto che come insieme di immagini (4,53); il più basso riguarda i momenti in cui l'esperienza è stata percepita come reale (4,13). Le risposte all'item 52 precisano da dove nasca il senso di presenza. Dodici partecipanti dichiarano senza riserve di aver avuto l'impressione di parlare con una persona reale, e solo due la qualificano, pur apprezzandone la resa: *I still could tell they were virtual people but I liked that they had voices and were moving* (P11). Le motivazioni si concentrano su due aspetti: la voce, le animazioni e i gesti dei personaggi (P06, P09, P11, P15), e la plausibilità delle conversazioni, percepite come scambi che si potrebbero avere nella realtà (P03, P04, P10). Nei termini di Steuer (§2.1), i partecipanti attribuiscono dunque la presenza soprattutto alla *vividness* dei personaggi e alla credibilità delle situazioni.

Usabilità e presenza risultano quindi elevate per l'intero gruppo. Questo consente di leggere i risultati delle altre domande di ricerca con maggiore sicurezza: le difficoltà registrate in RQ2 dipendono dal riconoscimento vocale e dalla pronuncia, e il buon esito del *post-test* non può essere attribuito a un sistema difficile da usare.

5.4 Discussione

Implicazioni dei risultati

Letti insieme, i risultati delineano un quadro coerente. I partecipanti, tutti principianti assoluti, lasciano la sessione riconoscendo la maggior parte delle formule incontrate (RQ1), e lo fanno all'interno di un ambiente che giudicano usabile e in cui si sentono presenti (RQ4). Il riconoscimento vocale è il punto di attrito del sistema: fallisce spesso al primo tentativo, ma non impedisce di portare a termine l'interazione (RQ2). Il log, infine, restituisce un'immagine della competenza diversa da quella del *post-test*, perché osserva un'altra abilità (RQ3).

Il primo dato da discutere è la distribuzione dei risultati del *post-test* per scena. Le formule meglio ricordate sono quelle legate a un'azione o a un'interazione situata: *Biglietto, prego*, associata alla consegna fisica del biglietto, raggiunge il punteggio pieno; le formule della scena domestica, presentate su pannelli e prive di interazione e di produzione orale, ottengono i risultati più bassi. Il confronto non è controllato, perché le scene differiscono per più variabili, ma è coerente con i presupposti dell'apprendimento incarnato e del *Task-Based Language Teaching* discussi ai §2.4.4 e §2.4.5: ciò che il giocatore deve usare per avanzare viene ricordato meglio di ciò che si limita a leggere. Gli errori ricorrenti offrono inoltre indicazioni progettuali precise. La confusione tra *Prego* e *Per favore*, prevista e sollecitata dai distrattori del bar, sopravvive alla sessione

e segnala un contrasto che richiede più esposizione di quanta il prototipo ne offra; la sovrapposizione tra *Sei nuovo qui?* e *Dove vai?* suggerisce che due domande ravvicinate nello stesso dialogo tendono a fondersi nel ricordo. A questo fattore se ne può affiancare un secondo, osservato durante le sessioni ma non verificabile con i dati raccolti: nel quartiere le battute di Lina sono accompagnate dal sottotitolo inglese, e il partecipante può comprendere la domanda leggendo la traduzione, senza elaborare la forma italiana. Il supporto della L1, in altri termini, potrebbe facilitare l'interazione nel momento in cui avviene e ridurre al tempo stesso l'attenzione rivolta all'italiano. L'ipotesi è coerente con l'assenza di un vantaggio della scena sottotitolata al *post-test*, ma non basta da sola a spiegare la confusione, dal momento che altre formule sottotitolate, come *Nessun problema*, sono state riconosciute da tutti i partecipanti.

Un ulteriore fattore che può contribuire a spiegare le differenze osservate tra le scene è la ripetizione delle associazioni tra forma linguistica e referente concreto. Nell'ambiente virtuale, alcune parole vengono presentate più volte in relazione allo stesso oggetto tridimensionale e all'interazione con esso. La reiterazione della stessa forma linguistica in presenza di un referente stabile può contribuire al consolidamento della traccia lessicale, rafforzando progressivamente l'associazione tra la parola italiana e il relativo significato. Questo meccanismo assume particolare rilievo per apprendenti principianti assoluti, per i quali il significato di una nuova parola non può essere ricavato da conoscenze pregresse della lingua target. Nel caso del *listen and repeat*, ad esempio, le formule non vengono presentate esclusivamente come elementi linguistici, ma ricorrono all'interno di una sequenza d'azione che comprende l'ascolto, la ripetizione e la manipolazione fisica dell'oggetto associato. La ripetizione della forma linguistica all'interno di un contesto percettivo e operativo stabile può quindi aver contribuito al suo riconoscimento nel *post-test*. Tale interpretazione resta tuttavia un'ipotesi, poiché il disegno dello studio non consente di separare l'effetto della ripetizione da quello dell'interazione situata, della multimodalità e della rilevanza della formula per il completamento del compito.

Il secondo dato riguarda il rapporto tra riconoscimento automatico e giudizio umano. Un accordo dell'81% indica che il sistema, nel complesso, premia ciò che un madrelingua comprenderebbe e rifiuta ciò che non comprenderebbe. Ma il disaccordo residuo non è casuale: si distribuisce in modo simmetrico tra eccessiva permissività ed eccessiva rigidità, e colpisce in modo diverso i diversi fenomeni. Lo scempiamento delle doppie, la sostituzione consonantica e la segmentazione anomala vengono trascritti fedelmente e portano per lo più a un rifiuto corretto; la resa non vibrante di /r/, tra i fenomeni più diffusi nella produzione dei partecipanti, viene invece normalizzata da Whisper e scompare dal testo trascritto. Ne derivano due conseguenze. Sul piano didattico, il sistema di dialogo non può fornire un *feedback* affidabile sulla pronuncia di /r/, che è proprio uno dei tratti più marcati dell'accento anglofono. Sul piano metodologico, un'analisi della pronuncia condotta sul solo log sottostimerebbe sistematicamente le difficoltà reali dei partecipanti: la valutazione percettiva non è un complemento facoltativo, ma la condizione per interpretare i dati automatici.

Un altro punto di discussione riguarda il valore dei dati comportamentali come misura dell'apprendimento. La divergenza sistematica tra *post-test* e riconoscimento in gioco non va letta come un fallimento di uno dei due strumenti, ma come l'effetto della differenza tra competenza ricettiva e produttiva, a cui nel caso del log si somma il filtro del riconoscimento automatico. Il log, in altre parole, misura contemporaneamente il discente e il sistema. Per usarlo come indicatore di apprendimento occorrerebbe separare le due componenti, per esempio incrociando in modo sistematico il log con la valutazione percettiva, come fatto in questo studio per RQ2. Allo stato attuale, i dati comportamentali sono più utili come strumento diagnostico, capace di indicare dove e perché l'interazione si inceppa, che come misura sostitutiva del *post-test*.
I punteggi elevati di usabilità e presenza rendono più leggibili i risultati precedenti. Le difficoltà di RQ2 non sono attribuibili a un'interfaccia inaccessibile, e l'unico aspetto dell'interazione che emerge come problematico, l'attivazione del microfono tramite gesto, è documentato in modo indipendente dalla categoria MIC e dai valori meno favorevoli della SUS sulla facilità d'uso e sulla sicurezza percepita. Il dato va letto come effetto di un compromesso progettuale più che di un errore. L'attesa di un secondo, responsabile della maggior parte delle attivazioni in ritardo, era stata introdotta proprio per evitare che il microfono registrasse ciò che il giocatore pronunciava prima della frase italiana e per filtrare le attivazioni involontarie: abbreviarla avrebbe ridotto i ritardi, ma aumentato i tentativi contaminati da parlato non pertinente. Due osservazioni, raccolte dalla ricercatrice durante le sessioni e non registrate dal log, completano il quadro. La prima riguarda la scena del chitarrista: nessun partecipante ne ha attivato il dialogo, che parte soltanto quando il giocatore deposita una banconota da dieci euro nella custodia della chitarra, ma almeno tre partecipanti hanno provato a manipolare direttamente le monete (Figura 10). Il compito era quindi riconoscibile, mentre non lo era il gesto necessario per avviarlo: un problema di *affordance* che nessun elemento della scena segnalava, e che indica la necessità di rendere esplicito il punto di ingresso delle interazioni opzionali. La seconda riguarda la scena della casa, dove circa dieci partecipanti su quindici hanno usato spontaneamente il giradischi, in assenza di qualsiasi richiesta. Il dato mostra che gli elementi privi di obbligo e di valutazione sono stati cercati e scelti, in linea con il ruolo del *piacere* nella motivazione dell'apprendente adulto (§2.3.1) e con i valori elevati di presenza (RQ4). Va letto, tuttavia, come indice di coinvolgimento e non di apprendimento: la scena della casa registra i risultati più bassi al *post-test*, e la canzone lavora sul sistema dei pronomi personali, che il *post-test* non verifica.



Figura 10: Le monete nella custodia del chitarrista: il compito è riconoscibile, il gesto che lo avvia no.

Tutti i quindici partecipanti consiglierebbero il gioco a chi vuole imparare l'italiano (item 54), motivando la scelta con il divertimento e con la possibilità di praticare il parlato in contesto; due lo condizionano alla familiarità con la realtà virtuale (P04, P12), in linea con la curva di adattamento emersa nelle prime interazioni. Il dato va letto con cautela, perché un'esperienza nuova e breve tende a produrre giudizi favorevoli per effetto della novità stessa. Più indicative sono le proposte di modifica (item 53): otto partecipanti non ne indicano, mentre le altre riguardano l'ampliamento dei contenuti, con più scene e situazioni (P09, P15), più canzoni per il giradischi (P02) e più oggetti afferrabili (P03), e l'introduzione dei gesti italiani (P05), quest'ultima discussa con il partecipante in una conversazione informale al termine della sessione. Sono indicazioni che coincidono con gli sviluppi proposti al §6.3.

Limiti metodologici dello studio

I risultati vanno letti alla luce di alcuni limiti metodologici, distinti dai limiti del prototipo discussi nelle conclusioni. Il primo riguarda il disegno. Lo studio adotta un disegno di solo *post-test* su un unico gruppo, senza pre-test, senza gruppo di controllo e senza verifica a distanza di tempo. Il livello di partenza dei partecipanti è stabilito per autovalutazione e, per quanto tutti si dichiarino principianti assoluti, il *post-test* non può escludere il contributo della trasparenza lessicale di alcune parole, come *cappuccino* o *problema*, né quello della risposta per esclusione in un formato a scelta multipla. Il *post-test* misura inoltre la ritenzione immediata e il solo riconoscimento del significato, non la produzione né la ritenzione a lungo termine.

Il secondo riguarda il campione. Quindici partecipanti, reclutati per convenienza e in larga parte provenienti dalla stessa area geografica, non consentono generalizzazioni né confronti tra varietà di inglese: l'analisi per accento prevista in fase di progettazione non è risultata praticabile. La presenza della ricercatrice come moderatrice, infine, può aver favorito risposte positive nei questionari di usabilità e presenza.

Il terzo riguarda la valutazione percettiva. La griglia è stata compilata da un'unica valutatrice: i criteri sono stati definiti a priori e calibrati sulle prime due sessioni, ma in assenza di un secondo valutatore indipendente non è stato possibile misurare l'affidabilità del giudizio. Le due sessioni pilota sono servite sia a costruire i codici dei fenomeni sia come dati dello studio. Le stesse, inoltre, si sono svolte senza due supporti all'interfaccia introdotti successivamente (§5.2.1).

Il quarto riguarda la raccolta dei dati. Il modulo post-sessione non ha registrato il `ParticipantId`, e il confronto previsto per RQ3 ha potuto essere condotto soltanto a livello di gruppo e su un numero ridotto di formule. Il *listen-and-repeat*, essendo facoltativo, è stato svolto da gruppi di partecipanti diversi per ciascuna parola, e le percentuali per parola non sono quindi pienamente confrontabili. Durante le sessioni, infine, le difficoltà di interazione sono documentate soltanto in modo indiretto, attraverso la categoria MIC e le risposte ai questionari.

Nessuno di questi limiti invalida i risultati, ma tutti ne delimitano la portata: le risposte alle quattro domande di ricerca vanno intese come indicazioni esplorative su questo prototipo, questi partecipanti e queste condizioni di test, coerentemente con l'impostazione della *Design-Based Research* descritta al §3.1.

6 Conclusioni

Questa tesi è partita da una domanda semplice da formulare e tutt'altro che semplice da verificare: può un ambiente di realtà virtuale, costruito attorno a compiti comunicativi situati e governato dalla voce del giocatore, accompagnare un adulto anglofono che non conosce una parola di italiano nei suoi primi passi nella lingua? *ItalianVRExperience* è la risposta progettuale a quella domanda: un paese che non esiste, ma in cui si arriva in treno, si porge il biglietto al controllore, si fa conoscenza con la vicina e si ordina un cappuccino al bar, pronunciando ad alta voce ogni battuta. Quindici partecipanti lo hanno attraversato, e il loro passaggio ha messo alla prova il prototipo su tre piani: didattico, tecnico e d'uso. Questo capitolo raccoglie ciò che quel passaggio ha rivelato: le risposte alle quattro domande di ricerca, i limiti che appartengono all'artefatto e le direzioni lungo cui il lavoro può proseguire.

RQ1 Dopo una sola sessione di circa trenta minuti, partecipanti che non avevano mai studiato l'italiano riconoscono in media 12,1 formule su 14 (86%), e nessuno scende sotto il 71%. Il disegno di solo *post-test* impone cautela, ma la distribuzione dei risultati racconta qualcosa di preciso: *Biglietto, prego*, legata al gesto fisico di porgere il biglietto, raggiunge il punteggio pieno, mentre la casa, l'unica scena in cui il giocatore legge senza mai dover parlare, registra i risultati più bassi (67%). Ciò che serve per andare avanti resta; ciò che si limita a scorrere davanti agli occhi, molto meno. È un risultato coerente, su scala ridotta, con i presupposti dell'apprendimento incarnato e del *Task-Based Language Teaching*. Sul ruolo dei sottotitoli, invece, lo studio resta aperto: il confronto tra quartiere e bar non mostra alcun vantaggio della traduzione inglese, e non consente conclusioni.

RQ2 Whisper riconosce la risposta al primo tentativo nel 62% dei casi, con forti differenze da un partecipante all'altro. Eppure la conversazione non si inceppa: dal 95 al 98% dei nodi vocali viene completato, con una media di 1,5 tentativi per nodo. Il riconoscimento automatico rallenta il dialogo, ma non lo interrompe. Il confronto con l'ascolto umano è altrettanto incoraggiante: il sistema concorda con il giudizio di un ascoltatore madrelingua nell'81% dei tentativi, e i due errori più insidiosi per l'uso didattico, l'*over-correction* e il falso negativo, restano contenuti (4,8% e 4,4%). Il disaccordo residuo, però, non è casuale. Lo scempiamento delle doppie e gli altri fenomeni che alterano la forma scritta della parola vengono trascritti fedelmente e

rifiutati a ragione; la /r/ non vibrante, uno dei tratti più diffusi nella pronuncia dei partecipanti, viene invece ricondotta da Whisper alla grafia italiana e scompare dal log (8 casi di *over-correction* su 38 occorrenze). La risposta a RQ2 ha quindi due facce: il sistema regge abbastanza da sostenere il dialogo, non abbastanza da sostituire l'orecchio umano.

RQ3 Per tutte le formule comuni ai due strumenti, il *post-test* restituisce percentuali pari o superiori a quelle del riconoscimento in gioco, con uno scarto che nel caso di *Nessun problema* arriva a 47 punti percentuali. Non è una contraddizione, ma la differenza tra due abilità: il *post-test* chiede di riconoscere il significato di una formula, il log registra la capacità di pronunciarla nel momento in cui serve, filtrata a sua volta dalla tenuta tecnica di Whisper. Il log, in altre parole, osserva nello stesso istante il discente e il sistema. Per questo i dati comportamentali non possono sostituire il *post-test*, ma possono affiancarlo, portando alla luce una dimensione che la scelta multipla non raggiunge. La risposta a RQ3 è di complementarità, non di equivalenza, entro i limiti di un confronto condotto sul gruppo.

RQ4 Il punteggio medio della System Usability Scale è di 86,7 su 100, tredici partecipanti su quindici superano la soglia dell'usabilità accettabile, e il senso di presenza raggiunge in media 4,36 su 5. Sono valori notevoli per un gruppo in cui undici persone su quindici non avevano mai indossato un visore. Nel disegno dello studio, il loro valore è soprattutto interpretativo: dicono che le difficoltà emerse in RQ2 nascono dal riconoscimento vocale e dalla pronuncia, non da un ambiente ostile, e che il buon esito del *post-test* non si deve a un sistema che si lascia usare a fatica. L'unico punto d'attrito, l'attivazione del microfono tramite gesto, emerge in modo concorde dai log, dalla categoria MIC e dagli item meno favorevoli della SUS.

Messe una accanto all'altra, le quattro risposte descrivono un prototipo che funziona come luogo di primo contatto con la lingua: coinvolge, si lascia usare e lascia una traccia misurabile. Il suo punto più fragile coincide con il suo tratto più distintivo, la voce, ed è proprio lì che si concentrano tanto i limiti quanto le possibilità più interessanti.

6.1 Limiti del prototipo

I limiti metodologici dello studio sono stati discussi al §5.4. Qui l'attenzione si sposta sull'artefatto: indipendentemente da come è stato messo alla prova, il prototipo porta con sé alcuni vincoli che è giusto dichiarare, perché definiscono il perimetro entro cui i risultati vanno letti.

Un limite fondamentale riguarda il sillabo. Come dichiarato al §3.2, la selezione delle funzioni comunicative è riduzionista per scelta metodologica: il repertorio acquisibile al termine dell'esperienza è un sottoinsieme minimo del livello A1, orientato alla sopravvivenza comunicativa immediata. Funzioni rilevanti del PLI A1, come chiedere e dire l'ora, chiedere il prezzo o descrivere persone, non sono incluse, e della macrocategoria relativa

agli stati emotivi è rappresentata soltanto l'espressione della soddisfazione. Questo limite è costitutivo di qualsiasi sillabo di sopravvivenza, come documentato da Nation e Crabbe, ma va riconosciuto esplicitamente come vincolo del prototipo attuale.

Il sistema di dialogo presenta a sua volta limiti strutturali. Le interazioni con gli NPC sono interamente predefinite: le battute sono fisse, le opzioni di risposta chiuse, la variazione pragmatica limitata a quanto previsto dagli alberi di dialogo. La scelta è motivata teoricamente, perché consente di controllare la complessità dell'input al livello A1, ma preclude qualsiasi forma di interazione spontanea: il discente che vuole uscire dal copione non ha strumenti per farlo. Anche il *feedback* correttivo risente di questa impostazione. L'unica reazione all'errore è la richiesta di chiarimento della barista, attivata dalla scelta di un distrattore; nessuna scena reagisce a come una formula è stata pronunciata, ma soltanto a quale formula è stata scelta.

La dipendenza da Whisper è il limite più vincolante di tutti, correlato all'attivazione del microfono. Il sistema funziona in locale e non richiede connessione di rete, ma la qualità del riconoscimento è legata alle caratteristiche del modello `ggml-small` e alla variabilità fonetica degli accenti anglofoni. Come documentato al §, alcuni fenomeni sistematici producono esiti che il sistema non è in grado di distinguere da una produzione genuinamente corretta o scorretta. A questo si aggiunge la rigidità del confronto tra trascrizione e risposta attesa. `VoiceMatchUtils` verifica la presenza di un *trigger* come sequenza di parole intere, senza alcuna misura di somiglianza tra stringhe: una trascrizione che si discosta anche di una sola lettera dalla forma attesa porta al rifiuto, indipendentemente dalla gravità dello scarto. Il sistema, inoltre, non sfrutta la confidenza che Whisper associa a ciascuna trascrizione, un'informazione che potrebbe distinguere un riconoscimento sicuro da uno incerto e che rientra tra gli sviluppi proposti al §6.3. Per quanto riguarda, infine, l'attivazione del microfono, l'8% dei tentativi vocali registrati è stato escluso dall'analisi per problemi di acquisizione, riconducibili all'attesa di sicurezza prima dell'apertura del microfono e al criterio di rilevamento della posa (§). Si tratta di dati contenuti, ma sufficienti a produrre frustrazione in un'esperienza in cui la voce è il canale primario di interazione.

6.2 Sviluppi futuri

Nella logica della *Design-Based Research* descritta al §3.1, un limite non è un punto di arrivo, ma l'inizio del ciclo successivo. Le direzioni di sviluppo che seguono nascono direttamente da ciò che lo studio ha mostrato e dagli stessi limiti del prototipo, e riguardano tanto l'artefatto quanto il modo di studiarlo.

Tra le aspettative future, la principale riguarda l'introduzione di un sistema di *feedback* sulla pronuncia, annunciato al §3.4. I risultati di RQ2 indicano una possibile direzione di sviluppo. I fenomeni che producono uno scarto tra la forma attesa e quella restituita dalla trascrizione, come lo scempiamento (*donna* → 'dona"), la sostituzione consonantica (*tazza* → 'tassa") o la segmentazione anomala, possono essere rilevati

mediante il confronto tra la risposta attesa e l'output di Whisper. Su questa base, un modello linguistico locale potrebbe classificare lo scarto e formulare un *feedback* mirato, distinguendo tuttavia tra errore di pronuncia e possibile errore di riconoscimento automatico. Ollama, già impiegato nel sistema per il tutor, potrebbe quindi essere utilizzato per la generazione strutturata del *feedback*, mentre l'analisi del segnale acustico e la valutazione propriamente fonetica richiederebbero componenti specifiche. Il gioco potrebbe così segnalare, senza uscire dalla finzione, che la barista ha capito "tassa" e non "tazza". La resa di /r/ richiede invece una soluzione diversa, perché la trascrizione ortografica la rende invisibile: servirebbe un riconoscimento a livello fonetico, capace di osservare la pronuncia prima che venga ricondotta alla grafia italiana.
Un secondo intervento di sviluppo riguarderebbe l'attivazione del microfono, per conciliare due esigenze opposte emerse dallo studio: non perdere l'inizio dell'enunciato e non registrare ciò che lo precede. Un primo passo è già stato compiuto dopo la conclusione delle sessioni, come nuovo ciclo di revisione nato direttamente dai risultati di RQ2: il segnale sonoro è stato spostato dalla fine della battuta dell'NPC all'istante in cui il microfono si apre effettivamente, così che il giocatore sia invitato a parlare solo quando la registrazione è attiva. La modifica mantiene l'attesa di sicurezza senza tagliare la frase, ma non è stata ancora valutata con nuovi partecipanti. Il rilevamento della posa, a sua volta, potrebbe tenere conto della posizione della mano rispetto alla testa, oltre che della direzione dell'indice, così da ridurre le attivazioni involontarie. Una direzione alternativa, già sperimentata durante lo sviluppo, riguarda il gesto stesso: il componente `GestureSpeechInput` prevedeva un secondo gesto di attivazione, ispirato al gesto italiano delle dita raccolte verso l'alto, scartato perché riconosciuto in modo meno affidabile dal sistema di *hand tracking*. Un gesto culturalmente marcato di questo tipo resta tuttavia una possibilità interessante, perché unirebbe al comando tecnico un elemento della comunicazione non verbale italiana. La stessa prospettiva permetterebbe di recuperare un'idea accantonata durante lo sviluppo (§4.3): un modello umanoide completo, visibile allo specchio, come punto di partenza per una missione dedicata ai gesti italiani, che unirebbe l'apprendimento incarnato (§2.4.5) a una dimensione culturale della comunicazione oggi assente dal sillabo. La proposta è emersa anche in una conversazione informale con uno dei partecipanti al termine della sessione, che l'ha poi riportata nel questionario: *a lecture on Italian gestures could be cool*.

Tra gli sviluppi futuri dell'applicazione si considerano i contenuti, con l'obiettivo di trasformare il prototipo in un'esperienza di apprendimento più completa. Le due scene opzionali che lo studio ha messo in luce per ragioni opposte potrebbero diventare missioni principali: il giradischi, usato spontaneamente da circa due terzi dei partecipanti, potrebbe accogliere le altre canzoni proposte da Mezzadri nel *Cantagramma*, ciascuna con la propria trascrizione animata, così da estendere a nuove strutture grammaticali il percorso avviato con *Acqua azzurra, acqua chiara* (§3.5) e da unire il coinvolgimento osservato a un obiettivo linguistico esplicito. La missione del chitarrista, che nessun partecipante ha attivato (§), richiede invece un punto di ingresso più evidente, come un invito esplicito del personaggio o un'indicazione visiva sulla custodia, e potrebbe

essere inserita nel percorso obbligato. Il sillabo, infine, può proseguire in nuove scene: un supermercato, per esempio, consentirebbe di introdurre le funzioni del PLI A1 oggi escluse, come chiedere il prezzo e indicare quantità, riprendendo il lessico numerico della scena del chitarrista in un nuovo contesto di servizio. Uno studio successivo dovrebbe superare i limiti del disegno attuale: un pre-test e un *post-test* differito per misurare il guadagno e la ritenzione a lungo termine, un gruppo di controllo, un secondo valutatore indipendente per la griglia percettiva e un campione più ampio, che includa varietà di inglese rotiche e non rotiche. Collegare il modulo post-sessione ai log tramite il `ParticipantId`, infine, consentirebbe di condurre il confronto di RQ3 a livello individuale, verificando se il riconoscimento in gioco e il *post-test* convergono per lo stesso partecipante.

In termini di immersività, l'esperienza potrebbe essere perfezionata attraverso una maggiore apertura e continuità delle interazioni sociali. In particolare, le attività potrebbero essere organizzate intorno a obiettivi comunicativi, lasciando al partecipante una maggiore libertà nella scelta delle strategie linguistiche con cui raggiungerli. Invece di seguire esclusivamente dialoghi con risposte predefinite, il partecipante potrebbe, ad esempio, dover acquistare un prodotto, chiedere un'informazione o chiedere all'interlocutore di ripetere, scegliendo autonomamente come formulare la richiesta. A questo potrebbe essere associato un sistema di memoria degli NPC, in grado di conservare alcune informazioni relative alle interazioni precedenti e utilizzarle nelle situazioni successive. Le azioni compiute dal partecipante potrebbero inoltre produrre conseguenze persistenti tra le diverse scene: ciò che viene acquistato, le persone incontrate o le attività completate potrebbero modificare gli eventi successivi. In questo modo le scene non costituirebbero episodi indipendenti, ma parti di una situazione comunicativa più ampia e continua. Sarebbe interessante anche in maniera inerente alle interazioni spontanee con gli NPC. Gli interlocutori potrebbero reagire non soltanto alle risposte previste dal sistema, ma anche a comportamenti e iniziative non esplicitamente programmati, come un saluto, una richiesta di chiarimento o l'avvicinamento a un personaggio. L'obiettivo sarebbe quindi ridurre progressivamente la struttura fortemente predefinita dei dialoghi e costruire situazioni nelle quali il partecipante debba utilizzare le risorse linguistiche disponibili per raggiungere un obiettivo comunicativo all'interno dell'ambiente.

6.3 Considerazioni finali

ItalianVRExperience non insegna l'italiano in trenta minuti, e non ha mai avuto questa ambizione. Ha mostrato però qualcosa di più circoscritto e, forse, di più interessante: che un adulto principiante assoluto, messo nella condizione di dover parlare per andare avanti, esce da una breve sessione riconoscendo gran parte delle formule che ha usato, dentro un ambiente che trova facile da usare e in cui si sente, a tutti gli effetti, altrove. Uno dei partecipanti lo ha riassunto con una frase che nessun questionario a scala avrebbe potuto restituire: *talking slower helped my confidence*. Parlare con lentezza, sbagliare, ripetere, e scoprire che la conversazione non si interrompe: è forse questo il

primo apprendimento che *ItalianVRExperience* offre, prima ancora di qualsiasi formula.

Lo studio ha mostrato anche dove la tecnologia vocale, oggi, si ferma. Whisper sostiene il dialogo, ma non sa ancora ascoltare la pronuncia come farebbe un parlante nativo: capisce che cosa il giocatore ha detto, non come lo ha detto. Tra questi due risultati si apre lo spazio del lavoro futuro. Se la voce è ciò che distingue *ItalianVRExperience* da un'applicazione di vocabolario, è anche il terreno su cui il prototipo ha più da guadagnare: un sistema capace di accorgersi di come una battuta è stata pronunciata trasformerebbe ogni frase detta ad alta voce in un'occasione di apprendimento. Bellavalle continua a non esistere su nessuna mappa; ma chi ci è passato, anche solo per mezz'ora, vi ha consegnato il biglietto a un controllore, fatto conoscenza con una vicina, chiesto scusa e ordinato un cappuccino, ripetendo, sbagliando e riprovando fino a farsi capire.

APPENDICE A

Dialoghi e confronto con il PLI

01_Stazione, dialogo con il controllore

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Biglietto, prego.	Chiedere in contesto servizio	—		
-> [consegna fisica biglietto]			Rispondere accettando	Sì
Grazie.	Esprimere gratitudine	Grazie (mille)		
-> Dov'è l'uscita?			Informarsi sul luogo	Dove /. . . ?
Ecco, l'uscita è a destra.	Rispondere dando informazioni sul luogo	Avverbio (a destra)		
-> Grazie.			Esprimere gratitudine	Grazie (mille)
-> Scusi? [help]			Attirare attenzione / chiedere ripetizione	Scusi (formulaico)
Benvenuto in città! Arrivederci!	Augurare / Prendere congedo	Buon. . . / Arrivederci		
-> Arrivederci!			Prendere congedo	Arrivederci / Ciao / A. . .

02_Quartiere, presentazione

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Ops, scusa!	Scusarsi	Scusa (formulaico)		
-> Nessun problema.				
-> Attenta!				
Sei nuovo qui?	Informarsi su un oggetto/persona	Proposizione interrogativa		
-> Sì, sono nuovo.			Asserire	Sono + gruppo nominale/aggettivo
-> Sono appena arrivato.			Asserire	Sono + gruppo nominale/aggettivo
Ciao, come ti chiami?	Informarsi sul modo	Come /. . . ?		
-> [cattura nome vocale]			Presentarsi	Sono / Mi chiamo + Nome
{playerName}, bel nome!	Congratularsi	Molto bene! / Bravo!		
-> Grazie!			Esprimere gratitudine	Grazie (mille)
-> Piacere!			Rispondere a presentazione	Piacere
Dove vai?	Informarsi sul luogo	Dove /. . . ?		
-> Vado a casa.			Rispondere dando informazioni sul luogo	Preposizione (a casa)
-> Vado al bar.			Rispondere dando informazioni sul luogo	Preposizione (al bar)

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Io sono la tua vicina.	Presentarsi	Sono + Nome		
-> Piacere!			Rispondere a presentazione	Piacere
-> Che bello!			Esprimere soddisfazione	Che bello!
Vieni, ti porto in giro!	Proporre di fare qualcosa insieme	Andiamo!		

02_Quartiere, arrivo a casa

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Ecco casa tua, {playerName}!	Accogliere qualcuno	Entra / Vieni		
-> Grazie mille!			Esprimere gratitudine	Grazie (mille)
-> Perfetto!			Esprimere soddisfazione	Perfetto!
Cammina verso la porta per entrare.	Dare istruzioni	/		
-> Sì, va bene!			Rispondere accettando	Sì
-> Prego?			Chiedere ripetizione	Non ho capito (formulaico)
Bene! A domani, arrivederci!	Prendere congedo	Arrivederci / A domani		
-> Ciao, grazie!			Prendere congedo / Esprimere gratitudine	Ciao! / Grazie (mille)
-> A domani!			Prendere congedo	A…

02_Quartiere, scena episodica

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Suono la chitarra!	Asserire	Proposizione		
-> I soldi sono tuoi?			Informarsi su un oggetto/persona	Proposizione interrogativa
Grazie mille! Mi aiuti a contare le monete?	Esprimere gratitudine / Proporre di fare qualcosa insieme	Grazie (mille) / Andiamo!		
-> Certo, volentieri!			Rispondere accettando	Sì
-> Non ho capito.			Strutturare interazione	Non ho capito (formulaico)
Perfetto. Contiamo le monete.	Proporre di fare qualcosa insieme	Andiamo!		
-> [missione fisica: contare monete]				
Grazie per l'aiuto. Sei un amico.	Esprimere gratitudine / Asserire	Grazie (mille)		
-> Prego!			Rispondere a ringraziamento	Prego
-> Amico?			Informarsi / Asserire interrogativo	Proposizione interrogativa

03_Bar, interazione con la barista

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Buongiorno! Desidera?	Salutare (formale) / Informarsi	Buongiorno / Proposizione interrogativa		
-> Un caffè, per favore.			Chiedere in contesto servizio	Gruppo nominale, per favore
-> Tutto bene, per favore.				
-> Va bene, per favore.				
Ecco il caffè! Vuole altro?	Offrire qualcosa	Vuole + gruppo nominale?		
-> No, grazie.			Rispondere rifiutando	No, grazie
-> No, prego.				
Non ho capito. Vuole ordinare qualcosa?[89]	Chiedere chiarimenti / Informarsi	Non ho capito (formulaico)		
-> Sì. Un cornetto.			Chiedere in contesto servizio	Gruppo nominale
-> Voglio una sedia.				
Grazie a lei. Spende 1 euro e 50 centesimi.	Esprimere gratitudine / Rispondere sulla quantità	Grazie (mille) / cifra numerica		
-> Ecco a lei.				
-> Non c'è bisogno.				
Perfetto. Una buona giornata, arrivederci.	Augurare / Prendere congedo	Buon... / Arrivederci		
-> Arrivederci!			Prendere congedo	Arrivederci
-> Scusi.				

Battuta NPC / Risposta player	**Funzione PLI**	**Esponente PLI**	**Funzione PLI (risposta)**	**Esponente PLI (risposta)**
Non ho capito. Vuole qualcosa?	Chiedere chiarimenti / Informarsi	Non ho capito (formulaico)		
-> No. Arrivederci!			Rispondere rifiutando / Prendere congedo	No / Arrivederci
-> Scusi.				

[89]Nodo di chiarimento raggiunto dopo la scelta di un distrattore. Se il distrattore è stato scelto in risposta a *Ecco il caffè! Vuole altro?*, la battuta è *Non ho capito. Vuole ordinare altro?*.

APPENDICE B

Gesti di interazione

Gesto	**Descrizione**	**Funzione**
Palmi in su	Uno o entrambi i palmi rivolti verso l'alto	Movimento in avanti nella direzione dello sguardo
Palmi in giù	Entrambi i palmi rivolti verso il basso	Stop al movimento
Gesto di presa	Chiusura naturale della mano attorno a un oggetto	Afferrare oggetti grabbabili
Pinch	Unione di pollice e indice	Interagire con pannelli, selezionare opzioni di risposta
Indice puntato verso l'alto	Indice esteso verso l'alto, altre dita raccolte	Attivare il microfono per la produzione vocale
Mano sinistra sul polso destro	Sovrapposizione della mano sinistra sul polso destro	Aprire e chiudere l'inventario

Tabella 14: Gesti di interazione implementati in *ItalianVRExperience* tramite *hand tracking* su Meta Quest 2.

APPENDICE C

Script significativi

Questa appendice riporta per intero gli script C# da cui sono tratti gli *snippet* del capitolo 4, nell'ordine in cui compaiono nel testo. I file corrispondono alla versione del prototipo impiegata nelle sessioni sperimentali. Per leggibilità sono state rimosse le parti non eseguite durante le sessioni, come un gesto alternativo di attivazione del microfono non adottato, discusso al §6.3, e la scorciatoia da tastiera per il test nell'editor di Unity senza visore,[90] insieme ai messaggi di debug e ai caratteri grafici non stampabili; la logica eseguita è riportata senza modifiche.

C.1 GestureSpeechInput.cs

Rilevamento dei gesti `PuntaSu` e dei palmi rivolti verso il basso, attesa di un secondo prima dell'apertura del microfono, gestione dell'audio e della locomozione durante la registrazione (§4.3, Listati 1 e 2).

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.Hands;
using UnityEngine.SceneManagement;
using Bellavalle.UI;
using Bellavalle.Voice;

namespace Bellavalle.Voice
{
    public class GestureSpeechInput : MonoBehaviour
    {
        [Header("Riferimenti fissi (persistenti)")]
        [SerializeField] PushToTalkRecorder pushToTalk;

        FistDynamicMoveProvider moveProvider;
        InventoryUI inventoryUI;

```

[90] Durante lo sviluppo, la barra spaziatrice sostituiva il gesto di attivazione del microfono, così da poterne verificare il funzionamento senza indossare il visore.

```
19          [Header("Rilevamento - PuntaSu")]
20          [Range(0f, 1f)]
21          [SerializeField] float pointUpThreshold = 0.7f;
22          [SerializeField] float curledDistance = 0.06f;
23
24          [Header("Rilevamento 'palmi in giu'")]
25          [Range(0f, 1f)]
26          [SerializeField] float palmDownThreshold = 0.7f;
27
28          [Header("Delay avvio microfono")]
29          [Tooltip("Secondi di attesa tra il gesto e l'avvio effettivo della
            ↪  registrazione")]
30          [SerializeField] float micStartDelay = 1f;
31
32          XRHandSubsystem _handSubsystem;
33          bool _listening;
34          Coroutine _startDelayCoroutine;
35
36          [Header("Durata minima registrazione")]
37          [Tooltip("Secondi minimi di registrazione prima che il gesto di stop venga
            ↪  accettato")]
38          [SerializeField] float minRecordingTime = 2f;
39
40          float _recordingStartTime;
41
42          List<AudioSource> _ambientMusicSources = new();
43          Dictionary<AudioSource, float> _ambientMusicVolumes = new();
44          HashSet<AudioSource> _playingAudioSources = new();
45
46          void Awake()
47          {
48              var subsystems = new List<XRHandSubsystem>();
49              SubsystemManager.GetSubsystems(subsystems);
50              if (subsystems.Count > 0)
51                  _handSubsystem = subsystems[0];
52          }
53
54          void OnEnable()
55          {
56              SceneManager.sceneLoaded += OnSceneLoaded;
57              if (pushToTalk != null) pushToTalk.OnRecordingStopped +=
                ↪  HandleRecordingStopped;
58              RefreshSceneReferences();
59          }
60
61          void OnDisable()
62          {
63              SceneManager.sceneLoaded -= OnSceneLoaded;
64              if (pushToTalk != null) pushToTalk.OnRecordingStopped -=
                ↪  HandleRecordingStopped;
65          }
66
```

```
67          void OnSceneLoaded(UnityEngine.SceneManagement.Scene scene, LoadSceneMode
            ↪  mode)
68          {
69              // questo componente e' persistente (DontDestroyOnLoad) e sopravvive al
                ↪  cambio
70              // scena. Se una registrazione era in corso, va fermata esplicitamente
                ↪  qui:
71              // altrimenti _listening resterebbe bloccato a true e ogni futuro
                ↪  tentativo
72              // verrebbe ignorato, perche' il microfono risulterebbe ancora occupato.
73              if (_listening)
74                  StopListening();
75
76              if (_startDelayCoroutine != null)
77              {
78                  StopCoroutine(_startDelayCoroutine);
79                  _startDelayCoroutine = null;
80              }
81
82              RefreshSceneReferences();
83          }
84
85          void RefreshSceneReferences()
86          {
87              moveProvider = FindFirstObjectByType<FistDynamicMoveProvider>();
88              inventoryUI = FindFirstObjectByType<InventoryUI>();
89
90              _ambientMusicSources.Clear();
91              _ambientMusicVolumes.Clear();
92              var ambientObjects = GameObject.FindGameObjectsWithTag("AmbientMusic");
93              foreach (var obj in ambientObjects)
94              {
95                  var audioSource = obj.GetComponent<AudioSource>();
96                  if (audioSource != null)
97                  {
98                      _ambientMusicSources.Add(audioSource);
99                      _ambientMusicVolumes[audioSource] = audioSource.volume;
100                 }
101             }
102         }
103
104         void Update()
105         {
106             if (_handSubsystem == null) return;
107
108             if (!_listening)
109             {
110                 if (IsTargetGesture(_handSubsystem.leftHand) ||
                    ↪  IsTargetGesture(_handSubsystem.rightHand))
111                 {
112                     if (_startDelayCoroutine == null)
```

```
                        _startDelayCoroutine =
                        ↪ StartCoroutine(StartListeningDelayed());
                }
                else
                {
                    // gesto rilasciato durante l'attesa: il microfono non si apre
                    if (_startDelayCoroutine != null)
                    {
                        StopCoroutine(_startDelayCoroutine);
                        _startDelayCoroutine = null;
                    }
                }
            }
            else
            {
                if (IsPalmDown(_handSubsystem.leftHand) &&
                ↪ IsPalmDown(_handSubsystem.rightHand))
                {
                    if (Time.time - _recordingStartTime >= minRecordingTime)
                        StopListening();
                }
            }
        }

        bool IsTargetGesture(XRHand hand) => IsPointingUp(hand);

        IEnumerator StartListeningDelayed()
        {
            yield return new WaitForSeconds(micStartDelay);

            _startDelayCoroutine = null;
            StartListening();
            _recordingStartTime = Time.time;
        }

        void StartListening()
        {
            if (pushToTalk == null) return;

            _listening = true;
            if (moveProvider != null) moveProvider.enabled = false;

            // prima mette in pausa tutte le sorgenti audio in riproduzione
            _playingAudioSources.Clear();
            var allAudioSources = FindObjectsOfType<AudioSource>();
            foreach (var audioSource in allAudioSources)
            {
                if (audioSource != null && audioSource.isPlaying)
                {
                    _playingAudioSources.Add(audioSource);
                    audioSource.Pause();
                }
```

```
163             }
164
165             // poi avvia il microfono
166             pushToTalk.StartRecording();
167             ListenAndRepeatActivator.ActiveInstance?.NotifyRecordingStarted();
168
169             _recordingStartTime = Time.time;
170         }
171
172         // chiamato solo dal gesto manuale "palmi in giu'": ferma attivamente la
173         // registrazione. Il ripristino dello stato (audio, movimento, _listening)
174         // avviene sempre in HandleRecordingStopped, che scatta da
175         // pushToTalk.OnRecordingStopped: cosi' funziona allo stesso modo anche
            ↪  quando
176         // e' il rilevamento del silenzio a fermare da solo il microfono
177         void StopListening()
178         {
179             pushToTalk.StopRecording();
180         }
181
182         void HandleRecordingStopped()
183         {
184             if (!_listening) return; // stato gia' ripristinato
185             _listening = false;
186
187             foreach (var audioSource in _playingAudioSources)
188             {
189                 if (audioSource != null)
190                     audioSource.Play();
191             }
192             _playingAudioSources.Clear();
193
194             bool inventoryStillOpen = inventoryUI != null && inventoryUI.IsOpen;
195             if (moveProvider != null && !inventoryStillOpen) moveProvider.enabled =
                ↪  true;
196         }
197
198         bool IsPointingUp(XRHand hand)
199         {
200             if (!hand.isTracked) return false;
201             try
202             {
203                 var indexTip = hand.GetJoint(XRHandJointID.IndexTip);
204                 var indexProximal = hand.GetJoint(XRHandJointID.IndexProximal);
205                 if (!indexTip.TryGetPose(out Pose tipPose)) return false;
206                 if (!indexProximal.TryGetPose(out Pose proxPose)) return false;
207
208                 Vector3 indexDir = (tipPose.position -
                    ↪  proxPose.position).normalized;
209                 if (Vector3.Dot(indexDir, Vector3.up) <= pointUpThreshold) return
                    ↪  false;
210
```

```
                var palmJoint = hand.GetJoint(XRHandJointID.Palm);
                if (!palmJoint.TryGetPose(out Pose palmPose)) return false;

                return IsFingerCurled(hand, XRHandJointID.MiddleTip,
                ↪  palmPose.position)
                    && IsFingerCurled(hand, XRHandJointID.RingTip,
                    ↪  palmPose.position)
                    && IsFingerCurled(hand, XRHandJointID.LittleTip,
                    ↪  palmPose.position);
            }
            catch { return false; }
        }

        bool IsFingerCurled(XRHand hand, XRHandJointID tipId, Vector3 palmPosition)
        {
            var tipJoint = hand.GetJoint(tipId);
            if (!tipJoint.TryGetPose(out Pose tipPose)) return false;
            return Vector3.Distance(tipPose.position, palmPosition) <
            ↪  curledDistance;
        }

        bool IsPalmDown(XRHand hand)
        {
            if (!hand.isTracked) return false;
            try
            {
                var palmJoint = hand.GetJoint(XRHandJointID.Palm);
                if (!palmJoint.TryGetPose(out Pose palmPose)) return false;

                float dot = Vector3.Dot(palmPose.rotation * Vector3.down,
                ↪  Vector3.down);
                return dot > palmDownThreshold;
            }
            catch { return false; }
        }
    }
}
```

C.2 DialogueUI.cs

Presentazione del nodo di dialogo: battuta dell'NPC, sottotitolo inglese, generazione dei bottoni e stile delle opzioni di supporto (§4.4).

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using Bellavalle.Core;
using Bellavalle.Data;

```

```
10  namespace Bellavalle.Scene
11  {
12      /// canvas di dialogo. Fa da supporto visivo:
13      /// fornisce la battuta dell'NPC + traduzione
14      /// da le opzioni cliccabili per i nodi a bottoni (comportamento legacy)
15      /// da le opzioni non cliccabili corrispondenti ai nodi useVoiceInput: le stesse
        ↪  frasi ma solo come testo
16      /// il player le legge e le dice ad alta voce, non le clicca più
17      /// stampa a schermo l'indicatore "speak now" (sostituito dal bip)
18      /// per i nodi isNameCapture: da solo la battuta e il prompt "say your name"
19      /// per i nodi senza opzioni di risposta: solo la battuta, nessun input
20      public class DialogueUI : MonoBehaviour
21      {
22          [Header("Canvas")]
23          [SerializeField] Canvas dialogueCanvas;
24          [SerializeField] Transform optionsContainer;
25          [SerializeField] TMP_Text npcLineText;
26          [SerializeField] TMP_Text translationText;
27          [SerializeField] GameObject optionButtonPrefab;
28
29          [Header("Input vocale - supporto visivo")]
30          [Tooltip("Riga sotto la battuta, tipo: 'Parla tu!' oppure il prompt del nodo
            ↪  (voicePromptIT).")]
31          [SerializeField] TMP_Text voicePromptText;
32          [Tooltip("Icona/testo che indica che il microfono sta ascoltando (es. un
            ↪  puntino rosso o '').")]
33          [SerializeField] GameObject listeningIndicator;
34          [Tooltip("Messaggio breve mostrato se la voce non combacia con nessuna frase
            ↪  suggerita.")]
35          [SerializeField] TMP_Text voiceRetryHintText;
36          [SerializeField] string defaultVoicePrompt = "Parla tu!";
37          [SerializeField] string retryHintMessage = "Non ho capito, riprova a dire
            ↪  una delle frasi.";
38          [SerializeField] string retryHintMessageStrong = "Prova a dire una di queste
            ↪  frasi, ad alta voce.";
39
40          // -- Stato ------------------------------------------------------
41          readonly List<GameObject> _spawnedButtons = new();
42          Action<int> _onOptionChosen;
43          Action _onHelpChosen;
44          Coroutine _retryHintCoroutine;
45
46          // -- Unity lifecycle --------------------------------------------
47          void Start()
48          {
49              // nasconde il contenuto ma tiene il canvas attivo
50              SetContentVisible(false);
51              SetVoiceUIVisible(false);
52          }
53
54          // -- API pubblica - nodi a bottoni ----------------------
```

```
55          public void ShowOptions(DialogueNode node, Action<int> onOption, Action
            ↪  onHelp)
56          {
57              _onOptionChosen = onOption;
58              _onHelpChosen = onHelp;
59
60              ClearButtons();
61              SetVoiceUIVisible(false);
62              ShowNpcLine(node);
63
64              for (int i = 0; i < node.options.Length; i++)
65                  SpawnOptionEntry(node.options[i], i, interactive: true);
66
67              RebuildLayout();
68              SetContentVisible(true);
69          }
70
71          // -- API pubblica - nodi useVoiceInput (risposta a voce) ---------
72          // le frasi sono mostrate come testo da leggere: sono un
73          // promemoria visivo, la risposta arriva sempre dal microfono
74          public void ShowVoiceOptions(DialogueNode node)
75          {
76              _onOptionChosen = null;
77              _onHelpChosen = null;
78
79              ClearButtons();
80              ShowNpcLine(node);
81
82              for (int i = 0; i < node.options.Length; i++)
83                  SpawnOptionEntry(node.options[i], i, interactive: false);
84
85              RebuildLayout();
86
87              string prompt = string.IsNullOrEmpty(node.voicePromptIT) ?
                ↪  defaultVoicePrompt : node.voicePromptIT;
88              SetVoicePrompt(prompt);
89              SetVoiceUIVisible(true);
90              HideRetryHint();
91
92              SetContentVisible(true);
93          }
94
95          // -- API pubblica - nodo isNameCapture (campo libero, gestito da
96          //    un handler esterno tipo NameInputHandler) -------------------
97          public void ShowNamePrompt(DialogueNode node)
98          {
99              _onOptionChosen = null;
100             _onHelpChosen = null;
101
102             ClearButtons();
103             ShowNpcLine(node);
104             RebuildLayout();
```

```
105
106             string prompt = string.IsNullOrEmpty(node.voicePromptIT)
107                 ? "Hold your finger up and speak."
108                 : node.voicePromptIT;
109             SetVoicePrompt(prompt);
110             SetVoiceUIVisible(true);
111             HideRetryHint();
112
113             SetContentVisible(true);
114         }
115
116         /// aggiorna solo il prompt vocale (usato da NameInputHandler per i messaggi
            ↪ di stato)
117         public void SetVoicePrompt(string message)
118         {
119             if (voicePromptText != null) voicePromptText.text = message;
120         }
121
122         // -- API pubblica - nodo lineare (solo battuta, nessun input) ----
123         public void ShowLinearLine(DialogueNode node)
124         {
125             _onOptionChosen = null;
126             _onHelpChosen = null;
127
128             ClearButtons();
129             SetVoiceUIVisible(false);
130             ShowNpcLine(node);
131             SetContentVisible(true);
132         }
133
134         // -- feedback "non ho capito" durante l'ascolto vocale -----------
135         public void ShowVoiceRetryHint(bool strong)
136         {
137             if (voiceRetryHintText == null) return;
138
139             voiceRetryHintText.text = strong ? retryHintMessageStrong :
                ↪ retryHintMessage;
140             voiceRetryHintText.gameObject.SetActive(true);
141
142             if (_retryHintCoroutine != null) StopCoroutine(_retryHintCoroutine);
143             _retryHintCoroutine = StartCoroutine(HideRetryHintAfter(2.5f));
144         }
145
146         void HideRetryHint()
147         {
148             if (_retryHintCoroutine != null) StopCoroutine(_retryHintCoroutine);
149             if (voiceRetryHintText != null)
                ↪ voiceRetryHintText.gameObject.SetActive(false);
150         }
151
152         IEnumerator HideRetryHintAfter(float seconds)
153         {
```

```
154             yield return new WaitForSeconds(seconds);
155             if (voiceRetryHintText != null)
                ↪  voiceRetryHintText.gameObject.SetActive(false);
156         }
157
158         // -- nascondi tutto a dialogo avvenuto
            ↪  ----------------------------------------------
159         public void HideOptions()
160         {
161             ClearButtons();
162             SetVoiceUIVisible(false);
163             SetContentVisible(false);
164         }
165
166         // -- battuta NPC + traduzione inglese ----------------------------
167         void ShowNpcLine(DialogueNode node)
168         {
169             if (npcLineText != null)
170             {
171                 npcLineText.text = ReplacePlaceholders(node.npcLine_IT);
172                 npcLineText.gameObject.SetActive(true);
173             }
174
175             if (translationText != null)
176             {
177                 bool showTranslation = ShouldShowTranslation() &&
                    ↪  !string.IsNullOrEmpty(node.npcLine_EN);
178                 translationText.gameObject.SetActive(showTranslation);
179                 if (showTranslation)
180                     translationText.text =
                        ↪  $"({ReplacePlaceholders(node.npcLine_EN)})";
181             }
182         }
183
184         // -- Righe/bottoni opzione ---------------------------------------
185         // interactive = true  -> bottone cliccabile
186         // interactive = false -> solo testo, promemoria di cosa dire a voce
187         void SpawnOptionEntry(PlayerOption option, int index, bool interactive)
188         {
189             var go = Instantiate(optionButtonPrefab, optionsContainer);
190             _spawnedButtons.Add(go);
191
192             var label = go.GetComponentInChildren<TMP_Text>();
193             if (label != null)
194             {
195                 // bottoni cliccabili: identico all'originale, solo text_IT
196                 // suggerimenti vocali (interactive = false): aggiunge la traduzione
197                 // tra parentesi quando i sottotitoli sono attivi, per aiutare
198                 // chi non sa ancora cosa dire
199                 label.text = (!interactive && ShouldShowTranslation() &&
                    ↪  !string.IsNullOrEmpty(option.text_EN))
200                     ? $"{option.text_IT}  ({option.text_EN})"
```

```
                    : option.text_IT;
            }

            if (option.isHelpOption)
                StyleAsHelpButton(go);

            var btn = go.GetComponent<Button>();
            if (btn == null) btn = go.AddComponent<Button>();

            if (interactive)
            {
                btn.interactable = true;
                int capturedIndex = index;
                bool capturedHelp = option.isHelpOption;
                btn.onClick.RemoveAllListeners();
                btn.onClick.AddListener(() =>
                {
                    if (capturedHelp) _onHelpChosen?.Invoke();
                    else _onOptionChosen?.Invoke(capturedIndex);
                });
            }
            else
            {
                // solo testo: nessuna interazione, stile leggermente attenuato
                // per far capire che non è un bottone da premere.
                btn.onClick.RemoveAllListeners();
                btn.interactable = false;
                StyleAsVoiceHint(go);
            }
        }

        void StyleAsHelpButton(GameObject go)
        {
            var label = go.GetComponentInChildren<TMP_Text>();
            if (label != null)
            {
                label.color = new Color(0.5f, 0.5f, 0.5f);
                label.fontSize *= 0.85f;
            }
        }

        void StyleAsVoiceHint(GameObject go)
        {
            var image = go.GetComponent<Image>();
            if (image != null)
                image.color = new Color(image.color.r, image.color.g, image.color.b,
                ↪  0.35f);

            var label = go.GetComponentInChildren<TMP_Text>();
            if (label != null)
                label.fontStyle |= FontStyles.Italic;
        }
```

```
252
253         void ClearButtons()
254         {
255             foreach (var b in _spawnedButtons)
256                 if (b != null) Destroy(b);
257             _spawnedButtons.Clear();
258         }
259
260         void RebuildLayout()
261         {
262             if (optionsContainer is RectTransform rt)
263                 LayoutRebuilder.ForceRebuildLayoutImmediate(rt);
264         }
265
266         // -- visibilità ------------------------------------------------
267         void SetContentVisible(bool visible)
268         {
269             if (npcLineText) npcLineText.gameObject.SetActive(visible);
270             if (optionsContainer != null)
                ↪  optionsContainer.gameObject.SetActive(visible);
271
272             // translationText, quando si mostra il contenuto, resta gestita da
273             // ShowNpcLine (che sa, nodo per nodo, se c'è davvero una traduzione da
                ↪  mostrare)
274             if (!visible && translationText)
                ↪  translationText.gameObject.SetActive(false);
275         }
276
277         void SetVoiceUIVisible(bool visible)
278         {
279             if (voicePromptText != null)
                ↪  voicePromptText.gameObject.SetActive(visible);
280             if (listeningIndicator != null) listeningIndicator.SetActive(visible);
281             if (!visible) HideRetryHint();
282         }
283
284         string ReplacePlaceholders(string text)
285         {
286             if (string.IsNullOrEmpty(text)) return text;
287             if (GameManager.Instance == null) return text;
288             return text.Replace("{playerName}",
                ↪  GameManager.Instance.State.playerName);
289         }
290
291         // -- subtitling ------------------------------------------
292         bool ShouldShowTranslation()
293         {
294             if (GameManager.Instance == null) return true;
295             var state = GameManager.Instance.State;
296             return state.confidenceScore < 0.5f || state.subtitlesEN;
297         }
298     }
```

```
}
```

C.3 NameInputHandler.cs

Cattura vocale del nome del giocatore e pulizia della trascrizione tramite `CleanTranscribedName` (§4.4).

```
using UnityEngine;
using TMPro;
using Bellavalle.Core;
using Bellavalle.Scene;
using Bellavalle.Voice;

namespace Bellavalle.Missions
{
    /// gestisce il momento in cui un NPC chiede il nome al player
    /// invece di mostrare bottoni A/B/C, il DialogueManager mostra un prompt
    /// "speak now" (tramite DialogueUI.ShowNamePrompt) e questo script ascolta
    /// il push-to-talk; il nome trascritto viene salvato in GameState.playerName
    /// e il dialogo avanza al nodo successivo
    /// è data-driven: si attiva se un nodo ha isNameCapture = true nel NPCData,
    /// e il nodo successivo è node.linearNextNodeId (lo stesso campo già usato per
    /// i nodi isVocabIntro); funziona quindi per la vicina ma anche per qualunque
    /// altro NPC che possa chiedere il nome in futuro
    public class NameInputHandler : MonoBehaviour
    {
        [Header("riferimenti")]
        [SerializeField] PushToTalkRecorder pushToTalkRecorder;
        [SerializeField] DialogueManager dialogueManager;

        [Header("passa dal canvas di dialogo")]
        [Tooltip("se assegnato, i messaggi di stato (speak now / listening / name
        ↪  inserted) " +
                 "vengono mostrati tramite DialogueUI.SetVoicePrompt, cioè nello
                 ↪  stesso " +
                 "canvas del dialogo per tenere un solo sistema visivo attivo")]
        [SerializeField] DialogueUI dialogueUI;

        [Header("pannello separato (usato solo se dialogueUI è vuoto)")]
        [SerializeField] GameObject speakPromptPanel;
        [SerializeField] TMP_Text speakPromptText;

        [Header("messaggi")]
        [SerializeField] string promptMessage = "Hold your finger up and say your
        ↪  name";
        [SerializeField] string retryMessage = "I didn't get it. Retry";
        [SerializeField] string confirmMessage = "Piacere di conoscerti, {name}!";

        [Header("fallback legacy (usati solo se il nodo non ha
        ↪  isNameCapture/linearNextNodeId)")]
        [SerializeField] string legacyListenNodeId = "";
        [SerializeField] string legacyNextNodeId = "";
```

```
42
43          bool _isListening;
44          bool _nameReceived;
45          string _pendingNextNodeId;
46
47          void OnEnable()
48          {
49              EventBus.On(GameEvent.NodeEntered, OnNodeEntered);
50
51              // WhisperManager persistente
52              if (PersistentWhisper.Instance != null)
53                  pushToTalkRecorder = PersistentWhisper.Instance.PushToTalkRecorder;
54
55              if (pushToTalkRecorder != null)
56                  pushToTalkRecorder.OnTranscriptionReady += OnNameTranscribed;
57          }
58
59          void OnDisable()
60          {
61              EventBus.Off(GameEvent.NodeEntered, OnNodeEntered);
62
63              if (pushToTalkRecorder != null)
64                  pushToTalkRecorder.OnTranscriptionReady -= OnNameTranscribed;
65          }
66
67          void Start()
68          {
69              if (speakPromptPanel != null)
70                  speakPromptPanel.SetActive(false);
71          }
72
73          // -- quando il dialogo raggiunge un nodo "chiedi nome" ----------
74          void OnNodeEntered(object data)
75          {
76              var node = data as Bellavalle.Data.DialogueNode;
77              if (node == null) return;
78
79              bool isTarget = node.isNameCapture ||
                ↪  (!string.IsNullOrEmpty(legacyListenNodeId) && node.nodeId ==
                ↪  legacyListenNodeId);
80              if (!isTarget) return;
81              if (_nameReceived) return;
82
83              _pendingNextNodeId = !string.IsNullOrEmpty(node.linearNextNodeId)
84                  ? node.linearNextNodeId
85                  : legacyNextNodeId;
86
87              _isListening = true;
88              ShowPrompt(promptMessage);
89          }
90
91          // -- quando Whisper trascrive il nome ----------------------------
```

```
92          void OnNameTranscribed(string transcribedText)
93          {
94              if (!_isListening) return;
95              _isListening = false;
96              _nameReceived = true;
97
98              string cleanName = CleanTranscribedName(transcribedText);
99
100             if (string.IsNullOrWhiteSpace(cleanName))
101             {
102                 // nome non valido: si chiede di ripetere
103                 ShowPrompt(retryMessage);
104                 _isListening = true;
105                 _nameReceived = false;
106                 return;
107             }
108
109             GameManager.Instance.State.playerName = cleanName;
110             GameManager.Instance.SaveGame();
111
112             ShowPrompt(confirmMessage.Replace("{name}", cleanName));
113
114             StartCoroutine(AdvanceAfterDelay(2f));
115         }
116
117         System.Collections.IEnumerator AdvanceAfterDelay(float delay)
118         {
119             yield return new WaitForSeconds(delay);
120             HidePrompt();
121
122             // senza un nodo successivo il dialogo non avanza
123             if (string.IsNullOrEmpty(_pendingNextNodeId)) yield break;
124
125             dialogueManager.GoToNodePublic(_pendingNextNodeId);
126         }
127
128         // -- UI prompt ------------------------------------------------
129         void ShowPrompt(string message)
130         {
131             if (dialogueUI != null)
132             {
133                 dialogueUI.SetVoicePrompt(message);
134                 return;
135             }
136
137             if (speakPromptText != null) speakPromptText.text = message;
138             if (speakPromptPanel != null) speakPromptPanel.SetActive(true);
139         }
140
141         void HidePrompt()
142         {
```

```
            if (dialogueUI != null) return; // il DialogueManager nasconde tutto al
            ↪  cambio nodo
            if (speakPromptPanel != null) speakPromptPanel.SetActive(false);
        }

        // -- pulizia testo trascritto ------------------------------------
        string CleanTranscribedName(string raw)
        {
            if (string.IsNullOrWhiteSpace(raw)) return null;

            string clean = raw.Trim()
                .TrimEnd('.', ',', '!', '?', ';', ':')
                .Trim();

            string lower = clean.ToLower();
            string[] prefixes = {
                "mi chiamo ", "sono ", "il mio nome è ", "il mio nome e ",
                "my name is ", "i'm ", "i am "
            };

            foreach (var prefix in prefixes)
            {
                if (lower.StartsWith(prefix))
                {
                    clean = clean.Substring(prefix.Length).Trim();
                    break;
                }
            }

            if (clean.Length > 0)
                clean = char.ToUpper(clean[0]) + clean.Substring(1);

            // nomi troppo lunghi o marcatori di Whisper come [BLANK_AUDIO] vengono
            ↪  scartati
            if (clean.Length > 30 || clean.StartsWith("["))
                return null;

            return clean;
        }
    }
}
```

C.4 PushToTalkRecorder.cs

Registrazione dell'audio e invio a Whisper (§4.4.1, Listato 3).

```
using UnityEngine;
using Whisper;
using Whisper.Utils;

namespace Bellavalle.Voice
{
```

```
7       public class PushToTalkRecorder : MonoBehaviour
8       {
9           [Header("Riferimenti")]
10          [SerializeField] WhisperManager whisperManager;
11          [SerializeField] MicrophoneRecord microphoneRecord;
12
13          public bool IsRecording => microphoneRecord != null &&
            ↪  microphoneRecord.IsRecording;
14
15          /// per il testo trascritto: usato da DialogueManager, NameInputHandler, ecc
16          public event System.Action<string> OnTranscriptionReady;
17
18          /// risultato completo di Whisper: usato da VoiceRecognitionLogger e da
            ↪  ListenAndRepeatActivator
19          public event System.Action<WhisperResult> OnTranscriptionResultReady;
20
21          /// scatta appena la registrazione finisce (gesto manuale di stop oppure
22          /// arresto automatico per silenzio), prima che Whisper trascriva. a
            ↪  differenza
23          /// degli eventi sopra, scatta sempre, anche se l'audio risulta troppo corto
            ↪  o senza testo riconosciuto
24          /// usato da GestureSpeechInput per sincronizzare il proprio stato quando e'
25          /// il rilevamento del silenzio a fermare il microfono
26          public event System.Action OnRecordingStopped;
27
28          void Awake()
29          {
30              if (microphoneRecord == null) return;
31              microphoneRecord.OnRecordStop += OnRecordStopHandler;
32          }
33
34          void OnDestroy()
35          {
36              if (microphoneRecord != null)
37                  microphoneRecord.OnRecordStop -= OnRecordStopHandler;
38          }
39
40          public void StartRecording()
41          {
42              if (microphoneRecord == null) return;
43              if (microphoneRecord.IsRecording) return;
44
45              // il gesto viene ignorato finche' il modello non e' stato caricato
46              if (whisperManager == null || !whisperManager.IsLoaded) return;
47
48              microphoneRecord.StartRecord();
49          }
50
51          public void StopRecording()
52          {
53              if (microphoneRecord == null) return;
54              if (!microphoneRecord.IsRecording) return;
```

```
55
56              microphoneRecord.StopRecord();
57          }
58
59          async void OnRecordStopHandler(AudioChunk recordedAudio)
60          {
61              OnRecordingStopped?.Invoke();
62
63              // registrazioni troppo brevi: scartate
64              if (recordedAudio.Length < 0.3f) return;
65
66              var result = await whisperManager.GetTextAsync(
67                  recordedAudio.Data, recordedAudio.Frequency,
                    ↪  recordedAudio.Channels);
68
69              if (result == null || string.IsNullOrWhiteSpace(result.Result)) return;
70
71              string text = result.Result.Trim();
72
73              OnTranscriptionResultReady?.Invoke(result);
74              OnTranscriptionReady?.Invoke(text);
75          }
76      }
77  }
```

C.5 VoiceMatchUtils.cs

Normalizzazione del testo e confronto tra trascrizione e *trigger* (§4.4.1, Listati 4, 5 e 6).

```
1   using System.Text;
2
3   namespace Bellavalle.Voice
4   {
5       /// normalizzazione e matching del testo trascritto da Whisper contro le
6       /// parole chiave (PlayerOption.voiceTriggers) definite nei nodi di dialogo
7       /// con useVoiceInput = true
8       /// formattazione semplice: minuscolo, accenti tolti, punteggiatura rimossa,
9       /// poi un confronto tra il testo normalizzato e singolo trigger normalizzato
10      public static class VoiceMatchUtils
11      {
12          public static string Normalize(string s)
13          {
14              if (string.IsNullOrWhiteSpace(s)) return string.Empty;
15
16              string lower = s.ToLowerInvariant();
17              lower = lower
18                  .Replace('à', 'a').Replace('á', 'a')
19                  .Replace('è', 'e').Replace('é', 'e')
20                  .Replace('ì', 'i').Replace('í', 'i')
21                  .Replace('ò', 'o').Replace('ó', 'o')
22                  .Replace('ù', 'u').Replace('ú', 'u');
23
```

```
24              var sb = new StringBuilder(lower.Length);
25              foreach (char c in lower)
26              {
27                  if (char.IsLetterOrDigit(c) || char.IsWhiteSpace(c))
28                      sb.Append(c);
29                  else
30                      sb.Append(' '); // la punteggiatura diventa spazio per non
                        ↪  fondere due parole
31              }
32
33              // comprimi spazi multipli
34              var collapsed = new StringBuilder(sb.Length);
35              bool lastWasSpace = false;
36              foreach (char c in sb.ToString().Trim())
37              {
38                  bool isSpace = char.IsWhiteSpace(c);
39                  if (isSpace && lastWasSpace) continue;
40                  collapsed.Append(c);
41                  lastWasSpace = isSpace;
42              }
43
44              return collapsed.ToString();
45          }
46
47          /// true se parola chiave 'trigger' compare come sotto-sequenza di parole
            ↪  trascritte
48          public static bool Matches(string transcribedRaw, string triggerRaw)
49          {
50              if (string.IsNullOrWhiteSpace(transcribedRaw) ||
                ↪  string.IsNullOrWhiteSpace(triggerRaw))
51                  return false;
52
53              string transcribed = Normalize(transcribedRaw);
54              string trigger = Normalize(triggerRaw);
55
56              if (transcribed.Length == 0 || trigger.Length == 0) return false;
57
58              // confronto su stringhe con spazi ai bordi evita match parziali dentro
                ↪  parole diverse
59              return (" " + transcribed + " ").Contains(" " + trigger + " ")
60                     || transcribed == trigger;
61          }
62
63          /// cerca tra le opzioni del nodo quella il cui voiceTriggers combacia per
            ↪  prima
64          /// con il testo trascritto. ritorna -1 se nessuna combacia
65          public static int FindMatchingOption(string transcribed,
            ↪  Bellavalle.Data.PlayerOption[] options)
66          {
67              if (options == null) return -1;
68
69              for (int i = 0; i < options.Length; i++)
```

```
            {
                var triggers = options[i].voiceTriggers;
                if (triggers == null) continue;

                foreach (var trig in triggers)
                    if (Matches(transcribed, trig))
                        return i;
            }
            return -1;
        }
    }
}
```

C.6 ListenAndRepeatActivator.cs

Logica vocale del *listen-and-repeat* e *feedback* visivo sugli oggetti etichettati (§4.4.2, Listato 7).

```
using System.Collections;
using TMPro;
using UnityEngine;
using UnityEngine.XR.Interaction.Toolkit;
using Bellavalle.Scene;
using Bellavalle.Voice;
using Whisper;

namespace Bellavalle.UI
{
    /// flusso:
    ///  1. hover sull'oggetto -> canvas si apre ma il testo del nome e' nascosto
    ///  2. player preme il bottone audio quante volte vuole
    ///  3. player fa PuntaSu -> microfono attivo -> ripete -> abbassa le mani
    ///  4. Whisper trascrive -> confronto con targetWord
    ///  5. match -> il testo appare + feedback visivo verde
    ///     no match -> feedback visivo rosso + puo' riprovare
    ///  6. ogni tentativo viene loggato nel CSV tramite VoiceRecognitionLogger
    public class ListenAndRepeatActivator : MonoBehaviour
    {
        [Header("Parola target")]
        [Tooltip("Parola da pronunciare, in minuscolo (es. 'tazza')")]
        [SerializeField] string targetWord;

        [Header("Riferimenti canvas")]
        [Tooltip("Il TMP_Text che mostra il nome - viene nascosto all'inizio e
        ↪  rivelato al match")]
        [SerializeField] TMP_Text wordText;

        [Header("Feedback visivo")]
        [Tooltip("Colore del testo quando il match e' corretto")]
        [SerializeField] Color successColor = Color.green;
        [Tooltip("Colore del testo quando il match e' sbagliato")]
        [SerializeField] Color failColor = Color.red;
```

```
34          [Tooltip("Colore normale del testo")]
35          [SerializeField] Color normalColor = Color.white;
36          [Tooltip("Secondi in cui resta visibile il feedback prima di tornare
            ↪  normale")]
37          [SerializeField] float feedbackDuration = 1.5f;
38
39          [Header("Riferimenti persistenti")]
40          [SerializeField] PushToTalkRecorder pushToTalkRecorder;
41          [SerializeField] DialogueManager dialogueManager;
42          [SerializeField] VoiceRecognitionLogger logger;
43
44          // stato interno
45          bool _isOpen;
46          bool _waitingForTranscription;
47          int _attemptNumber;
48          Coroutine _feedbackCoroutine;
49
50          // -- Singleton leggero per "uno alla volta" ----------------------
51          static ListenAndRepeatActivator _activeInstance;
52
53          // espone l'istanza attualmente aperta (hover in corso) a chi deve
54          // notificarle l'inizio della registrazione (GestureSpeechInput):
55          // senza questo, verrebbe presa un'istanza arbitraria tra le tante
56          // presenti in scena, non necessariamente quella aperta.
57          public static ListenAndRepeatActivator ActiveInstance => _activeInstance;
58
59          void OnEnable()
60          {
61              // ascolta l'apertura/chiusura del canvas tramite XRInteractable
62              var interactable =
                ↪  GetComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRBaseInteractable>();
63              if (interactable != null)
64              {
65                  interactable.hoverEntered.AddListener(OnHoverEntered);
66                  interactable.hoverExited.AddListener(OnHoverExited);
67              }
68
69              // dalla seconda scena in poi il campo assegnato in Inspector puo'
                ↪  puntare
70              // alla copia locale (gia' distrutta) del prefab WhisperManager: si
                ↪  risolve
71              // sempre dall'istanza sopravvissuta
72              if (PersistentWhisper.Instance != null)
73                  pushToTalkRecorder = PersistentWhisper.Instance.PushToTalkRecorder;
74
75              if (pushToTalkRecorder != null)
76                  pushToTalkRecorder.OnTranscriptionResultReady +=
                    ↪  OnTranscriptionResultReady;
77          }
78
79          void OnDisable()
80          {
```

```
81              var interactable =
                ↪  GetComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRBaseInteractable>();
82              if (interactable != null)
83              {
84                  interactable.hoverEntered.RemoveListener(OnHoverEntered);
85                  interactable.hoverExited.RemoveListener(OnHoverExited);
86              }
87
88              if (pushToTalkRecorder != null)
89                  pushToTalkRecorder.OnTranscriptionResultReady -=
                    ↪  OnTranscriptionResultReady;
90
91              if (_activeInstance == this)
92                  _activeInstance = null;
93          }
94
95          // -- Apertura canvas ------------------------------------------
96          void OnHoverEntered(HoverEnterEventArgs args)
97          {
98              // se c'e' gia' un altro oggetto aperto, lo chiude prima
99              if (_activeInstance != null && _activeInstance != this)
100                 _activeInstance.CloseCanvas();
101
102             _activeInstance = this;
103             _isOpen = true;
104             _attemptNumber = 0;
105
106             // nasconde il testo del nome: il player deve sentirlo prima di vederlo
107             if (wordText != null)
108             {
109                 wordText.gameObject.SetActive(false);
110                 wordText.color = normalColor;
111             }
112         }
113
114         void OnHoverExited(HoverExitEventArgs args)
115         {
116             CloseCanvas();
117         }
118
119         void CloseCanvas()
120         {
121             _isOpen = false;
122             _waitingForTranscription = false;
123
124             if (_activeInstance == this)
125                 _activeInstance = null;
126
127             if (_feedbackCoroutine != null)
128             {
129                 StopCoroutine(_feedbackCoroutine);
130                 _feedbackCoroutine = null;
```

```
131             }
132         }
133 
134         // -- Ricezione trascrizione ------------------------------------
135         void OnTranscriptionResultReady(WhisperResult result)
136         {
137             // ignora se il canvas non e' aperto, se c'e' un dialogo attivo
138             // o se il gesto non e' stato eseguito
139             if (!_isOpen) return;
140             if (dialogueManager != null && dialogueManager.IsActive) return;
141             if (!_waitingForTranscription) return;
142 
143             _waitingForTranscription = false;
144             _attemptNumber++;
145 
146             string transcribed = (result.Result ?? "").Trim().ToLowerInvariant();
147             bool isMatch = transcribed.Contains(targetWord.ToLowerInvariant());
148 
149             logger?.LogListenAndRepeat(targetWord, _attemptNumber, isMatch, result);
150 
151             // feedback visivo
152             if (_feedbackCoroutine != null) StopCoroutine(_feedbackCoroutine);
153             _feedbackCoroutine = StartCoroutine(ShowFeedback(isMatch));
154         }
155 
156         IEnumerator ShowFeedback(bool isMatch)
157         {
158             if (wordText == null) yield break;
159 
160             // mostra sempre il testo durante il feedback
161             wordText.gameObject.SetActive(true);
162             wordText.color = isMatch ? successColor : failColor;
163 
164             yield return new WaitForSeconds(feedbackDuration);
165 
166             if (isMatch)
167             {
168                 // match corretto: il testo resta visibile col colore normale
169                 wordText.color = normalColor;
170             }
171             else
172             {
173                 // match sbagliato: nasconde di nuovo il testo, il player puo'
                    ↪  riprovare
174                 wordText.gameObject.SetActive(false);
175                 _waitingForTranscription = false; // reset per il prossimo tentativo
176             }
177 
178             _feedbackCoroutine = null;
179         }
180 
181         // -- API pubblica ----------------------------------------------
```

```
182         // chiamata da GestureSpeechInput quando parte la registrazione:
183         // segnala che il player ha fatto il gesto e sta per parlare.
184         public void NotifyRecordingStarted()
185         {
186             if (!_isOpen) return;
187             if (dialogueManager != null && dialogueManager.IsActive) return;
188             _waitingForTranscription = true;
189         }
190     }
191 }
```

C.7 AIGrammarHelperService.cs

Prompt di estrazione, lettura della risposta di Mistral e cascata di risoluzione nei database verificati (§4.5, Listati 8 e 9).

```
1  using System;
2  using System.Collections;
3  using System.Collections.Generic;
4  using System.Text;
5  using System.Text.RegularExpressions;
6  using UnityEngine;
7  using UnityEngine.Networking;
8  using Bellavalle.Data;
9  using Bellavalle.Voice;
10 using Bellavalle.Core;
11 using Whisper;
12
13 namespace Bellavalle.AI
14 {
15     public class AIGrammarHelperService : MonoBehaviour
16     {
17         [Header("Ollama")]
18         [SerializeField] string ollamaUrl = "http://localhost:11434/api/generate";
19         [SerializeField] string model = "mistral";
20         [Tooltip("secondi di attesa massimi per una risposta di Ollama prima di
           ↪  considerarla fallita: " +
21                  "senza questo, un server bloccato o lento lascerebbe
                    ↪  IsWaitingResponse bloccato per sempre")]
22         [SerializeField] int requestTimeoutSeconds = 15;
23
24         [Header("Dati")]
25         [SerializeField] VerbConjugationDatabase verbDatabase;
26         [SerializeField] VocabularyDatabase vocabularyDatabase;
27
28         [Header("Whisper (per il toggle inglese/italiano)")]
29         [SerializeField] WhisperManager whisperManager;
30
31         [Header("Input")]
32         [Tooltip("da assegnare nell'Inspector; l'abbonamento all'evento avviene solo
           ↪  " +
33                  "quando il pannello AI è aperto (EnterEnglishMode), non sempre")]
```

```
34          [SerializeField] PushToTalkRecorder pushToTalkRecorder;
35
36          /// risposta valida pronta da mostrare dai database verificati
37          public event Action<string> OnAnswerReady;
38
39          /// domanda capita ma fuori dallo scope attuale
40          public event Action<string> OnHelpNeeded;
41
42          /// problema tecnico (Ollama non raggiungibile, risposta vuota)
43          public event Action<string> OnError;
44
45          bool _isWaitingResponse;
46          bool _isActive; // true solo quando il pannello AI è aperto
47          string _languageBeforeEnglishMode;
48
49          public bool IsWaitingResponse => _isWaitingResponse;
50
51          void Awake()
52          {
53              // se esiste ollama_config.txt, il suo contenuto sostituisce l'indirizzo
                ↪  predefinito
54              string configPath =
                ↪  System.IO.Path.Combine(Application.persistentDataPath,
                ↪  "ollama_config.txt");
55              if (System.IO.File.Exists(configPath))
56              {
57                  string overrideUrl = System.IO.File.ReadAllText(configPath).Trim();
58                  if (!string.IsNullOrEmpty(overrideUrl))
59                      ollamaUrl = overrideUrl;
60              }
61          }
62
63          // -- toggle lingua Whisper + abbonamento evento -------------------
64          public void EnterEnglishMode()
65          {
66              if (_isActive) return;
67              if (whisperManager == null) return;
68
69              _languageBeforeEnglishMode = whisperManager.language;
70              whisperManager.language = "en";
71              _isActive = true;
72
73              if (pushToTalkRecorder != null)
74                  pushToTalkRecorder.OnTranscriptionReady += AskAboutVerb;
75          }
76
77          public void ExitEnglishMode()
78          {
79              if (!_isActive) return;
80              if (whisperManager == null) return;
81
```

```
82              whisperManager.language =
                ↪  string.IsNullOrEmpty(_languageBeforeEnglishMode)
83                  ? "it"
84                  : _languageBeforeEnglishMode;
85              _isActive = false;
86
87              if (pushToTalkRecorder != null)
88                  pushToTalkRecorder.OnTranscriptionReady -= AskAboutVerb;
89          }
90
91          void OnDisable()
92          {
93              // se il GameObject viene disattivato mentre una richiesta a Ollama è in
                ↪  volo,
94              // la coroutine SendExtractionRequest viene interrotta a metà da Unity
                ↪  senza mai
95              // raggiungere la riga che resetta _isWaitingResponse: lo si resetta qui
96              // per non restare bloccati "in attesa" per sempre
97              _isWaitingResponse = false;
98
99              if (pushToTalkRecorder != null)
100                 pushToTalkRecorder.OnTranscriptionReady -= AskAboutVerb;
101         }
102
103         // -- API pubblica ------------------------------------------------
104         public void AskAboutVerb(string playerQuestionEN)
105         {
106             // trascrizioni ignorate se il pannello è chiuso o la domanda è vuota
107             if (!_isActive) return;
108             if (string.IsNullOrWhiteSpace(playerQuestionEN)) return;
109
110             if (verbDatabase == null)
111             {
112                 OnError?.Invoke("Database verbi non configurato.");
113                 return;
114             }
115
116             // una sola richiesta alla volta
117             if (_isWaitingResponse) return;
118
119             StartCoroutine(SendExtractionRequest(playerQuestionEN));
120         }
121
122         string BuildExtractionPrompt(string playerQuestionEN)
123         {
124             string mappingBlock = verbDatabase.BuildEnglishToItalianMappingBlock();
125
126             return
127                 "You are a strict information-extraction tool inside an
                    ↪  Italian-learning VR game.\n" +
128                 "The player is an English speaker with NO knowledge of Italian yet.
                    ↪  They will ask,\n" +
```

```
129                 "entirely in English, about verbs or words they encountered in the
                    ↪  game.\n\n" +
130                 "If asking about a verb:\n" +
131                 "  - They name it in ENGLISH (e.g. \"have\", \"to have\", \"how do I
                    ↪  conjugate 'to go'?\")\n" +
132                 "  - Extract the Italian infinitive from the list below and the
                    ↪  tense if specified\n\n" +
133                 "If asking about a word:\n" +
134                 "  - They ask \"What does X mean?\" or \"What is X in English?\"\n"
                    ↪  +
135                 "  - Extract the Italian word and mark it as a generic word (not a
                    ↪  verb)\n\n" +
136                 "Here are the verbs you recognize:\n" +
137                 $"{mappingBlock}\n" +
138                 "Your ONLY job is to extract the word/verb and tense (if any), and
                    ↪  answer with\n" +
139                 "EXACTLY one line of JSON, nothing else - no explanation, no
                    ↪  markdown:\n" +
140                 "{\"verbo\": \"<Italian verb infinitive or null>\", \"parola\":
                    ↪  \"<Italian word or null>\", \"tempo\": \"<tense or null>\"}\n\n"
                    ↪  +
141                 "Rules:\n" +
142                 "1. If it's a verb question, fill \"verbo\" with the ITALIAN
                    ↪  infinitive from the list.\n" +
143                 "   Match by meaning even if the player says \"have\", \"to have\",
                    ↪  \"how to conjugate to go\", etc.\n" +
144                 "2. If it's NOT a verb question (e.g. \"What does 'caffè' mean?\"),
                    ↪  fill \"parola\" with\n" +
145                 "   the Italian word the player asked about, in lowercase.\n" +
146                 "3. \"tempo\" is ONLY for verbs: \"presente\" by default, or other
                    ↪  tense if clearly specified.\n" +
147                 "4. NEVER output conjugated forms or translations yourself. NEVER
                    ↪  add commentary.\n" +
148                 "5. Output ONLY the JSON object, on a single line. Set unused fields
                    ↪  to null.\n\n" +
149                 $"Question: \"{playerQuestionEN}\"";
150         }
151
152         IEnumerator SendExtractionRequest(string playerQuestionEN)
153         {
154             _isWaitingResponse = true;
155
156             string prompt = BuildExtractionPrompt(playerQuestionEN);
157             string jsonBody = BuildJsonRequest(model, prompt);
158
159             using (var request = new UnityWebRequest(ollamaUrl, "POST"))
160             {
161                 byte[] bodyRaw = Encoding.UTF8.GetBytes(jsonBody);
162                 request.uploadHandler = new UploadHandlerRaw(bodyRaw);
163                 request.downloadHandler = new DownloadHandlerBuffer();
164                 request.SetRequestHeader("Content-Type", "application/json");
165                 request.timeout = requestTimeoutSeconds;
```

```
166
167             yield return request.SendWebRequest();
168
169             _isWaitingResponse = false;
170
171             if (request.result != UnityWebRequest.Result.Success)
172             {
173                 OnError?.Invoke($"Errore Ollama: {request.error}.");
174                 yield break;
175             }
176
177             string rawGenerated =
                ↪ ExtractOllamaResponseField(request.downloadHandler.text);
178             if (string.IsNullOrWhiteSpace(rawGenerated))
179             {
180                 OnError?.Invoke("Risposta vuota da Ollama.");
181                 yield break;
182             }
183
184             ResolveAnswer(rawGenerated);
185         }
186     }
187
188     void ResolveAnswer(string rawGenerated)
189     {
190         string verbo = ExtractJsonField(rawGenerated, "verbo");
191         string tempo = ExtractJsonField(rawGenerated, "tempo");
192         string parola = ExtractJsonField(rawGenerated, "parola");
193
194         // prima prova a riconoscere la domanda come verbo
195         if (!string.IsNullOrEmpty(verbo))
196         {
197             var verbEntry = verbDatabase.Get(verbo);
198             if (verbEntry != null)
199             {
200                 // il database copre solo il presente indicativo
201                 if (string.IsNullOrEmpty(tempo) || !tempo.Equals("presente",
                    ↪ StringComparison.OrdinalIgnoreCase))
202                 {
203                     OnHelpNeeded?.Invoke(
204                         $"For now I can only explain
                            ↪ \"{verbEntry.infinitiveEN}\" in the present
                            ↪ tense.");
205                     return;
206                 }
207
208                 // aggiunge contesto se la parola è stata vista nei dialoghi
209                 string baseResponse = verbEntry.BuildPresenteText();
210                 if (GameManager.Instance != null &&
211
                        ↪ GameManager.Instance.State.learnedWords.Contains(verbo.ToLowerInvariant()
212                 {
```

```
213                         baseResponse += "\n\n(You already saw this word in the
                            ↪  game!)";
214                     }
215                     OnAnswerReady?.Invoke(baseResponse);
216                     return;
217                 }
218             }
219 
220             // poi prova a riconoscerla come parola o frase generica
221             if (!string.IsNullOrEmpty(parola))
222             {
223                 // prima la frase intera così com'è: gestisce voci già nel
224                 // vocabolario come "va bene", "per favore", "nessun problema"
225                 string exactAnswer = TryResolveSingleTerm(parola);
226                 if (exactAnswer != null)
227                 {
228                     OnAnswerReady?.Invoke(exactAnswer);
229                     return;
230                 }
231 
232                 // se la frase ha più parole, le spiega una per una
233                 // invece di arrendersi sulla frase intera
234                 var words = parola.Split(new[] { ' ' },
                    ↪  StringSplitOptions.RemoveEmptyEntries);
235                 if (words.Length > 1)
236                 {
237                     var explanations = new List<string>();
238                     foreach (var w in words)
239                     {
240                         string wordAnswer = TryResolveSingleTerm(w);
241                         if (wordAnswer != null)
242                             explanations.Add(wordAnswer);
243                     }
244 
245                     if (explanations.Count > 0)
246                     {
247                         OnAnswerReady?.Invoke(string.Join("\n\n---\n\n",
                            ↪  explanations));
248                         return;
249                     }
250                 }
251             }
252 
253             // nessuno dei due casi: messaggio fisso di aiuto
254             OnHelpNeeded?.Invoke(
255                 "I didn't understand your question. Try asking about a specific word
                    ↪  or verb you saw in the game, " +
256                 "e.g. \"What does 'caffè' mean?\" or \"How do you conjugate 'to
                    ↪  go'?\"");
257         }
258 
259         /// prova a spiegare un singolo termine italiano: prima come voce del
```

```
260         /// vocabolario, poi come forma verbale (infinito o coniugata); restituisce
261         /// null se non lo trova da nessuna parte: nessun testo generato da
262         /// Ollama entra mai in questo metodo, solo dati verificati a mano
263         string TryResolveSingleTerm(string term)
264         {
265             if (string.IsNullOrWhiteSpace(term)) return null;
266 
267             var vocabEntry = vocabularyDatabase?.Get(term);
268             if (vocabEntry != null)
269             {
270                 string response = vocabEntry.GetDefinition();
271                 if (GameManager.Instance != null &&
272 
                        ↪ GameManager.Instance.State.learnedWords.Contains(term.ToLowerInvariant()))
273                 {
274                     response += "\n\n(You already saw this word in the game!)";
275                 }
276                 return response;
277             }
278 
279             var conjugatedEntry = verbDatabase.GetByAnyForm(term);
280             if (conjugatedEntry != null)
281             {
282                 string response = conjugatedEntry.BuildPresenteText();
283                 if (GameManager.Instance != null &&
284 
                        ↪ GameManager.Instance.State.learnedWords.Contains(term.ToLowerInvariant()))
285                 {
286                     response += "\n\n(You already saw this word in the game!)";
287                 }
288                 return response;
289             }
290 
291             return null;
292         }
293 
294         static string ExtractJsonField(string rawText, string fieldName)
295         {
296             if (string.IsNullOrEmpty(rawText)) return null;
297 
298             // cattura l'intero valore tra virgolette (può contenere spazi)
299             // oppure il valore null senza virgolette
300             var pattern = $"\"{fieldName}\"\\s*:\\s*(\"([^\"]*)\"|null)";
301             var match = Regex.Match(rawText, pattern, RegexOptions.IgnoreCase);
302             if (!match.Success) return null;
303 
304             string value = match.Groups[2].Success ? match.Groups[2].Value.Trim() :
                ↪ null;
305             return string.IsNullOrEmpty(value) ? null : value.ToLowerInvariant();
306         }
307 
308         string BuildJsonRequest(string modelName, string prompt)
```

```
        {
            string escapedPrompt = EscapeJson(prompt);
            string escapedModel = EscapeJson(modelName);
            return "{\"model\":\"" + escapedModel + "\"," +
                   "\"prompt\":\"" + escapedPrompt + "\"," +
                   "\"stream\":false}";
        }

        string EscapeJson(string s)
        {
            return s.Replace("\\", "\\\\")
                    .Replace("\"", "\\\"")
                    .Replace("\n", "\\n")
                    .Replace("\r", "");
        }

        string ExtractOllamaResponseField(string json)
        {
            const string key = "\"response\":\"";
            int start = json.IndexOf(key);
            if (start < 0) return null;
            start += key.Length;

            var sb = new StringBuilder();
            for (int i = start; i < json.Length; i++)
            {
                char c = json[i];
                if (c == '\\' && i + 1 < json.Length)
                {
                    char next = json[i + 1];
                    if (next == 'n') { sb.Append(' '); i++; continue; }
                    if (next == '"') { sb.Append('"'); i++; continue; }
                    if (next == '\\') { sb.Append('\\'); i++; continue; }
                    continue;
                }
                if (c == '"') break;
                sb.Append(c);
            }
            return sb.ToString().Trim();
        }
    }
}
```

C.8 VoiceRecognitionLogger.cs

Registrazione dei tentativi vocali e assegnazione del campo `IsMatch` (§4.7).

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
```

```
using UnityEngine;
using UnityEngine.SceneManagement;
using Bellavalle.Core;
using Bellavalle.Data;
using Whisper;

namespace Bellavalle.Voice
{
    public class VoiceRecognitionLogger : MonoBehaviour
    {
        [Header("Riferimenti")]
        [SerializeField] PushToTalkRecorder pushToTalkRecorder;

        [Header("Sessione di test")]
        [Tooltip("Da incrociare col questionario demografico compilato durante il test. Impostalo prima di ogni sessione.")]
        [SerializeField] string participantId = "";

        [Tooltip("Nome del file audio della sessione (es. P01_recording.m4a). Usato per sincronizzare la valutazione umana con il CSV.")]
        [SerializeField] string audioFileName = "";

        string _filePath;
        bool _fileReady = false;

        DialogueNode _currentNode;
        string _currentTreeId = "";

        // -- Contatore tentativi per nodo ------------------------------
        // chiave: nodeId; si azzera a ogni nuovo dialogo
        // conta solo i tentativi su nodi con opzioni vocali (IsMatch 1/0)
        readonly Dictionary<string, int> _nodeAttempts = new Dictionary<string, int>();

        // -- Marcatori di audio vuoto prodotti da Whisper ---------------
        // per assegnare n/d_audio invece di valutare il match
        static readonly HashSet<string> _emptyAudioMarkers = new HashSet<string>(
            StringComparer.OrdinalIgnoreCase)
        {
            "[blank_audio]", "[ silence ]", "[silence]",
            "[musica]", "[music]", "[rumore]", "[noise]", ""
        };

        // -- Singleton persistente -------------------------------------
        public static VoiceRecognitionLogger Instance { get; private set; }

        void Awake()
        {
            if (Instance != null && Instance != this)
            {
                Destroy(gameObject);
                return;
```

```
55              }
56
57              Instance = this;
58              DontDestroyOnLoad(gameObject);
59          }
60
61          // -- Unity lifecycle ------------------------------------------
62          void OnEnable()
63          {
64              if (pushToTalkRecorder != null)
65                  pushToTalkRecorder.OnTranscriptionResultReady +=
                    ↪ OnTranscriptionResultReady;
66
67              EventBus.On(GameEvent.NodeEntered, OnNodeEntered);
68              EventBus.On(GameEvent.DialogueStarted, OnDialogueStarted);
69              EventBus.On(GameEvent.DialogueEnded, OnDialogueEnded);
70          }
71
72          void OnDisable()
73          {
74              if (pushToTalkRecorder != null)
75                  pushToTalkRecorder.OnTranscriptionResultReady -=
                    ↪ OnTranscriptionResultReady;
76
77              EventBus.Off(GameEvent.NodeEntered, OnNodeEntered);
78              EventBus.Off(GameEvent.DialogueStarted, OnDialogueStarted);
79              EventBus.Off(GameEvent.DialogueEnded, OnDialogueEnded);
80          }
81
82          void Start()
83          {
84              // il file di sessione viene creato se il ParticipantId
85              // e' stato impostato in Inspector prima dell'avvio
86              if (!string.IsNullOrWhiteSpace(participantId))
87                  InitialiseFile();
88          }
89
90          public void SetParticipantId(string id)
91          {
92              // un identificativo vuoto viene ignorato così da non avere file senza
                ↪ nome
93              if (string.IsNullOrWhiteSpace(id)) return;
94
95              participantId = id;
96              InitialiseFile();
97          }
98
99          public void SetAudioFileName(string name) => audioFileName = name;
100
101         // -- Inizializzazione file ------------------------------------
102         void InitialiseFile()
103         {
```

```
104             string safeId =
                ↪  string.Concat(participantId.Split(Path.GetInvalidFileNameChars()));
105             string timestamp = DateTime.Now.ToString("yyyy-MM-dd_HH-mm-ss");
106             string fileName = $"VoiceLog_{safeId}_{timestamp}.csv";
107             _filePath = Path.Combine(Application.persistentDataPath, fileName);
108
109             string header = string.Join(",",
110                 "InteractionType", "AttemptNumber",
111                 "Timestamp", "ParticipantId", "AudioFile", "Scene", "TreeId",
                    ↪  "NodeId",
112                 "NpcLineIT", "ExpectedPhrases", "TranscribedText",
113                 "MatchedOption", "IsMatch", "AvgConfidence", "TokenCount");
114
115             File.WriteAllText(_filePath, header + Environment.NewLine,
                ↪  Encoding.UTF8);
116             _fileReady = true;
117         }
118
119         // -- Tracciamento contesto dialogo ------------------------------
120         void OnDialogueStarted(object data)
121         {
122             _currentTreeId = data as string ?? "";
123             _nodeAttempts.Clear();
124         }
125
126         void OnDialogueEnded(object data)
127         {
128             _currentTreeId = "";
129             _currentNode = null;
130         }
131
132         void OnNodeEntered(object data) => _currentNode = data as DialogueNode;
133
134         // -- Dialogo: riceve il risultato completo da Whisper -----------
135         void OnTranscriptionResultReady(WhisperResult result)
136         {
137             if (result == null) return;
138
139             string text = (result.Result ?? "").Trim();
140             float avgConfidence = ComputeAverageConfidence(result);
141             int tokenCount = CountRealTokens(result);
142
143             string expectedPhrases = "";
144             string matchedOption = "";
145             string isMatch;
146             string attemptNumber = "";
147
148             var node = _currentNode;
149
150             // -- IsMatch granulare ------------------------------------
151             // n/d_audio -> Whisper non ha ricevuto segnale utile
152             // n/d_nome  -> nodo cattura nome: nessun match calcolabile
```

```
153             // n/d_vuoto -> nodo senza opzioni vocali (pannello puro)
154             // 1 / 0 -> match riuscito / fallito su opzioni vocali
155             if (_emptyAudioMarkers.Contains(text))
156             {
157                 isMatch = "n/d_audio";
158             }
159             else if (node == null)
160             {
161                 isMatch = "n/d_vuoto";
162             }
163             else if (node.isNameCapture)
164             {
165                 expectedPhrases = "[nome libero]";
166                 isMatch = "n/d_nome";
167             }
168             else if (node.options != null && node.options.Length > 0)
169             {
170                 expectedPhrases = string.Join(" | ", node.options.Select(o =>
                    ↪  o.text_IT));
171                 int matchIndex = VoiceMatchUtils.FindMatchingOption(text,
                    ↪  node.options);
172                 isMatch = matchIndex >= 0 ? "1" : "0";
173                 matchedOption = matchIndex >= 0 ? node.options[matchIndex].text_IT :
                    ↪  "";
174 
175                 // AttemptNumber: incrementa solo per nodi vocali valutabili
176                 string nodeKey = node.nodeId ?? "";
177                 if (!_nodeAttempts.ContainsKey(nodeKey))
178                     _nodeAttempts[nodeKey] = 0;
179                 _nodeAttempts[nodeKey]++;
180                 attemptNumber = _nodeAttempts[nodeKey].ToString();
181             }
182             else
183             {
184                 isMatch = "n/d_vuoto";
185             }
186 
187             WriteRow(
188                 "dialogo",
189                 attemptNumber,
190                 DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
191                 participantId,
192                 audioFileName,
193                 SceneManager.GetActiveScene().name,
194                 _currentTreeId,
195                 node?.nodeId ?? "",
196                 node?.npcLine_IT ?? "",
197                 expectedPhrases,
198                 text,
199                 matchedOption,
200                 isMatch,
201                 avgConfidence >= 0f ? avgConfidence.ToString("F3") : "",
```

```
                tokenCount.ToString()
            );
        }

        // -- Listen and repeat: chiamato da ListenAndRepeatActivator -----
        public void LogListenAndRepeat(
            string targetWord,
            int attemptNumber,
            bool isMatch,
            WhisperResult result)
        {
            if (result == null) return;

            string text = (result.Result ?? "").Trim();
            float avgConfidence = ComputeAverageConfidence(result);
            int tokenCount = CountRealTokens(result);

            WriteRow(
                "listen_repeat",
                attemptNumber.ToString(),
                DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
                participantId,
                audioFileName,
                SceneManager.GetActiveScene().name,
                "",
                "",
                "",
                targetWord,
                text,
                isMatch ? targetWord : "",
                isMatch ? "1" : "0",
                avgConfidence >= 0f ? avgConfidence.ToString("F3") : "",
                tokenCount.ToString()
            );
        }

        // -- Export griglia valutazione percettiva ----------------------
        // tasto destro sul componente nell'Inspector -> esporta griglia valutazione
        // legge il VoiceLog_*.csv della sessione corrente e produce
        // GrigliaValutazione_*.csv nella stessa cartella, con le colonne
        // pre-compilate dai dati del logger e le colonne vuote (T, P, C,
        // Fenomeno, Note) pronte per la valutazione manuale
        // salta le righe n/d_nome (cattura nome) e n/d_vuoto (non valutabili)
        [ContextMenu("Esporta griglia valutazione")]
        public void ExportGriglia()
        {
            if (!_fileReady) return;
            if (!File.Exists(_filePath)) return;

            try
            {
                string dir = Path.GetDirectoryName(_filePath);
```

```
254                 string baseName = Path.GetFileNameWithoutExtension(_filePath);
255                 string grigliePath = Path.Combine(dir,
256                     baseName.Replace("VoiceLog_", "GrigliaValutazione_") + ".csv");
257
258                 var sb = new StringBuilder();
259                 sb.AppendLine(string.Join(",",
260                     "PID", "Timestamp", "AudioFile", "Tipo",
261                     "FraseAttesa", "Trascritto", "IM",
262                     "T", "P", "C", "Fenomeno", "Note"));
263
264                 string[] logLines = File.ReadAllLines(_filePath, Encoding.UTF8);
265
266                 for (int i = 1; i < logLines.Length; i++)
267                 {
268                     string line = logLines[i].Trim();
269                     if (string.IsNullOrEmpty(line)) continue;
270
271                     string[] fields = ParseCsvLine(line);
272                     if (fields.Length < 15) continue;
273
274                     // legenda indici colonne VoiceLog:
275                     // 0=InteractionType  1=AttemptNumber  2=Timestamp
276                     // 3=ParticipantId    4=AudioFile      5=Scene
277                     // 6=TreeId           7=NodeId         8=NpcLineIT
278                     // 9=ExpectedPhrases  10=TranscribedText
279                     // 11=MatchedOption   12=IsMatch
280                     // 13=AvgConfidence   14=TokenCount
281
282                     string im = fields[12];
283
284                     // salta righe non valutabili con la griglia percettiva
285                     if (im == "n/d_nome") continue;
286                     if (im == "n/d_vuoto") continue;
287
288                     sb.AppendLine(string.Join(",",
289                         CsvEscape(fields[3]),   // PID
290                         CsvEscape(fields[2]),   // Timestamp
291                         CsvEscape(fields[4]),   // AudioFile
292                         CsvEscape(fields[0]),   // Tipo
293                         CsvEscape(fields[9]),   // FraseAttesa
294                         CsvEscape(fields[10]),  // Trascritto
295                         CsvEscape(im),          // IM
296                         "", "", "", "", ""       // T P C Fenomeno Note
297                     ));
298                 }
299
300                 File.WriteAllText(grigliePath, sb.ToString(), Encoding.UTF8);
301             }
302             catch (Exception)
303             {
304                 // in caso di errore la griglia non viene prodotta
305             }
```

```
306         }
307
308         // -- Parser CSV minimale ---------------------------------------
309         // gestisce campi tra virgolette con virgole interne
310         static string[] ParseCsvLine(string line)
311         {
312             var fields = new List<string>();
313             var current = new StringBuilder();
314             bool inQuotes = false;
315
316             for (int i = 0; i < line.Length; i++)
317             {
318                 char c = line[i];
319                 if (inQuotes)
320                 {
321                     if (c == '"')
322                     {
323                         if (i + 1 < line.Length && line[i + 1] == '"')
324                         { current.Append('"'); i++; }
325                         else
326                         { inQuotes = false; }
327                     }
328                     else { current.Append(c); }
329                 }
330                 else
331                 {
332                     if (c == '"') { inQuotes = true; }
333                     else if (c == ',') { fields.Add(current.ToString());
                        ↪  current.Clear(); }
334                     else { current.Append(c); }
335                 }
336             }
337             fields.Add(current.ToString());
338             return fields.ToArray();
339         }
340
341         // -- Confidenza media ------------------------------------------
342         // richiede l'opzione Enable Tokens attiva su WhisperManager:
343         // senza token restituisce -1 e il campo nel CSV resta vuoto
344         float ComputeAverageConfidence(WhisperResult result)
345         {
346             if (result.Segments == null) return -1f;
347
348             float sum = 0f;
349             int count = 0;
350
351             foreach (var seg in result.Segments)
352             {
353                 if (seg.Tokens == null) continue;
354                 foreach (var tok in seg.Tokens)
355                 {
356                     if (tok.IsSpecial) continue;
```

```
357                     sum += tok.Prob;
358                     count++;
359                 }
360             }
361 
362             return count > 0 ? sum / count : -1f;
363         }
364 
365         int CountRealTokens(WhisperResult result)
366         {
367             if (result.Segments == null) return 0;
368 
369             int count = 0;
370             foreach (var seg in result.Segments)
371             {
372                 if (seg.Tokens == null) continue;
373                 foreach (var tok in seg.Tokens)
374                     if (!tok.IsSpecial) count++;
375             }
376             return count;
377         }
378 
379         // -- Scrittura CSV ---------------------------------------------
380         void WriteRow(params string[] fields)
381         {
382             // nessuna riga viene scritta prima della creazione del file
383             if (!_fileReady) return;
384 
385             try
386             {
387                 string line = string.Join(",", fields.Select(CsvEscape));
388                 File.AppendAllText(_filePath, line + Environment.NewLine,
                    ↪  Encoding.UTF8);
389             }
390             catch (Exception)
391             {
392                 // in caso di errore la riga viene scartata
393             }
394         }
395 
396         string CsvEscape(string field)
397         {
398             field ??= "";
399             bool needsQuotes = field.Contains(',') || field.Contains('"') ||
400                                field.Contains('\n') || field.Contains('\r');
401             if (needsQuotes)
402                 field = "\"" + field.Replace("\"", "\"\"") + "\"";
403             return field;
404         }
405     }
406 }
```

APPENDICE D

Modulo di raccolta dati

Il modulo, predisposto su Google Moduli, è riportato nella versione compilata da tutti i partecipanti. Le sezioni A e B sono state compilate prima di indossare il visore, le sezioni successive subito dopo la sessione di gioco, a visore tolto. La numerazione degli item corrisponde a quella del modulo. Le risposte corrette del post-test sono indicate in corsivo tra parentesi e non erano visibili ai partecipanti.

Modulo 1: Consenso e anagrafica

Section A: Informed consent "You are about to take part in a test session for a Master's thesis in Digital Humanities on a virtual reality video game for learning Italian. Participation is voluntary and you may stop at any time without giving any reason. During gameplay, the game records only the textual transcriptions of what you say. Separately, the researcher will make an audio recording of the session, used solely to compare your speech with the system's transcriptions and deleted once the analysis is complete. Data will be handled anonymously using a participant code and used solely for research purposes."

1. I confirm that I have read the above information and consent to taking part in the study. [Casella di spunta, obbligatoria]

Section B: Demographics and background

2. Indicate your age [Risposta breve]
3. Indicate your gender [Scelta singola]: Male / Female / Prefer not to say / Opzione 4[91]
4. Native language [Risposta breve]
5. Variety of English you usually speak [Risposta breve]
6. City of origin [Risposta breve]

[91] Voce segnaposto rimasta per errore nel modulo.

7. Other languages you know besides English and approximate level [Paragrafo]

8. Prior knowledge of Italian before today [Scala 1–5: 1 = No knowledge at all (can't read or speak); 5 = Advanced (can hold a conversation and speak)]. Criterio di esclusione: punteggio superiore a 1.

9. If you know some words in italian how do you know them? [Scelta multipla]: None / Tv shows/Films / Socials / Altro (risposta libera)

10. Prior experience with virtual reality [Scala 1–5: 1 = Never used VR; 5 = Use it very frequently]

11. Frequency of video game play [Scala 1–5: 1 = Never play video games; 5 = Daily or almost daily]

12. Familiarity with voice assistants [Scala 1–5: 1 = Never used one; 5 = Use it constantly / very familiar]

13. Which voice assistant(s) do you use? [Scelta multipla]: Siri / Alexa / Google Assistant / Cortana / I don't use any / Altro (risposta libera)

14. How do you typically activate your voice assistant? [Scelta multipla]: Wake word (e.g. "Hey Siri", "Alexa") / Button press / I don't use any / Altro (risposta libera)

15. How often does your voice assistant fail to recognize what you say? [Scala 1–5: 1 = Never / almost never; 5 = Very often / almost always]

Modulo 2: Post-sessione

2.1 Section C: Knowledge post-test e autovalutazione

Istruzione: *Answer based on what you remember from the game.* Tutti gli item del post-test sono a scelta singola.

16. Come ti chiami?
    How would you answer if someone asked you this?
    Mi chiamo... *(corretta)* / Sto bene / Sono le tre / Arrivederci

17. "Biglietto, prego"?
    What does this mean?
    A greeting / A request to show your ticket *(corretta)* / A thank you / A question about the timetable

18. How do you say "please" in Italian?
    Grazie / Per favore *(corretta)* / Scusa / Va bene

19. Which of these words is NOT a drink?
    Acqua / Cappuccino / Sedia *(corretta)* / Caffè

20. What would you say in reply to “Grazie”?
    Prego *(corretta)* / Per favore / Buongiorno / Scusa
21. What is the pastry often eaten at breakfast at the bar with coffee?
    Cornetto *(corretta)* / Tazza / Frigorifero / Panca
22. The conductor says “Biglietto, prego.” What does he want?
    He is greeting you / He is asking you to buy a ticket / He is asking you to show your ticket *(corretta)* / He is telling you the price
23. “Arrivederci” is used when...
    You arrive somewhere / You order something / You say goodbye *(corretta)* / You say thank you
24. The neighbour asks “Sei nuovo qui?” What is she asking?
    What your name is / Where you are going / Whether you are new to the area *(corretta)* / Whether you need help
25. You introduce yourself to your neighbour. She replies “Piacere!” What does she mean?
    Thank you / Nice to meet you *(corretta)* / You’re welcome / See you tomorrow
26. You walk into the living room and see a panel saying “C’è un divano.” What is in the room?
    A bed / A sofa *(corretta)* / A table / A plant
27. A panel says “La cucina.” Where are you?
    In the bedroom / In the bathroom / In the kitchen *(corretta)* / In the living room
28. What does “Nessun problema” mean?
    See you tomorrow / No problem *(corretta)* / Nice to meet you / You’re welcome
29. What does “Scusa” mean?
    Thank you / Sorry / Excuse me *(corretta)* / Please / Goodbye
30. How much do you feel you learned during the game? [Scala 1–5: 1 = Nothing at all; 5 = A great deal]
31. In which part of the game did you feel you learned the most? [Scelta multipla]:
    The bar / Meeting the neighbour Lina / The ticket inspector at the station / The house / The grammar guide in the inventory / None in particular
32. Why? Tell us in your own words what you remember about that part. [Paragrafo]

2.2 Slater-Usoh-Steed (Presence)

Istruzione: *Rate each statement on a scale from 1 to 5.*

33. I had a sense of “being there” in ItalianVRExperience. [1 = Not at all / At no time; 5 = Very much / Almost all the time]

34. There were times during the experience when ItalianVRExperience felt like real to me. [1 = Not at all / At no time; 5 = Very much / Almost all the time]
35. ItalianVRExperience seems to me to be more like... [1 = Images that I saw; 5 = Somewhere that I visited]
36. I had a stronger sense of... [1 = Being elsewhere; 5 = Being in an Italian place]
37. I think of ItalianVRExperience as a place in a way similar to other places I have been. [1 = Not at all / At no time; 5 = Very much / Almost all the time]

Punteggio: media delle risposte ai cinque item, su scala da 1 a 5 (§5.2.3).

2.3 System Usability Scale

Istruzione: *Rate each statement: 1 = Strongly disagree, 5 = Strongly agree.*

38. I think I would like to use this game often, if I could
39. I found the game unnecessarily complex
40. I thought the game was easy to use
41. I think I would need the support of an expert to use this game
42. I found the various functions of the game well integrated.
43. I found there was too much inconsistency in this game
44. I imagine that most people would learn to use this game very quickly
45. I found the game very cumbersome to use
46. I felt confident using the game
47. I needed to learn a lot of things before I could get going with this game

Punteggio: per gli item dispari si sottrae 1 al valore indicato; per gli item pari si sottrae il valore indicato da 5. La somma dei contributi, moltiplicata per 2,5, produce un punteggio da 0 a 100.

2.4 Speech Recognition

48. In which moments did you have to repeat a sentence because it was not recognised? [Scelta multipla]: With the barista / With the neighbour / In the grammar guide / This never happened to me / At the bar with the objects
49. Speaking Italian out loud felt... [Scelta singola]: More natural / Makes no difference / More difficult
50. Did you ever feel the need to speak more clearly or more slowly to be understood by the game? [Scelta singola]: Yes / No / I don't remember

51. If yes, when did that happen? [Risposta breve]
52. Think of a moment when you spoke with a character: did it feel like talking to a real person? Why? [Paragrafo]

2.5 Final open-ended questions

53. What would you change about the game? [Paragrafo]
54. Would you recommend this game to someone who wants to learn Italian? Why? [Paragrafo]

2.6 Considerazioni etiche e privacy

- La partecipazione è stata volontaria, con la possibilità di interrompere la sessione in qualsiasi momento senza fornire motivazioni. Ciascun partecipante ha espresso il proprio consenso informato prima dell'inizio della sessione (Modulo 1, Section A).
- Il gioco non salva alcun file audio: il log registra soltanto le trascrizioni testuali prodotte da Whisper e i relativi metadati.
- La sessione è stata registrata in audio dalla ricercatrice tramite dispositivo mobile, previo consenso esplicito dei partecipanti, al solo scopo di confrontare il parlato con le trascrizioni del sistema. Le registrazioni non sono condivise con terzi e saranno eliminate al termine dell'analisi.
- Nei log e nei moduli compare soltanto il `ParticipantId`, mai il nome del partecipante.
- La corrispondenza tra nome e `ParticipantId` è conservata su un foglio cartaceo, separato dai dati raccolti, e sarà eliminata al termine dello studio.

APPENDICE E

Griglia di valutazione percettiva delle trascrizioni

E.1 Istruzioni

La griglia è stata compilata dopo ciascuna sessione in un foglio di calcolo, con un foglio per partecipante. Nelle prime colonne sono riportati i dati del log di `VoiceRecognitionLogger`; le colonne di valutazione sono state compilate ascoltando la registrazione audio della sessione, individuando il segmento corrispondente a ciascun tentativo tramite le colonne di servizio descritte al §5.2.4. Per ogni tentativo vocale si segue questo ordine:

1. **T, fedeltà della trascrizione.** Ascoltare il tentativo e confrontarlo con `TranscribedText`: Whisper riflette ciò che il partecipante ha effettivamente pronunciato?
   - Y: Whisper ha scritto ciò che il partecipante ha pronunciato, anche se con accento;
   - N: Whisper ha prodotto un testo diverso da quello udito, perché lo ha corretto autonomamente o perché lo ha frainteso.
2. **P, accettabilità della pronuncia.** Indipendentemente da T: la pronuncia sarebbe stata comprensibile per un madrelingua italiano nel contesto della frase?
   - Y: nonostante l'accento, la parola era riconoscibile nel contesto comunicativo;
   - N: l'errore rendeva la parola incomprensibile o ne cambiava il significato (per esempio *tazza* pronunciata come *tassa*, un'altra parola italiana).

   Quando T è N, P non incide sulla categoria, ma viene comunque compilato quando l'ascolto lo consente.
3. **Fenomeno.** Ogni volta che nella pronuncia è udibile una deviazione dalla norma, indicarne il codice (§E.3), anche quando P è Y. Sono disponibili due colonne, per i casi in cui cooccorrono due fenomeni. L'annotazione è obbligatoria quando T o P è N. Il raddoppiamento fonosintattico (*a casa* → [ak'ka:za]) e la [z] intervocalica (*casa* → ['ka:za]), tratti dell'italiano regionale, sono annotati come tali nelle note e non come deviazioni.
4. **Note.** Descrivere in forma libera ciò che si è udito ogni volta che si indica un fenomeno o si usa il codice ALTRO.

5. **Problemi del microfono.** Se il tentativo è compromesso dall'acquisizione del segnale, la nota inizia con una di tre formule fisse: *MIC: attivazione accidentale*, *MIC: attivazione in ritardo*, *MIC: interruzione brusca*. In questi casi la categoria è MIC e il tentativo è escluso dal calcolo delle sei categorie di valutazione; si annotano soltanto i fenomeni di pronuncia effettivamente percepiti, non il taglio del segnale.
6. **Pannelli e modalità vocale.** Indicare se il partecipante stava consultando i pannelli di supporto durante il tentativo (Y/N) e la modalità di emissione: neutra, voce alta (per esempio per frustrazione dopo più ripetizioni) oppure scandita (parole separate e articolate con enfasi).
7. **Categoria.** È calcolata automaticamente dal foglio di calcolo incrociando T e P con il valore di `IsMatch`, secondo la legenda riportata al §E.4. Le righe con `IsMatch` = n/d (cattura del nome, domande al tutor, nodi senza risposta vocale attesa) sono escluse dalla valutazione.

E.2 Colonne della griglia

Colonna	Descrizione
Tipo	Tipo di interazione: `dialogo` o `listen_repeat`.
Scena, Nodo	Scena e nodo di dialogo in cui è avvenuto il tentativo.
Tent.	Numero progressivo del tentativo sul nodo.
Battuta NPC	Battuta che precede la risposta; per il *listen-and-repeat*, la parola target.
Frase attesa	Opzioni di risposta previste dal nodo (`ExpectedPhrases`).
Trascritto	Testo prodotto da Whisper (`TranscribedText`).
MatchedOption	Opzione riconosciuta dal sistema, se presente.
IM	Esito del riconoscimento (`IsMatch`): 1, 0 oppure n/d.
T	Trascrizione fedele? (Y/N)
P	Pronuncia accettabile per un madrelingua? (Y/N)
Fenomeno 1, 2	Codice del fenomeno fonologico udito (§E.3).
Note	Osservazioni libere della valutatrice.
Pannelli	Uso dei pannelli di supporto durante il tentativo (Y/N).
Modalità vocale	Neutra, voce alta o scandita.
C	Categoria: MG, MI, OC, FN, RC, TI oppure MIC, calcolata automaticamente.

E.3 Codici dei fenomeni fonologici

I codici seguenti descrivono i fenomeni fonologici attesi per apprendenti anglofoni di italiano di livello A1 e osservati nelle sessioni iniziali (P01, P02) o nelle successive. Gli esempi riportano la forma attesa e la trascrizione prodotta da Whisper; la frequenza di ciascun codice è riportata nella Tabella 7.

Codice	**Fenomeno**	**Esempi**
GEM	Scempiamento: consonante doppia resa come semplice	*donna* → “dona”; *cappuccino* → “capucino”
GEM+	Ipercorrettismo geminato: consonante semplice resa come doppia	*arrivato* → “arrivatto”; *tavolino* → “tavollino”
SUB	Sostituzione consonantica con un suono vicino: affricata resa come sibilante, /r/ resa come /l/, desonorizzazione	*tazza* → “tassa”; *frigorifero* → “frigolifero”; *vado* → “fado”
PAL	Palatalizzazione: lettura di *ch* come /tʃ/ per interferenza dell’ortografia inglese	*che bello* → “ce bello”
GR	Nesso consonantico iniziale semplificato, desonorizzato o spezzato	*sgabello* → “scabello”, “sabello”
RHO	Resa non vibrante di /r/: approssimante inglese o assente; rilevabile solo all’ascolto	*arrivederci*, *frigorifero*
VOC-a	Chiusura di /a/ verso /o/	*acqua* → “ocqua”
VOC-e	Alterazione di /e/ finale	*mille* → “millie”
VOC-d	Dittongazione o timbro vocalico inglese	*persona* → “pearsona”; *piacere* → “piaciere”
ELO	Allungamento vocalico	*uomo* → “ooomo”
FUS	Segmentazione anomala dei confini di parola: parole fuse o spezzate	*a domani* → “Ado mani”; *acqua* → “A qua”
ALTRO	Fenomeno non classificabile con i codici precedenti, descritto nelle note	

E.4 Legenda delle categorie

C	Condizione	Interpretazione
MG	IM=1, T=Y, P=Y	**Match genuino.** Whisper ha trascritto fedelmente una pronuncia accettabile e il sistema l'ha riconosciuta correttamente. Annotare l'eventuale fenomeno udibile.
MI	IM=1, T=Y, P=N	**Match su pronuncia insufficiente.** Whisper è stato fedele, ma il sistema ha accettato una pronuncia che un madrelingua non avrebbe compreso: il riconoscimento è troppo permissivo. Annotare il fenomeno.
OC	IM=1, T=N	***Over-correction.*** Whisper ha prodotto un testo diverso da quello udito, riconducendolo alla forma attesa per effetto del proprio modello linguistico interno, e il sistema lo ha riconosciuto. Annotare il fenomeno.
FN	IM=0, T=Y, P=Y	**Falso negativo.** Whisper ha trascritto fedelmente una pronuncia accettabile, ma il sistema non l'ha riconosciuta: il limite è nel meccanismo di confronto, non nella pronuncia né nella trascrizione. Annotare l'eventuale fenomeno udibile.
RC	IM=0, T=Y, P=N	**Rifiuto corretto.** Whisper ha trascritto fedelmente una pronuncia non accettabile, e il sistema l'ha rifiutata correttamente. Annotare il fenomeno.
TI	IM=0, T=N	**Trascrizione infedele.** Whisper ha frainteso la pronuncia e il riconoscimento non è riuscito: l'errore è nella trascrizione, non nel meccanismo di confronto. Annotare il fenomeno.
MIC	nota "MIC: . . . "	**Problema del microfono.** L'errore dipende dall'acquisizione del segnale (attivazione accidentale, in ritardo o interruzione brusca). Escluso dal calcolo delle sei categorie.

E.5 Nota interpretativa sulle categorie OC e TI

Le categorie OC e TI raccolgono casi fonologicamente eterogenei, che le sole etichette non permettono di distinguere. Nei casi OC (IsMatch = 1, T = N), la correzione autonoma di Whisper può riguardare una deviazione minima, come una geminata assente o una vocale finale ridotta, oppure una pronuncia lontana dalla norma, in cui il modello linguistico ha ricostruito la parola dal contesto senza che il segnale audio fosse sufficientemente vicino alla forma italiana: nel primo caso la difficoltà fonologica è marginale, nel secondo è reale e mascherata dal sistema. Nei casi TI (IsMatch = 0, T = N), la trascrizione infedele può derivare da una deviazione fonetica minima che ha disorientato Whisper, oppure da un segnale sostanzialmente irriconoscibile. In entrambe le categorie, le colonne Fenomeno e Note sono gli unici strumenti che permettono di distinguere questi sottotipi: per questa ragione la loro compilazione è obbligatoria nelle righe con T = N o P = N. La colonna Fenomeno consente inoltre di aggregare i casi per tipo di fenomeno, indipendentemente dalla categoria; l'incrocio tra fenomeni e categorie è presentato al §5.3.

E.6 Implementazione nel foglio di calcolo

La griglia è organizzata in un foglio di calcolo per partecipante, in cui ogni riga corrisponde a un tentativo vocale del log. Le colonne ricavate dal log (tipo di interazione, scena, nodo, frase attesa, trascrizione, `IsMatch`) sono precompilate da codice; le colonne T, P, Fenomeno, Note, Pannelli e Modalità vocale sono compilate all'ascolto, con menu a tendina per i valori ammessi.

La colonna Categoria contiene una formula che applica la legenda del §E.4 verificando le condizioni in ordine: se la nota inizia con *MIC:*, la categoria è MIC; se `IsMatch` non è compilato, la cella resta vuota; se T è N, la categoria è OC con `IsMatch` = 1 e TI con `IsMatch` = 0; se T è Y, la categoria è MG o MI con `IsMatch` = 1 e FN o RC con `IsMatch` = 0, a seconda del valore di P. In questo modo nessuna categoria è assegnata manualmente, e ogni correzione di T, P o `IsMatch` si riflette automaticamente sulla categoria.

Un foglio di riepilogo aggrega i dati dei quindici partecipanti tramite formule di conteggio condizionato, che contano le righe corrispondenti a uno o più criteri contemporaneamente, per esempio le righe con tipo *dialogo* e categoria RC. Il riepilogo restituisce la distribuzione delle categorie per partecipante, per scena, per tipo di interazione, per uso dei pannelli e per modalità vocale, e l'incrocio tra fenomeni fonologici e categorie.

Bibliografia

Arnaud, Pierre S. L. “Nattinger, James R. et Jeanette S. Decarrico (1992): Lexical Phrases and Language Teaching”. In: *Meta: Journal des traducteurs* 38.3 (1993), pp. 565–567. ISSN: 0026-0452. DOI: 10.7202/001902ar.

Balboni, Paolo E. *Didattica dell'italiano come lingua seconda e straniera*. Torino: Loescher/Bonacci, 2014.

– *Le sfide di Babele: insegnare le lingue nelle società complesse*. Nuova ed. Torino: UTET Università, 2008. ISBN: 978-88-02-07952-9.

Bangor, Aaron, Philip T. Kortum e James T. Miller. “An Empirical Evaluation of the System Usability Scale”. In: *International Journal of Human–Computer Interaction* 24.6 (2008), pp. 574–594. DOI: 10.1080/10447310802205776.

Barsalou, Lawrence W. “Grounded Cognition”. In: *Annual Review of Psychology* 59 (2008), pp. 617–645. ISSN: 0066-4308, 1545-2085. DOI: 10.1146/annurev.psych.59.103006.093639.

Çakır, Nilay. “Virtual Reality Games in Foreign Language Education: A Systematic Review”. In: *Language Education and Technology (LET Journal)* 4.1 (2024), pp. 13–35.

Canale, Michael. “From Communicative Competence to Communicative Language Pedagogy”. In: *Language and Communication*. A cura di Jack C. Richards e Richard W. Schmidt. London: Longman, 1983, pp. 2–27.

Canale, Michael e Merrill Swain. “Theoretical Bases of Communicative Approaches to Second Language Teaching and Testing”. In: *Applied Linguistics* 1.1 (1980), pp. 1–47.

Chini, Marina. “Elementi utili per una didattica dell'italiano L2 alla luce della ricerca acquisizionale”. In: *Italiano LinguaDue* 2 (2016), pp. 1–18.

Clark, James M. e Allan Paivio. “Dual Coding Theory and Education”. In: *Educational Psychology Review* 3.3 (1991), pp. 149–210. ISSN: 1040-726X, 1573-336X. DOI: 10.1007/BF01320076.

Cornillie, Frederik, Steven L. Thorne e Piet Desmet. “Digital Games for Language Learning: From Hype to Insight?” In: *ReCALL* 24.3 (2012), pp. 243–256. ISSN: 0958-3440, 1474-0109. DOI: 10.1017/S0958344012000134.

De Paula Forero Pataquiva, Francisco e Blanka Klimova. “Systematic Review of Virtual Reality in the Acquisition of Second Language”. In: *International Journal of Emerging Technologies in Learning (iJET)* 17.15 (2022), pp. 43–53. ISSN: 1863-0383. DOI: 10.3991/ijet.v17i15.31781.

Deterding, Sebastian. “Gamification: Designing for Motivation”. In: *Interactions* 19.4 (2012), pp. 14–17. ISSN: 1072-5520. DOI: 10.1145/2212877.2212883.

Ek, Jan Ate van e John Leslie Melville Trim. *Waystage 1990*. Cambridge: Cambridge University Press, 1998. ISBN: 978-0-521-56707-7.

Ellis, Rod. *Task-Based Language Learning and Teaching*. Oxford: Oxford University Press, 2003. ISBN: 978-0-19-442160-7.

Gee, James Paul. *Situated Language and Learning: A Critique of Traditional Schooling*. New York: Routledge, 2004. ISBN: 978-0-415-31775-5.

– *What Video Games Have to Teach Us About Learning and Literacy*. New York: Palgrave Macmillan, 2004. ISBN: 978-1-4039-6538-7.

Glenberg, Arthur M. e Michael P. Kaschak. “Grounding Language in Action”. In: *Psychonomic Bulletin & Review* 9.3 (2002), pp. 558–565.

Graham, Calbert e Nathan Roll. “Evaluating OpenAI’s Whisper ASR: Performance Analysis across Diverse Accents and Speaker Traits”. In: *JASA Express Letters* 4.2 (2024), p. 025206. ISSN: 2691-1191. DOI: 10.1121/10.0024876.

Horwitz, Elaine K., Michael B. Horwitz e Joann Cope. “Foreign Language Classroom Anxiety”. In: *The Modern Language Journal* 70.2 (1986), pp. 125–132. ISSN: 0026-7902, 1540-4781. DOI: 10.1111/j.1540-4781.1986.tb05256.x.

Immerse. *Immerse: Build Real Language Fluency*. 2026. URL: https://www.meta.com/experiences/immerse-build-real-language-fluency/3181432891940455/ (visitato il giorno 28/07/2026).

ImmerseMe. *ImmerseMe*. 2026. URL: https://immerseme.co/ (visitato il giorno 28/07/2026).

Kaplan-Rakowski, Regina e Alice Gruber. “Low-Immersion versus High-Immersion Virtual Reality: Definitions, Classification, and Examples with a Foreign Language Focus”. In: *Proceedings of the Innovation in Language Learning International Conference 2019*. Firenze: Pixel, 2019, pp. 552–556.

– “The Impact of High-Immersion Virtual Reality on Foreign Language Anxiety”. In: *Smart Learning Environments* 10.1 (2023), p. 46. ISSN: 2196-7091. DOI: 10.1186/s40561-023-00263-9.

Knowles, Malcolm S. *The Modern Practice of Adult Education: From Pedagogy to Andragogy*. New York: The Adult Education Company, 1980.

Krashen, Stephen D. *The Input Hypothesis: Issues and Implications*. London: Longman, 1992. ISBN: 978-0-582-55379-7.

Language Lab VR. *Language Lab VR*. 2026. URL: https://www.meta.com/experiences/language-lab/3802077353226534/ (visitato il giorno 28/07/2026).

Lave, Jean e Etienne Wenger. *Situated Learning: Legitimate Peripheral Participation*. Cambridge: Cambridge University Press, 1991. ISBN: 978-0-521-42374-8.

Legault, Jennifer et al. “Immersive Virtual Reality as an Effective Tool for Second Language Vocabulary Learning”. In: *Languages* 4.1 (2019), p. 13. ISSN: 2226-471X. DOI: 10.3390/languages4010013.

Leto, Fabrizio. "Costruire e analizzare un corpus di interlingua all'interno di un corso di italiano LS a studenti sinofoni". In: *Italiano LinguaDue* 15.2 (2023), pp. 84–103. ISSN: 2037-3597. DOI: 10.54103/2037-3597/21940.

Lewis, Michael. *The Lexical Approach: The State of ELT and a Way Forward*. Hove: Language Teaching Publications, 1993.

Long, Michael H. "A Role for Instruction in Second Language Acquisition: Task-Based Language Teaching". In: *Modelling and Assessing Second Language Acquisition*. A cura di Kenneth Hyltenstam e Manfred Pienemann. Clevedon: Multilingual Matters, 1985, pp. 77–99.

– "Native Speaker/Non-Native Speaker Conversation in the Second Language Classroom". In: *Working Papers in ESL* 2.1 (1983), pp. 99–103.

Lyster, Roy e Leila Ranta. "Corrective Feedback and Learner Uptake: Negotiation of Form in Communicative Classrooms". In: *Studies in Second Language Acquisition* 19.1 (1997), pp. 37–66. ISSN: 0272-2631, 1470-1545. DOI: 10.1017/S0272263197001034.

Macedonia, M., A. E. Lehner e C. Repetto. "Positive Effects of Grasping Virtual Objects on Memory for Novel Words in a Second Language". In: *Scientific Reports* 10.1 (2020), p. 10760. ISSN: 2045-2322. DOI: 10.1038/s41598-020-67539-9.

Metcalfe, Janet. "Learning from Errors". In: *Annual Review of Psychology* 68 (2017), pp. 465–489. ISSN: 0066-4308, 1545-2085. DOI: 10.1146/annurev-psych-010416-044022.

Mondly. *Mondly VR*. 2026. URL: https://www.meta.com/experiences/mondly-learn-languages-in-vr/4214902388537196/ (visitato il giorno 28/07/2026).

Nation, Paul e David Crabbe. "A Survival Language Learning Syllabus for Foreign Travel". In: *System* 19.3 (1991), pp. 191–201.

Nhung, Nguyen Thi Phuong. "Noticing Hypothesis in Second Language Acquisition". In: *IOSR Journal of Humanities and Social Science* 25.6 (2020), pp. 26–30.

Noun Town. *Noun Town*. 2026. URL: https://noun.town/ (visitato il giorno 28/07/2026).

Nunan, David. *Task-Based Language Teaching*. Cambridge: Cambridge University Press, 2004. ISBN: 978-0-521-01273-6.

Peterson, Mark. "Computerized Games and Simulations in Computer-Assisted Language Learning: A Meta-Analysis of Research". In: *Simulation & Gaming* 41.1 (2010), pp. 72–93. ISSN: 1046-8781, 1552-826X. DOI: 10.1177/1046878109355684.

Qin, Jie e Lei Lei. "Research Trends in Task-Based Language Teaching: A Bibliometric Analysis from 1985 to 2020". In: *Studies in Second Language Learning and Teaching* 12.3 (2022), pp. 381–404. ISSN: 2084-1965. DOI: 10.14746/ssllt.2022.12.3.3.

Rodriguez, Michael C., Ryan J. Kettler e Stephen N. Elliott. "Distractor Functioning in Modified Items for Test Accessibility". In: *Sage Open* 4.4 (2014). ISSN: 2158-2440. DOI: 10.1177/2158244014553586.

Schmidt, Richard. "The Role of Consciousness in Second Language Learning". In: *Applied Linguistics* 11.2 (1990), pp. 129–158.

Shehadeh, Ali. "Output Hypothesis". In: *International Encyclopedia of Language and Linguistics*. Elsevier, 2026, pp. 819–824. DOI: 10.1016/B978-0-323-95504-1.01159-5.

– "Task-Based Language Teaching". In: *Reference Collection in Social Sciences*. Elsevier, 2025, pp. 1–12. DOI: 10.1016/B978-0-323-95504-1.01011-5.

Shortt, Mitchell et al. “Gamification in Mobile-Assisted Language Learning: A Systematic Review of Duolingo Literature from Public Release of 2012 to Early 2020”. In: *Computer Assisted Language Learning* (2021). Pubblicato online il 2 giugno 2021; poi in vol. 36, n. 3 (2023), pp. 517–554. DOI: 10.1080/09588221.2021.1933540.

Siemens, George. “Learning Analytics and Open, Flexible, and Distance Learning”. In: *Distance Education* 40.3 (2019), pp. 414–418. ISSN: 0158-7919. DOI: 10.1080/01587919.2019.1656153.

Skehan, Peter. *Individual Differences in Second Language Learning*. London: Edward Arnold, 1989.

Spinelli, Barbara, Fiorenza Parizzi e Joseph Sheils. *Profilo della lingua italiana: livelli di riferimento del QCER A1, A2, B1, B2*. Firenze: La Nuova Italia, 2010. ISBN: 978-88-221-6327-1.

Sun, Lu e Xiacheng Song. “The Effectiveness of Virtual Reality for K-12 Foreign Language Learning: a Systematic Review of Recent Randomized Controlled Trials”. In: *Frontiers in Psychology* 16 (2026), p. 1714481. ISSN: 1664-1078. DOI: 10.3389/fpsyg.2025.1714481.

Sweller, John. “Cognitive Load Theory”. In: *Psychology of Learning and Motivation*. Vol. 55. Elsevier, 2011, pp. 37–76. ISBN: 978-0-12-387691-1. DOI: 10.1016/B978-0-12-387691-1.00002-8.

The Design-Based Research Collective. “Design-Based Research: An Emerging Paradigm for Educational Inquiry”. In: *Educational Researcher* 32.1 (2003), pp. 5–8. ISSN: 0013-189X. DOI: 10.3102/0013189X032001005.

Vygotskij, Lev Semenovič. *Mind in Society: The Development of Higher Psychological Processes*. A cura di Michael Cole et al. Cambridge, MA: Harvard University Press, 1978.

Wilkins, David. *Notional Syllabuses*. Oxford: Oxford University Press, 1976.

Wray, Alison. *Formulaic Language and the Lexicon*. Cambridge: Cambridge University Press, 2002. ISBN: 978-0-521-77309-6.